Machine Learning

Supervised Learning, Regression, Classification, Neural Networks, Unsupervised Learning

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 12 tháng 10 năm 2026

Mục lục

1 Tổng quan **3**
- 1.1 Cách đọc tài liệu . . . 3
- 1.2 Nội dung . . . 3

2 Supervised Machine Learning: Regression và Classification **4**
- 2.1 Giới thiệu về Machine Learning . . . 4
  - 2.1.1 Học máy là gì . . . 4
  - 2.1.2 Học có giám sát . . . 5
  - 2.1.3 Hai bài toán: hồi quy và phân loại . . . 6
  - 2.1.4 Học không giám sát (nhìn nhanh) . . . 6
- 2.2 Mô hình hồi quy tuyến tính . . . 6
  - 2.2.1 Biểu diễn mô hình . . . 6
  - 2.2.2 Hàm chi phí: đo mức độ sai . . . 7
  - 2.2.3 Gradient descent: tự tìm đáy . . . 7
- 2.3 Hồi quy tuyến tính với nhiều đặc trưng . . . 9
  - 2.3.1 Nhiều đặc trưng . . . 9
  - 2.3.2 Vector hoá: viết gọn và chạy nhanh . . . 10
  - 2.3.3 Gradient descent cho nhiều đặc trưng . . . 10
  - 2.3.4 Chuẩn hoá đặc trưng . . . 11
  - 2.3.5 Kiểm tra hội tụ và chọn tốc độ học . . . 11
  - 2.3.6 Kỹ thuật đặc trưng và hồi quy đa thức . . . 12
  - 2.3.7 Thực hành: hồi quy tuyến tính với scikit-learn . . . 12
- 2.4 Phân loại với hồi quy logistic . . . 13
  - 2.4.1 Vì sao hồi quy tuyến tính không hợp cho phân loại . . . 13
  - 2.4.2 Hàm sigmoid và mô hình logistic . . . 13
  - 2.4.3 Ranh giới quyết định . . . 14
  - 2.4.4 Hàm chi phí cho hồi quy logistic . . . 14
  - 2.4.5 Gradient descent cho hồi quy logistic . . . 15
  - 2.4.6 Thực hành: hồi quy logistic với scikit-learn . . . 15
- 2.5 Quá khớp và chính quy hoá . . . 15
  - 2.5.1 Quá khớp và thiếu khớp . . . 15
  - 2.5.2 Ba cách giảm quá khớp . . . 16
  - 2.5.3 Chính quy hoá hoạt động thế nào . . . 16
  - 2.5.4 Chính quy hoá trong gradient descent . . . 17
- 2.6 Tổng kết chương . . . 17

3 Advanced Learning Algorithms **19**
- 3.1 Mạng nơ-ron: suy luận . . . 19
  - 3.1.1 Từ nơ-ron sinh học tới nơ-ron nhân tạo . . . 19
  - 3.1.2 Nơ-ron và lớp . . . 19
  - 3.1.3 Lan truyền xuôi: cách mạng đưa ra dự đoán . . . 20

3.1.4 Suy luận với TensorFlow/Keras . . . 20
3.1.5 Mạng nơ-ron chỉ là các phép tính trên danh sách số . . . 21
3.1.6 Thực hành tuần 1: nhận diện chữ số viết tay . . . 22
3.2 Huấn luyện mạng nơ-ron . . . 22
3.2.1 Ba bước huấn luyện với TensorFlow/Keras . . . 22
3.2.2 Lan truyền ngược: trực giác, không công thức . . . 24
3.2.3 Hàm kích hoạt: chọn thế nào . . . 24
3.2.4 Phân loại đa lớp và softmax . . . 25
3.2.5 Adam: tự điều chỉnh tốc độ học . . . 26
3.2.6 Thực hành tuần 2: nhận diện chữ số 0–9 . . . 26
3.3 Lời khuyên thực tiễn khi áp dụng Machine Learning . . . 26
3.3.1 Chia dữ liệu: train / cross-validation / test . . . 27
3.3.2 Bias và variance: la bàn chẩn đoán . . . 28
3.3.3 Mức chuẩn và tác động của chính quy hoá . . . 29
3.3.4 Đường cong học . . . 29
3.3.5 Quy trình phát triển lặp . . . 29
3.3.6 Dữ liệu lệch: precision và recall . . . 30
3.3.7 Thực hành tuần 3 . . . 31
3.4 Cây quyết định . . . 31
3.4.1 Cây quyết định là gì . . . 31
3.4.2 Chọn câu hỏi để chia: độ tinh khiết và information gain . . . 32
3.4.3 Xây cây: chia đệ quy . . . 32
3.4.4 Đặc trưng danh mục và đặc trưng liên tục . . . 34
3.4.5 Cây hồi quy . . . 34
3.4.6 Ensemble: nhiều cây mạnh hơn một cây . . . 34
3.4.7 Thực hành tuần 4 . . . 35
3.4.8 Cây quyết định so với mạng nơ-ron . . . 35
3.5 Tổng kết chương . . . 36

4 Unsupervised Learning, Recommenders, Reinforcement Learning 38
4.1 Học không giám sát . . . 38
4.1.1 Học không giám sát là gì . . . 38
4.1.2 Phân cụm K-means: trực giác . . . 39
4.1.3 K-means với scikit-learn . . . 41
4.1.4 Phát hiện bất thường: trực giác . . . 42
4.1.5 Phát hiện bất thường trong thực hành . . . 44
4.1.6 Phát hiện bất thường hay học có giám sát? . . . 44
4.2 Hệ gợi ý . . . 45
4.2.1 Lọc cộng tác: trực giác . . . 46
4.2.2 Lọc cộng tác với TensorFlow . . . 47
4.2.3 Lọc dựa trên nội dung: trực giác . . . 48
4.2.4 Lọc nội dung bằng học sâu . . . 48
4.2.5 Mở rộng quy mô: truy hồi và xếp hạng . . . 49
4.2.6 Giảm chiều dữ liệu với PCA . . . 50
4.3 Học tăng cường . . . 51
4.3.1 Khung bài toán: trạng thái, hành động, phần thưởng . . . 51
4.3.2 Hàm giá trị hành động và phương trình Bellman . . . 53
4.3.3 Trạng thái liên tục và tàu đổ bộ Mặt Trăng . . . 54
4.3.4 Deep Q-Network: học hàm Q bằng mạng nơ-ron . . . 55
4.3.5 Học tăng cường trong bức tranh lớn . . . 58

Chương 1

Tổng quan

Tài liệu này là bộ ghi chú học tập về học máy (Machine Learning), bám theo ba khoá của *Machine Learning Specialization*. Mỗi chương trình bày một chủ đề theo hướng **ý tưởng trước, ứng dụng sau**: hiểu vì sao một kỹ thuật ra đời, nó giải quyết vấn đề gì, rồi dùng nó qua thư viện như thế nào.

1.1 Cách đọc tài liệu

- **Lý thuyết** – giải thích bằng ví dụ và sơ đồ, hạn chế công thức.
- **Code** – ví dụ dùng Python với NumPy, scikit-learn và TensorFlow/Keras. Code đặt trong khung, có chú thích tiếng Việt.
- **Áp dụng** – checklist, bảng chọn thuật toán, lỗi thường gặp khi đưa vào dự án.

1.2 Nội dung

Tài liệu gồm ba chương, đi từ các thuật toán học có giám sát cơ bản đến học không giám sát, hệ gợi ý và học tăng cường. Các chương đang được biên soạn dần.

Chương	Nội dung chính
Chương 1: Tổng quan	mục đích, cách đọc và cấu trúc tài liệu (chương này).
Chương 2: Supervised Machine Learning: Regression và Classification	hồi quy tuyến tính, gradient descent, hồi quy logistic cho phân loại và quá khớp/chính quy hoá.
Chương 3: Advanced Learning Algorithms	mạng nơ-ron, huấn luyện với TensorFlow, lời khuyên thực tiễn và cây quyết định.
Chương 4: Unsupervised Learning, Recommenders, Reinforcement Learning	phân cụm, phát hiện bất thường, hệ gợi ý và học tăng cường.

Bảng 1.1: Cấu trúc tài liệu

Chương 2

Supervised Machine Learning: Regression và Classification

Chương này đi qua nền tảng của học máy có giám sát: từ câu hỏi "học máy là gì" đến hai mô hình được dùng nhiều nhất trong thực tế là **hồi quy tuyến tính** (dự đoán một con số) và **hồi quy logistic** (dự đoán một nhóm). Cách tiếp cận ở đây là *ý tưởng trước, công cụ sau*: hiểu trực giác vì sao một kỹ thuật ra đời, rồi dùng nó qua thư viện như scikit-learn thay vì tự viết lại thuật toán từ đầu.

Tài liệu cố ý diễn giải mọi công thức bằng lời và sơ đồ. Thay vì ký hiệu toán học, ta gọi tên các đại lượng theo đúng ý nghĩa của chúng (ví dụ "trọng số", "độ chệch", "sai số", "chi phí"), để người đọc nắm được logic mà không bị rào cản ký hiệu.

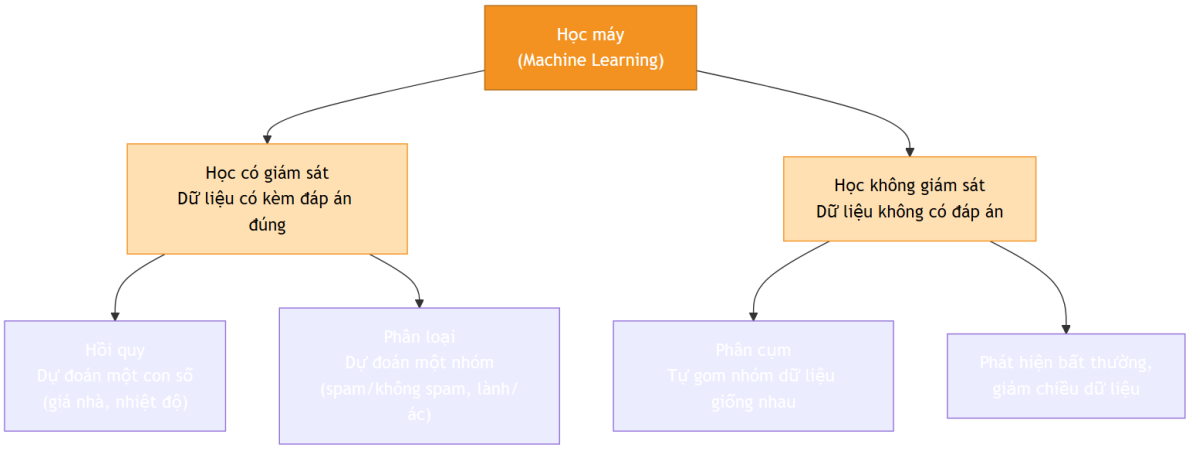
2.1 Giới thiệu về Machine Learning

2.1.1 Học máy là gì

Học máy là ngành nghiên cứu giúp máy tính **học từ dữ liệu** mà không cần được lập trình tường minh cho từng tình huống. Một định nghĩa kinh điển (Arthur Samuel, thập niên 1950) mô tả học máy là lĩnh vực cho máy tính khả năng học mà không cần được lập trình một cách rõ ràng. Samuel từng viết một chương trình chơi cờ đam; bản thân ông không phải kỳ thủ giỏi, nhưng ông cho máy tự chơi với chính nó hàng chục nghìn ván. Qua đó máy dần nhận ra thế cờ nào dẫn tới thắng, thế nào dẫn tới thua, và cuối cùng chơi giỏi hơn cả người tạo ra nó. Bài học rút ra: càng cho thuật toán nhiều cơ hội học, nó thường càng làm tốt.

Học máy đã len lỏi vào đời sống hằng ngày: xếp hạng trang web khi tìm kiếm, nhận diện bạn bè trong ảnh, gợi ý phim, chuyển giọng nói thành văn bản, lọc thư rác, tối ưu điện gió, hỗ trợ chẩn đoán y tế, kiểm tra lỗi sản phẩm trên dây chuyền. Lý do học máy phổ biến là vì nhiều bài toán thú vị (nhận dạng giọng nói, thị giác máy tính, xe tự lái) gần như không thể viết chương trình tường minh để giải; cách duy nhất là để máy tự học.

Hai nhóm thuật toán học máy chính là **học có giám sát** (supervised learning) và **học không giám sát** (unsupervised learning). Trong đó học có giám sát là nhóm được dùng nhiều nhất và tiến bộ nhanh nhất trong các ứng dụng thực tế. Chương này tập trung hoàn toàn vào học có giám sát.



Hình 2.1: Bản đồ các nhóm thuật toán học máy. Chương này đi theo nhánh học có giám sát, gồm hồi quy và phân loại.

2.1.2 Học có giám sát

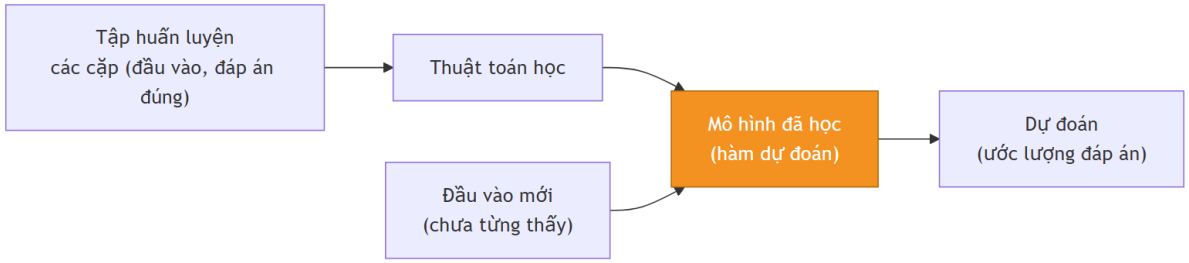
Học có giám sát là nhóm thuật toán học cách ánh xạ từ **đầu vào** sang **đầu ra**. Điểm mấu chốt: ta cho thuật toán các ví dụ *kèm đáp án đúng*. Nghĩa là với mỗi đầu vào, ta cung cấp luôn nhãn đúng mà ta mong muốn mô hình trả ra. Nhờ xem đủ nhiều cặp (đầu vào, đáp án đúng), mô hình học được quy luật, rồi khi gặp một đầu vào mới chưa từng thấy, nó đưa ra dự đoán hợp lý cho đầu ra.

Nhiều ứng dụng giá trị nhất hiện nay thuộc nhóm này:

Đầu vào	**Đầu ra**	**Ứng dụng**
Email	Spam / không spam	Lọc thư rác
Đoạn âm thanh	Văn bản	Nhận dạng giọng nói
Câu tiếng Anh	Câu tiếng Việt	Dịch máy
Thông tin quảng cáo và người dùng	Có nhấp / không nhấp	Quảng cáo trực tuyến
Ảnh và dữ liệu cảm biến	Vị trí các xe khác	Xe tự lái
Ảnh sản phẩm	Có lỗi / không lỗi	Kiểm tra ngoại quan

Bảng 2.1: Một số ứng dụng tiêu biểu của học có giám sát

Quy trình chung của học có giám sát (Hình 2.2): đưa tập huấn luyện gồm các cặp (đầu vào, đáp án đúng) cho thuật toán học, thuật toán sinh ra một mô hình (một hàm dự đoán); sau đó đưa đầu vào mới vào mô hình để nhận dự đoán.



Hình 2.2: Luồng xử lý của học có giám sát: huấn luyện từ dữ liệu có nhãn, rồi dự đoán trên đầu vào mới.

2.1.3 Hai bài toán: hồi quy và phân loại

Học có giám sát có hai dạng bài toán lớn:

- **Hồi quy (regression)**: dự đoán một *con số* trong vô số giá trị có thể. Ví dụ dự đoán giá nhà theo diện tích: giá có thể là 150 nghìn, 170 nghìn, 183 nghìn đô la hay bất kỳ số nào ở giữa. Ta có thể khớp một đường thẳng hoặc một đường cong vào dữ liệu rồi đọc ra dự đoán.
- **Phân loại (classification)**: dự đoán một *nhóm* (lớp) trong một tập hữu hạn, nhỏ các lựa chọn. Ví dụ chẩn đoán khối u là lành tính hay ác tính (hai nhóm), hay nhận diện ảnh là mèo hay chó. Nhóm có thể biểu diễn bằng số (0, 1, 2) nhưng điểm khác biệt là chỉ có một số ít giá trị rời rạc, không phải mọi con số ở giữa.

Khác biệt cốt lõi: hồi quy chọn trong *vô hạn* con số; phân loại chọn trong một *tập nhỏ hữu hạn* các nhóm. Trong phân loại, ta cũng có thể dùng nhiều hơn một đặc trưng đầu vào, chẳng hạn vừa dùng kích thước khối u vừa dùng tuổi bệnh nhân; khi đó thuật toán tìm một *ranh giới* ngăn cách các nhóm với nhau.

2.1.4 Học không giám sát (nhìn nhanh)

Ở học không giám sát, dữ liệu chỉ có đầu vào mà *không* có nhãn đáp án. Nhiệm vụ là tự tìm ra cấu trúc hoặc quy luật thú vị trong dữ liệu. Dạng phổ biến nhất là **phân cụm**: tự gom các điểm dữ liệu giống nhau vào cùng nhóm. Ví dụ Google News gom các bài báo cùng chủ đề mỗi ngày; phân nhóm khách hàng theo hành vi; gom các cá thể theo dữ liệu gen. Ngoài phân cụm còn có phát hiện bất thường (phát hiện giao dịch gian lận) và giảm chiều dữ liệu (nén dữ liệu mà mất ít thông tin nhất). Các chủ đề này được bàn kỹ ở chương sau; ở đây chỉ cần nhớ điểm khác biệt: học có giám sát có đáp án đúng để học theo, học không giám sát thì không.

2.2 Mô hình hồi quy tuyến tính

2.2.1 Biểu diễn mô hình

Hồi quy tuyến tính nghĩa là **khớp một đường thẳng** vào dữ liệu. Đây có lẽ là thuật toán học máy được dùng rộng rãi nhất. Lấy ví dụ kinh điển: dự đoán giá nhà theo diện tích. Ta có một tập dữ liệu các căn nhà ở Portland với diện tích (tính bằng foot vuông) và giá bán gần nhất (tính bằng nghìn đô la). Khi vẽ lên đồ thị, mỗi căn nhà là một điểm. Một người môi giới có thể dùng mô hình này để ước lượng giá cho một căn nhà cần bán.

Ta thống nhất một số thuật ngữ dùng xuyên suốt:

Thuật ngữ	Ý nghĩa
Tập huấn luyện (training set)	Dữ liệu dùng để huấn luyện mô hình.
Đặc trưng đầu vào (feature)	Thông tin dùng để dự đoán, ví dụ diện tích nhà.
Mục tiêu (target)	Giá trị đúng cần dự đoán, ví dụ giá nhà thật.
Số mẫu huấn luyện	Số dòng trong tập huấn luyện (ví dụ 47 căn nhà).
Một mẫu huấn luyện	Một cặp (đặc trưng, mục tiêu), tương ứng một dòng dữ liệu.

Bảng 2.2: Thuật ngữ cơ bản của học có giám sát

Quá trình huấn luyện: đưa tập huấn luyện (gồm cả đặc trưng và mục tiêu) cho thuật toán học; thuật toán sinh ra một **hàm dự đoán** (gọi là mô hình). Hàm này nhận một đầu vào mới và trả ra một *ước lượng* cho mục tiêu. Ta phân biệt rõ: *mục tiêu* là giá trị đúng có thật trong tập huấn luyện, còn *dự đoán* chỉ là ước lượng của mô hình, có thể đúng hoặc chưa đúng.

Với hồi quy tuyến tính một biến, hàm dự đoán là một đường thẳng, được mô tả bằng hai tham số: **độ dốc** (còn gọi là *trọng số*) cho biết đường thẳng nghiêng bao nhiêu, và **độ chệch** (còn gọi là điểm cắt trục tung) cho biết đường thẳng cắt trục giá tại đâu khi diện tích bằng 0. Thay đổi hai tham số này sẽ cho những đường thẳng khác nhau. Tên gọi đầy đủ của mô hình một biến này là *hồi quy tuyến tính đơn biến* (univariate linear regression) — "đơn biến" nghĩa là chỉ có một đặc trưng đầu vào.

Vì sao chọn đường thẳng? Vì nó đơn giản và dễ làm việc, là nền tảng để sau này tiến tới các mô hình phức tạp, phi tuyến hơn.

2.2.2 Hàm chi phí: đo mức độ sai

Để chọn được đường thẳng khớp dữ liệu tốt, ta cần một thước đo mức độ "khớp tốt" hay "khớp dở". Thước đo đó là **hàm chi phí** (cost function) — một trong những ý tưởng quan trọng và phổ quát nhất của học máy.

Cách xây dựng hàm chi phí, diễn giải bằng lời:

1. Với mỗi căn nhà, lấy **dự đoán** của đường thẳng trừ đi **giá thật**. Hiệu này gọi là *sai số* — cho biết dự đoán lệch bao xa so với thực tế.
2. **Bình phương** sai số đó (để sai lệch dương hay âm đều thành số dương, và để phạt nặng hơn các lệch lớn).
3. **Cộng** bình phương sai số của tất cả các căn nhà lại.
4. **Lấy trung bình** (chia cho số mẫu) để chi phí không tự phình to khi tập dữ liệu lớn hơn. Theo quy ước, người ta còn chia thêm cho 2 nữa, chỉ nhằm làm gọn các phép tính về sau; chia hay không chia cho 2 đều không đổi bản chất.

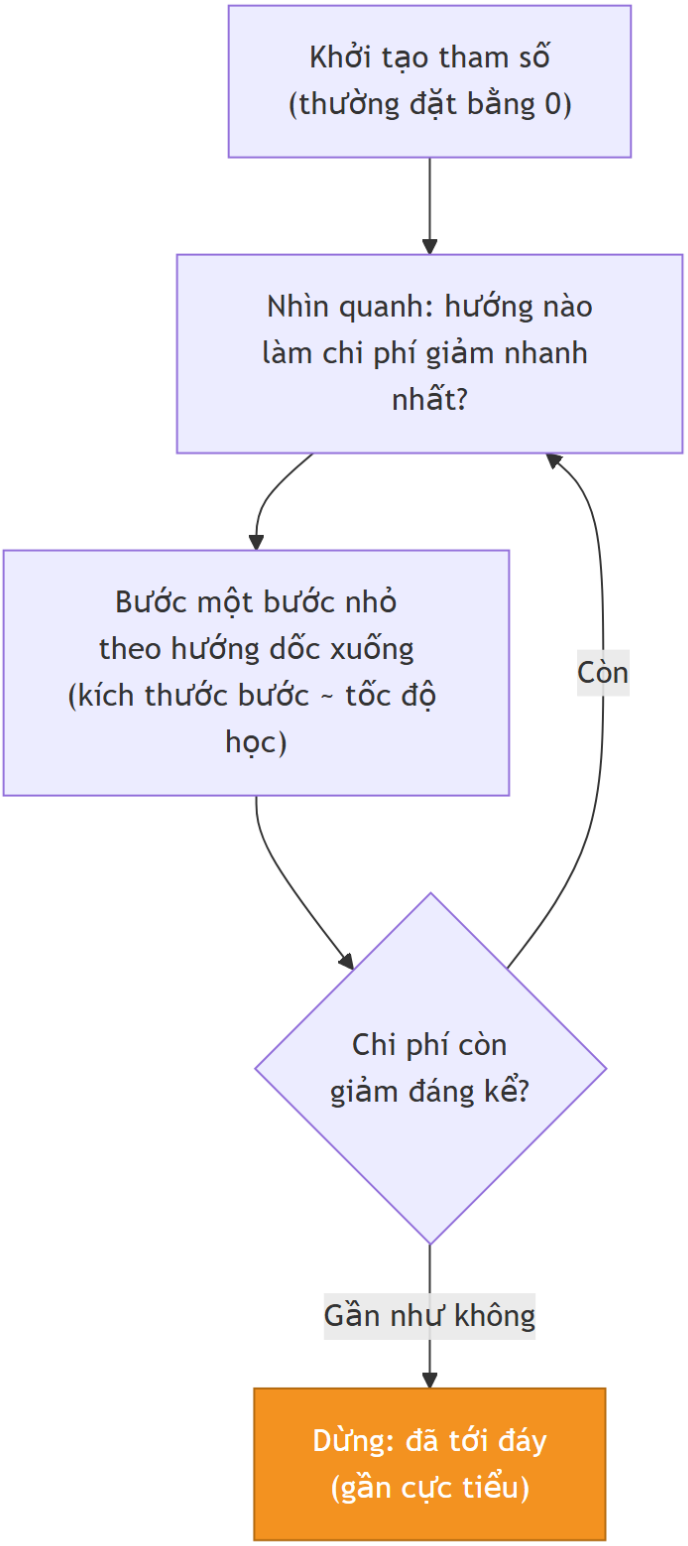
Kết quả gọi là **hàm chi phí sai số bình phương** (squared error cost function) — loại hàm chi phí dùng phổ biến nhất cho hồi quy. Trực giác: chi phí nhỏ nghĩa là đường thẳng đi sát các điểm dữ liệu; chi phí lớn nghĩa là đường thẳng lệch xa dữ liệu. Mục tiêu huấn luyện là tìm trọng số và độ chệch sao cho chi phí *nhỏ nhất có thể*.

Hình dung trực quan: nếu chỉ xét một tham số (trọng số), đồ thị chi phí theo tham số đó có dạng cái bát hình chữ U, đáy bát là điểm chi phí nhỏ nhất. Nếu xét cả hai tham số (trọng số và độ chệch), đồ thị chi phí là một mặt cong hình cái bát trong không gian ba chiều; nhìn từ trên xuống nó trông như các đường đồng mức hình bầu dục đồng tâm (giống bản đồ địa hình), tâm của các đường đồng mức chính là đáy bát — nơi chi phí nhỏ nhất.

2.2.3 Gradient descent: tự tìm đáy

Việc dò bằng mắt trên đường đồng mức để tìm tham số tốt nhất là không khả thi, nhất là với mô hình phức tạp. Ta cần một thuật toán tự động làm việc đó: **gradient descent** (hạ gradient). Đây là một trong những thuật toán quan trọng nhất của học máy, dùng để huấn luyện không chỉ hồi quy tuyến tính mà cả những mô hình lớn và phức tạp nhất trong AI.

Trực giác bằng hình ảnh "xuống đồi": tưởng tượng mặt chi phí là một vùng đồi núi. Bạn đang đứng ở một điểm trên đồi và muốn đi xuống thung lũng nhanh nhất. Ở mỗi bước, bạn nhìn quanh 360 độ và tự hỏi: nếu bước một bước nhỏ thì hướng nào xuống dốc nhanh nhất? Rồi bạn bước theo hướng đó. Lặp lại nhiều lần cho tới khi tới đáy. Đó chính là gradient descent.



Hình 2.3: Vòng lặp của gradient descent: liên tục chọn hướng dốc xuống và bước một bước nhỏ, cho tới khi chi phí gần như không giảm thêm.

Cơ chế cập nhật, diễn giải bằng lời: ở mỗi bước, mỗi tham số được điều chỉnh bằng cách lấy giá trị hiện tại trừ đi một lượng nhỏ. Lượng nhỏ đó bằng **tốc độ học** nhân với **độ dốc** của hàm chi phí tại điểm hiện tại. Độ dốc cho biết nên đi về hướng nào; tốc độ học (một số dương nhỏ) quyết định bước đi lớn

hay nhỏ. Một chi tiết quan trọng: phải *cập nhật đồng thời* tất cả tham số — tính toàn bộ phần điều chỉnh dựa trên giá trị cũ, rồi mới ghi đè sang giá trị mới cùng lúc.

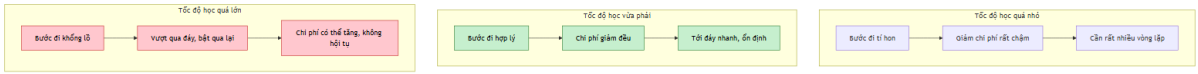
Vì sao công thức cập nhật có lý? Xét trường hợp một tham số:

- Nếu đang ở phía mà đồ thị dốc lên về bên phải (độ dốc dương), phép trừ sẽ kéo tham số giảm, tức đi sang trái, về phía đáy.
- Nếu đang ở phía mà đồ thị dốc xuống về bên phải (độ dốc âm), trừ đi một số âm hoá ra là cộng, kéo tham số tăng, tức đi sang phải, cũng về phía đáy.

Dù xuất phát ở đâu, các bước đều đưa ta tiến dần về điểm thấp nhất.

Tốc độ học quan trọng thế nào. Tốc độ học ảnh hưởng rất lớn tới hiệu quả:

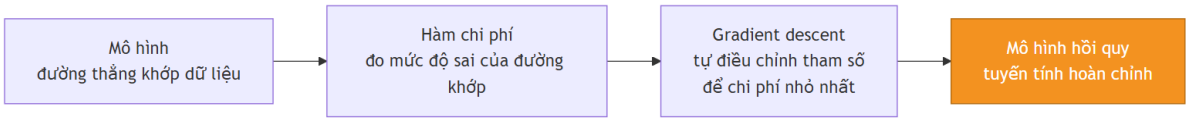
- **Quá nhỏ**: mỗi bước là một bước tí hon, chi phí giảm nhưng cực chậm, cần rất nhiều vòng lặp.
- **Quá lớn**: bước đi khổng lồ có thể vượt qua đáy, bật qua lại, thậm chí làm chi phí tăng lên và *không hội tụ*.
- **Vừa phải**: chi phí giảm đều và tới đáy nhanh.



Hình 2.4: Ảnh hưởng của tốc độ học tới gradient descent.

Một điểm tinh tế: khi càng tới gần đáy, độ dốc tự động nhỏ dần, nên các bước đi cũng tự nhỏ lại ngay cả khi tốc độ học giữ cố định. Nhờ vậy gradient descent có thể dừng đúng ở cực tiểu mà không cần giảm tốc độ học thủ công. Nếu đã ở đúng cực tiểu, độ dốc bằng không nên tham số không đổi nữa.

Gộp lại thành thuật toán hồi quy tuyến tính. Khi áp gradient descent lên hàm chi phí sai số bình phương của hồi quy tuyến tính, ta có thuật toán huấn luyện hoàn chỉnh (Hình 2.5). Một tin tốt: hàm chi phí sai số bình phương của hồi quy tuyến tính luôn có dạng cái bát (ta gọi là *hàm lồi*), chỉ có *một* đáy duy nhất. Do đó, với tốc độ học phù hợp, gradient descent luôn hội tụ về cực tiểu toàn cục, không bị kẹt ở các cực tiểu cục bộ.



Hình 2.5: Ghép mô hình, hàm chi phí và gradient descent thành thuật toán hồi quy tuyến tính.

Phiên bản gradient descent xét *toàn bộ* tập huấn luyện ở mỗi bước cập nhật được gọi là **batch gradient descent** (hạ gradient theo lô). Có những biến thể khác chỉ xét một phần dữ liệu mỗi bước, nhưng với hồi quy tuyến tính ta dùng bản theo lô.

2.3 Hồi quy tuyến tính với nhiều đặc trưng

2.3.1 Nhiều đặc trưng

Trong thực tế, hiếm khi ta chỉ dự đoán giá nhà từ mỗi diện tích. Ta thường biết thêm số phòng ngủ, số tầng, tuổi của căn nhà… Mỗi thông tin như vậy là một **đặc trưng**. Có nhiều đặc trưng nghĩa là có nhiều thông tin hơn để dự đoán, thường cho kết quả tốt hơn.

Mô hình khi đó không còn là một đường thẳng theo một biến, mà là một tổ hợp: mỗi đặc trưng có một **trọng số** riêng, lấy từng đặc trưng nhân với trọng số của nó rồi cộng tất cả lại, sau đó cộng thêm **độ chệch**. Ta có thể diễn giải ý nghĩa các tham số rất tự nhiên. Ví dụ với giá nhà (nghìn đô la): độ chệch là "giá nền" của một căn nhà; trọng số của diện tích cho biết mỗi foot vuông tăng thêm thì giá tăng bao nhiêu; trọng số của số phòng ngủ cho biết mỗi phòng ngủ tăng thêm thì giá tăng bao nhiêu; trọng số có thể âm (ví dụ tuổi nhà càng cao giá càng giảm).

Mô hình nhiều đặc trưng này gọi là **hồi quy tuyến tính bội** (multiple linear regression), để phân biệt với bản đơn biến chỉ có một đặc trưng.

2.3.2 Vector hoá: viết gọn và chạy nhanh

Khi có nhiều đặc trưng, việc viết tay từng phép nhân (trọng số 1 nhân đặc trưng 1, cộng trọng số 2 nhân đặc trưng 2, …) vừa dài vừa chậm. **Vector hoá** (vectorization) là kỹ thuật gộp các đặc trưng thành một danh sách số (một *vector*) và các trọng số thành một danh sách số khác, rồi thực hiện toàn bộ phép "nhân từng cặp rồi cộng lại" bằng một thao tác duy nhất. Trong Python, thao tác đó là *tích vô hướng* (dot product) của thư viện NumPy.

Vector hoá mang lại hai lợi ích rõ rệt:

- **Code ngắn hơn**: thay vì vòng lặp dài hoặc chuỗi phép cộng, chỉ còn một dòng.
- **Chạy nhanh hơn nhiều**: NumPy tận dụng phần cứng xử lý song song (CPU, hoặc GPU). Thay vì tính tuần tự từng cặp một, máy tính nhân tất cả các cặp *cùng lúc*, rồi cộng dồn rất hiệu quả. Với dữ liệu lớn, khác biệt có thể là vài phút so với nhiều giờ.

Vector hoá cũng áp dụng cho bước cập nhật tham số trong gradient descent: thay vì cập nhật từng trọng số một trong vòng lặp, ta cập nhật toàn bộ danh sách trọng số cùng lúc trong một thao tác. Dưới đây là một ví dụ ngắn minh hoạ cú pháp NumPy (chỉ để làm quen, không phải để đo hiệu năng):

```
import numpy as np

# Danh sách trọng số và đặc trưng (mỗi cái là một vector 3 phần tử)
w = np.array([1.0, 2.5, -3.3])   # trọng số
x = np.array([10.0, 20.0, 30.0]) # đặc trưng
b = 4.0                          # độ chệch

# Dự đoán = tích vô hướng (nhân từng cặp rồi cộng) + độ chệch, tất cả trong một
↪  dòng
f = np.dot(w, x) + b
print(f)
```

2.3.3 Gradient descent cho nhiều đặc trưng

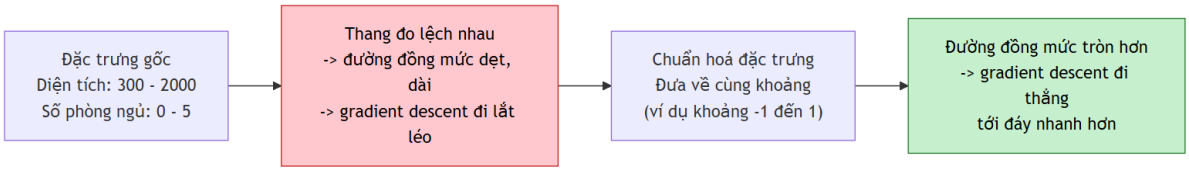
Ý tưởng gradient descent không đổi khi có nhiều đặc trưng: vẫn lặp lại việc điều chỉnh mỗi tham số một lượng nhỏ theo hướng làm chi phí giảm. Khác biệt duy nhất là giờ ta cập nhật một danh sách trọng số (mỗi đặc trưng một trọng số) thay vì chỉ một trọng số, cộng với độ chệch. Phần "sai số" trong mỗi bước vẫn là "dự đoán trừ mục tiêu" như trước.

Một lựa chọn khác: phương trình chuẩn. Ngoài gradient descent, riêng hồi quy tuyến tính còn có một cách giải trực tiếp gọi là **phương trình chuẩn** (normal equation): dùng đại số tuyến tính để tìm ngay trọng số và độ chệch mà không cần lặp. Nhược điểm: cách này chỉ dùng được cho hồi quy tuyến tính (không tổng quát cho các thuật toán khác như hồi quy logistic hay mạng nơ-ron), và chạy chậm khi số đặc trưng rất lớn. Thông thường ta không tự cài đặt nó; nhưng một số thư viện học máy có thể dùng nó ở "hậu trường" khi bạn gọi hồi quy tuyến tính.

2.3.4 Chuẩn hoá đặc trưng

Khi các đặc trưng có *thang đo* rất khác nhau, gradient descent có thể chạy chậm. Ví dụ diện tích nhà dao động từ 300 đến 2000 (số lớn), còn số phòng ngủ chỉ từ 0 đến 5 (số nhỏ). Hệ quả: một thay đổi nhỏ ở trọng số của diện tích đã tác động rất mạnh tới dự đoán, trong khi trọng số của số phòng ngủ phải đổi nhiều mới ảnh hưởng. Trên đồ thị đường đồng mức, điều này tạo ra các hình bầu dục dẹt và dài, khiến gradient descent đi lắt léo, bật qua lại rất lâu mới tới đáy.

Chuẩn hoá đặc trưng (feature scaling) đưa các đặc trưng về cùng một khoảng giá trị tương đương nhau. Khi đó đường đồng mức tròn hơn, gradient descent đi gần như thẳng tới đáy và nhanh hơn nhiều (Hình 2.6).



Hình 2.6: Vì sao cần chuẩn hoá đặc trưng và nó giúp gì cho gradient descent.

Ba cách chuẩn hoá thường gặp, diễn giải bằng lời:

- **Chia cho giá trị lớn nhất**: lấy mỗi giá trị chia cho giá trị lớn nhất của đặc trưng đó, đưa khoảng về quanh 0 đến 1.
- **Chuẩn hoá theo trung bình** (mean normalization): lấy mỗi giá trị trừ đi giá trị trung bình rồi chia cho độ rộng (lớn nhất trừ nhỏ nhất), đưa các đặc trưng về quanh 0, có cả giá trị âm và dương (thường trong khoảng âm một đến dương một).
- **Chuẩn hoá điểm Z** (z-score): lấy mỗi giá trị trừ trung bình rồi chia cho *độ lệch chuẩn* (một đại lượng đo mức phân tán của dữ liệu). Đây là cách scikit-learn dùng trong công cụ `StandardScaler`.

Nguyên tắc ngón tay cái: cố đưa mỗi đặc trưng về khoảng cỡ âm một đến dương một; các khoảng như âm ba đến dương ba, hay âm không-phẩy-ba đến dương không-phẩy-ba đều chấp nhận được. Chỉ cần chuẩn hoá khi một đặc trưng có thang đo quá lớn hoặc quá nhỏ so với phần còn lại. Gần như không có hại gì khi chuẩn hoá, nên khi phân vân thì cứ chuẩn hoá.

2.3.5 Kiểm tra hội tụ và chọn tốc độ học

Làm sao biết gradient descent có đang chạy đúng không? Cách hữu ích nhất là vẽ **đường cong học** (learning curve): trục ngang là số vòng lặp đã chạy, trục dọc là giá trị chi phí sau mỗi vòng. Nếu mọi thứ ổn, chi phí phải *giảm sau mỗi vòng lặp*. Khi đường cong phẳng dần ra, nghĩa là gradient descent đã gần như hội tụ. Số vòng lặp cần để hội tụ thay đổi rất nhiều giữa các bài toán (vài chục, vài nghìn, hay hàng trăm nghìn), nên nhìn đường cong là cách thực tế nhất.

Dấu hiệu bất ổn và cách xử lý:

- Nếu chi phí *tăng* sau một vòng, hoặc lúc tăng lúc giảm thất thường: thường do tốc độ học quá lớn, hoặc có lỗi trong code.
- Mẹo gỡ lỗi: đặt tốc độ học rất nhỏ. Nếu khi đó chi phí vẫn không giảm đều sau mỗi vòng, nhiều khả năng có lỗi trong code (ví dụ vô tình cộng thay vì trừ trong bước cập nhật).

Cách chọn tốc độ học: thử một dãy giá trị, mỗi giá trị lớn hơn giá trị trước khoảng ba lần (ví dụ 0,001 rồi 0,003 rồi 0,01 rồi 0,03…). Với mỗi giá trị, chạy vài vòng và xem đường cong học. Tìm giá trị lớn nhất mà chi phí vẫn giảm nhanh và đều, rồi chọn giá trị đó hoặc một giá trị nhỏ hơn nó một chút.

2.3.6 Kỹ thuật đặc trưng và hồi quy đa thức

Kỹ thuật đặc trưng (feature engineering) là dùng hiểu biết về bài toán để tạo ra đặc trưng mới, thường bằng cách biến đổi hoặc kết hợp các đặc trưng gốc, giúp mô hình dự đoán tốt hơn. Ví dụ dự đoán giá nhà từ chiều rộng và chiều sâu của mảnh đất: thay vì dùng riêng hai đặc trưng này, ta có thể tạo đặc trưng mới là *diện tích* bằng chiều rộng nhân chiều sâu — vì diện tích thường dự báo giá tốt hơn. Khi đó mô hình được phép tự quyết định xem chiều rộng, chiều sâu hay diện tích là yếu tố quan trọng nhất.

Hồi quy đa thức (polynomial regression) là một dạng của kỹ thuật đặc trưng cho phép khớp *đường cong*, không chỉ đường thẳng. Ý tưởng: ngoài đặc trưng gốc, ta thêm các đặc trưng là *luỹ thừa* của nó (bình phương, lập phương…), hoặc dạng khác như căn bậc hai. Chẳng hạn với dữ liệu giá nhà, một đường cong bậc hai hoặc bậc ba có thể khớp tốt hơn đường thẳng. Lưu ý: khi tạo các đặc trưng luỹ thừa, thang đo của chúng chênh nhau rất lớn (diện tích, diện tích bình phương, diện tích lập phương), nên *chuẩn hoá đặc trưng càng quan trọng*.

Làm sao biết nên dùng đặc trưng nào? Chương sau sẽ trình bày quy trình đo lường và so sánh các mô hình để quyết định. Ở đây chỉ cần nhớ: bạn có quyền lựa chọn đặc trưng, và kỹ thuật đặc trưng cùng hồi quy đa thức có thể cải thiện mô hình đáng kể.

2.3.7 Thực hành: hồi quy tuyến tính với scikit-learn

Trong thực tế, ta hiếm khi tự viết lại gradient descent; thay vào đó dùng thư viện mã nguồn mở **scikit-learn**, được dùng rộng rãi ở nhiều công ty hàng đầu. Có hai cách thường dùng cho hồi quy tuyến tính.

Cách 1 — gradient descent với `SGDRegressor`. Công cụ này huấn luyện bằng gradient descent và hoạt động tốt nhất khi dữ liệu đã được chuẩn hoá, nên ta dùng kèm `StandardScaler` (chuẩn hoá điểm Z). Đoạn code dưới đây lấy từ bài thực hành của khoá học (dùng bộ dữ liệu nhà gồm bốn đặc trưng: diện tích, số phòng ngủ, số tầng, tuổi nhà):

```
import numpy as np
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler

# X_train: các đặc trưng; y_train: giá nhà (đã nạp từ bộ dữ liệu nhà)
# Bốn đặc trưng: diện tích (sqft), số phòng ngủ, số tầng, tuổi nhà

# Bước 1: chuẩn hoá đặc trưng (z-score) để gradient descent chạy nhanh
scaler = StandardScaler()
X_norm = scaler.fit_transform(X_train)

# Bước 2: tạo và huấn luyện mô hình bằng gradient descent
sgdr = SGDRegressor(max_iter=1000)
sgdr.fit(X_norm, y_train)

# Bước 3: xem tham số đã học (gắn với dữ liệu đã chuẩn hoá)
b_norm = sgdr.intercept_   # độ chệch
w_norm = sgdr.coef_        # danh sách trọng số

# Bước 4: dự đoán
y_pred = sgdr.predict(X_norm)
```

Kết quả đã lưu trong bài thực hành cho thấy mô hình học được bốn trọng số xấp xỉ `109,94`; `-20,96`; `-32,33`; `-38,11` và độ chệch xấp xỉ `363,18`. Các giá trị này gần như trùng với kết quả của bản tự cài đặt bằng NumPy trong bài trước (bốn trọng số `110,56`; `-21,27`; `-32,71`; `-37,97` và độ chệch `363,16`), cho thấy công cụ thư viện và bản tự viết cho cùng một mô hình. Dự đoán trên bốn căn nhà đầu là `295,2`; `485,9`; `389,7`; `492,1` so với giá thật `300`; `509,8`; `394`; `540` — sát nhưng không hoàn hảo, đúng như kỳ vọng cho một mô hình tuyến tính.

Cách 2 — lời giải trực tiếp với `LinearRegression`. Công cụ này dùng phương trình chuẩn (lời giải dạng đóng), *không* cần chuẩn hoá và cho kết quả gần như tức thời trên dữ liệu nhỏ:

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Ví dụ nhỏ: nhà 1000 sqft bán 300 nghìn, nhà 2000 sqft bán 500 nghìn
X_train = np.array([1.0, 2.0])   # đặc trưng (nghìn sqft)
y_train = np.array([300, 500])   # mục tiêu (nghìn đô)

linear_model = LinearRegression()
linear_model.fit(X_train.reshape(-1, 1), y_train)  # X phải là ma trận 2 chiều

b = linear_model.intercept_      # độ chệch
w = linear_model.coef_           # trọng số

# Dự đoán cho căn nhà 1200 sqft
X_test = np.array([[1200]])
print(linear_model.predict(X_test))
```

Theo bài thực hành, trên bộ dữ liệu nhà bốn đặc trưng, lời giải trực tiếp cho tham số và dự đoán rất sát với bản chạy gradient descent không chuẩn hoá kéo dài hàng giờ ở bài trước — nhưng ở đây gần như tức thời. Lời giải trực tiếp hiệu quả trên dữ liệu nhỏ, nhưng tốn kém tính toán khi dữ liệu rất lớn.

2.4 Phân loại với hồi quy logistic

2.4.1 Vì sao hồi quy tuyến tính không hợp cho phân loại

Ở bài toán phân loại, đầu ra chỉ nhận một trong một số ít giá trị. Dạng cơ bản nhất là **phân loại nhị phân** (binary classification): chỉ hai nhóm, thường ký hiệu "không / có", "sai / đúng", hoặc số 0 và 1. Theo quy ước, nhóm 0 gọi là *lớp âm* và nhóm 1 là *lớp dương*; "âm" và "dương" ở đây chỉ sự *vắng mặt* hay *hiện diện* của thứ ta quan tâm (ví dụ email có phải spam không), không mang nghĩa tốt/xấu. Ví dụ: email spam hay không, giao dịch có gian lận không, khối u lành hay ác tính.

Vì sao không dùng thẳng hồi quy tuyến tính rồi đặt ngưỡng? Giả sử ta khớp một đường thẳng vào dữ liệu khối u (trục ngang là kích thước, trục dọc là nhãn 0 hoặc 1), rồi quy ước: dự đoán trên 0,5 thì xếp vào ác tính, dưới 0,5 thì lành tính. Với dữ liệu "đẹp" thì có vẻ ổn. Nhưng chỉ cần thêm *một* mẫu mới nằm xa về bên phải (một khối u rất lớn, hiển nhiên là ác tính), đường thẳng tối ưu bị *kéo lệch* đi, làm dịch chuyển cả ranh giới phân loại và khiến một số mẫu bị xếp sai. Trực giác cho thấy thêm một mẫu rõ ràng như vậy lẽ ra không nên thay đổi ranh giới, nhưng hồi quy tuyến tính lại bị ảnh hưởng. Hồi quy tuyến tính cũng trả ra những giá trị vượt ngoài khoảng 0 đến 1, điều vô nghĩa với phân loại. Vì vậy ta cần một mô hình khác: **hồi quy logistic**.

2.4.2 Hàm sigmoid và mô hình logistic

Dù tên có chữ "hồi quy", **hồi quy logistic** (logistic regression) thực chất dùng cho *phân loại* — đây là thuật toán phân loại được dùng rộng rãi nhất. Thay vì khớp đường thẳng, nó khớp một *đường cong hình chữ S* có đầu ra luôn nằm giữa 0 và 1.

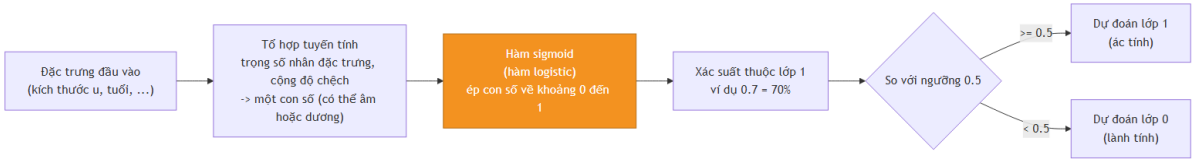
Trái tim của mô hình là **hàm sigmoid** (còn gọi là hàm logistic). Diễn giải bằng lời: hàm này nhận một con số bất kỳ (có thể âm hoặc dương) và ép nó về khoảng giữa 0 và 1. Tính chất của đường cong chữ S:

- Khi đầu vào là số dương rất lớn, đầu ra tiến gần tới 1.
- Khi đầu vào là số âm rất lớn, đầu ra tiến gần tới 0.

- Khi đầu vào bằng 0, đầu ra đúng bằng 0,5 (điểm giữa của chữ S).

Mô hình hồi quy logistic hoạt động hai bước (Hình 2.7):

1. **Tổ hợp tuyến tính**: giống hồi quy tuyến tính, lấy từng đặc trưng nhân trọng số của nó rồi cộng lại, cộng thêm độ chệch, ra một con số (có thể âm hoặc dương).
2. **Đưa qua sigmoid**: ép con số đó về khoảng 0 đến 1.



Hình 2.7: Kiến trúc hồi quy logistic: tổ hợp tuyến tính rồi đưa qua sigmoid để ra xác suất, sau đó so với ngưỡng để quyết định nhóm.

Cách diễn giải đầu ra: hãy xem đầu ra của hồi quy logistic là **xác suất** mẫu thuộc lớp 1. Ví dụ với một bệnh nhân, nếu mô hình trả ra 0,7 nghĩa là nó cho rằng có 70% khả năng khối u là ác tính. Vì chỉ có hai nhóm và tổng xác suất bằng 100%, nên xác suất thuộc lớp 0 là 30% còn lại.

2.4.3 Ranh giới quyết định

Để chuyển từ xác suất sang quyết định nhóm, ta đặt một **ngưỡng**, thường là 0,5: nếu xác suất từ 0,5 trở lên thì dự đoán lớp 1, dưới 0,5 thì lớp 0. Vì sigmoid trả 0,5 đúng khi con số đầu vào (tổ hợp tuyến tính) bằng 0, nên quyết định thực chất phụ thuộc vào dấu của tổ hợp tuyến tính: dương thì lớp 1, âm thì lớp 0.

Ranh giới quyết định (decision boundary) là tập các điểm mà tổ hợp tuyến tính đúng bằng 0 — nơi mô hình "phân vân" giữa hai nhóm. Với hai đặc trưng và tổ hợp tuyến tính đơn giản, ranh giới này là một *đường thẳng* chia mặt phẳng thành hai vùng: một bên dự đoán lớp 1, bên kia lớp 0.

Nếu đưa thêm các đặc trưng đa thức (ví dụ bình phương của các đặc trưng), ranh giới quyết định không còn là đường thẳng mà có thể là *đường tròn, elip*, hay những hình phức tạp hơn. Nhờ vậy hồi quy logistic có thể khớp dữ liệu khá phức tạp. Ngược lại, nếu chỉ dùng các đặc trưng gốc (không thêm đa thức), ranh giới luôn là một đường thẳng.

2.4.4 Hàm chi phí cho hồi quy logistic

Ta có thể dùng lại hàm chi phí sai số bình phương như hồi quy tuyến tính không? Không nên. Nếu dùng, mặt chi phí của hồi quy logistic trở nên *gồ ghề*, nhiều hõm (ta gọi là *không lồi*), khiến gradient descent dễ kẹt ở các cực tiểu cục bộ. Ta cần một hàm chi phí khác để mặt chi phí trở lại dạng cái bát trơn (lồi), bảo đảm gradient descent hội tụ về cực tiểu toàn cục.

Trước hết định nghĩa **mất mát** (loss) trên *một* mẫu — đo mức sai của dự đoán so với nhãn đúng của mẫu đó. Hàm mất mát cho hồi quy logistic, diễn giải bằng lời:

- **Khi nhãn đúng là 1**: mất mát là *âm lô-ga-rit của xác suất mô hình dự đoán*. Trực giác: nếu mô hình dự đoán xác suất gần 1 (rất tự tin và đúng), mất mát gần 0; nếu mô hình dự đoán xác suất càng gần 0 (tự tin nhưng sai), mất mát càng lớn, tiến tới vô cùng. Nói cách khác, mô hình bị phạt nặng khi tự tin dự đoán sai.
- **Khi nhãn đúng là 0**: mất mát là *âm lô-ga-rit của phần bù xác suất* (tức của "một trừ xác suất dự đoán lớp 1"). Nếu mô hình dự đoán xác suất gần 0 (đúng), mất mát gần 0; nếu dự đoán gần 1 (sai), mất mát rất lớn.

Có thể gộp hai trường hợp trên thành *một* biểu thức duy nhất, nhờ việc nhãn chỉ nhận giá trị 0 hoặc 1: khi nhãn là 1 thì chỉ còn phần thứ nhất, khi nhãn là 0 thì chỉ còn phần thứ hai. Cách viết gộp này giúp cài đặt gọn hơn.

Hàm chi phí toàn cục là **trung bình mất mát** trên toàn bộ tập huấn luyện. Hàm chi phí này được suy ra từ một nguyên lý thống kê gọi là *ước lượng hợp lý cực đại* và có tính chất đẹp là *lồi*, nên gradient descent sẽ hội tụ đáng tin cậy về cực tiểu toàn cục.

2.4.5 Gradient descent cho hồi quy logistic

Để huấn luyện, ta lại dùng gradient descent để tìm trọng số và độ chệch làm chi phí nhỏ nhất. Điều thú vị: công thức cập nhật trông *giống hệt* hồi quy tuyến tính — vẫn là "dự đoán trừ mục tiêu, nhân đặc trưng, lấy trung bình". Nhưng đây *không* phải cùng một thuật toán, vì bản chất hàm dự đoán đã khác: ở hồi quy tuyến tính dự đoán là tổ hợp tuyến tính; ở hồi quy logistic dự đoán là tổ hợp tuyến tính *đưa qua sigmoid*.

Mọi mẹo đã học vẫn áp dụng: vẽ đường cong học để kiểm tra hội tụ, chuẩn hoá đặc trưng để tăng tốc, và vector hoá để chạy nhanh.

2.4.6 Thực hành: hồi quy logistic với scikit-learn

Với scikit-learn, huấn luyện một bộ phân loại logistic chỉ gồm vài dòng. Đoạn code và kết quả dưới đây lấy từ bài thực hành của khoá học:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Dữ liệu nhỏ: 6 điểm, 2 đặc trưng, nhãn 0 hoặc 1
X = np.array([[0.5, 1.5], [1, 1], [1.5, 0.5], [3, 0.5], [2, 2], [1, 2.5]])
y = np.array([0, 0, 0, 1, 1, 1])

# Tạo và huấn luyện mô hình
lr_model = LogisticRegression()
lr_model.fit(X, y)

# Dự đoán nhóm cho từng điểm
y_pred = lr_model.predict(X)
print("Dự đoán trên tập huấn luyện:", y_pred)

# Độ chính xác (tỉ lệ dự đoán đúng)
print("Độ chính xác:", lr_model.score(X, y))
```

Theo kết quả đã lưu trong bài thực hành, mô hình dự đoán đúng toàn bộ: dự đoán là `[0 0 0 1 1 1]` (trùng với nhãn thật) và độ chính xác đạt `1.0` (100%) trên tập dữ liệu nhỏ này. Nhiều kỹ sư học máy dùng scikit-learn thường xuyên trong công việc, nên làm quen với mẫu `fit` / `predict` / `score` này rất hữu ích.

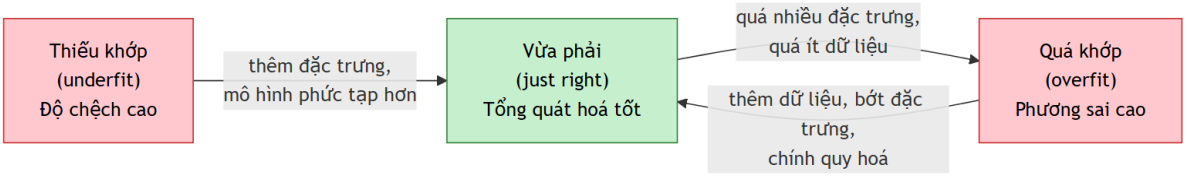
2.5 Quá khớp và chính quy hoá

2.5.1 Quá khớp và thiếu khớp

Đôi khi mô hình chạy kém vì rơi vào một trong hai thái cực (Hình 2.8):

- **Thiếu khớp** (underfit), hay *độ chệch cao* (high bias): mô hình quá đơn giản, không nắm được quy luật rõ ràng trong dữ liệu. Ví dụ dùng một đường thẳng cho dữ liệu giá nhà vốn cong lại ở các căn lớn. Giống như mô hình có một "định kiến" quá mạnh rằng dữ liệu phải tuyến tính.

- **Quá khớp** (overfit), hay *phương sai cao* (high variance): mô hình quá phức tạp, cố đi qua *mọi* điểm huấn luyện, tạo ra đường ngoằn ngoèo. Nó khớp dữ liệu huấn luyện gần như hoàn hảo nhưng *tổng quát hoá kém* — dự đoán tệ trên dữ liệu mới. Chỉ cần đổi dữ liệu huấn luyện một chút là mô hình đổi hẳn, nên gọi là phương sai cao.
- **Vừa phải** (just right): không thiếu khớp cũng không quá khớp; tổng quát hoá tốt sang các ví dụ chưa thấy. Đây là mục tiêu.



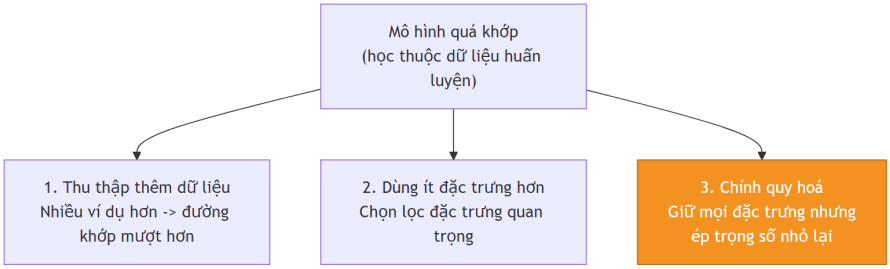
Hình 2.8: Phổ từ thiếu khớp tới quá khớp, và hướng điều chỉnh giữa chúng.

Hiện tượng này xảy ra ở cả hồi quy lẫn phân loại. "Tổng quát hoá tốt" nghĩa là mô hình dự đoán tốt ngay cả trên các ví dụ hoàn toàn mới — đây mới là điều ta thực sự cần.

2.5.2 Ba cách giảm quá khớp

Khi nghi ngờ mô hình quá khớp, có ba hướng xử lý (Hình 2.9):

1. **Thu thập thêm dữ liệu**: đây là công cụ mạnh nhất. Với nhiều ví dụ hơn, dù mô hình phức tạp, nó vẫn học được đường khớp mượt mà hơn. Nhược điểm: không phải lúc nào cũng có thêm dữ liệu.
2. **Dùng ít đặc trưng hơn** (chọn lọc đặc trưng): nếu có quá nhiều đặc trưng nhưng ít dữ liệu, mô hình dễ quá khớp. Chọn một tập con các đặc trưng quan trọng nhất có thể giúp. Nhược điểm: vứt bớt đặc trưng là vứt bớt thông tin có thể hữu ích.
3. **Chính quy hoá** (regularization): giữ *tất cả* đặc trưng nhưng ép các trọng số nhỏ lại, giảm ảnh hưởng quá mức của chúng. Đây là cách nhẹ nhàng hơn so với việc loại bỏ hẳn đặc trưng, và được dùng rất thường xuyên trong thực tế.



Hình 2.9: Ba hướng giảm quá khớp.

2.5.3 Chính quy hoá hoạt động thế nào

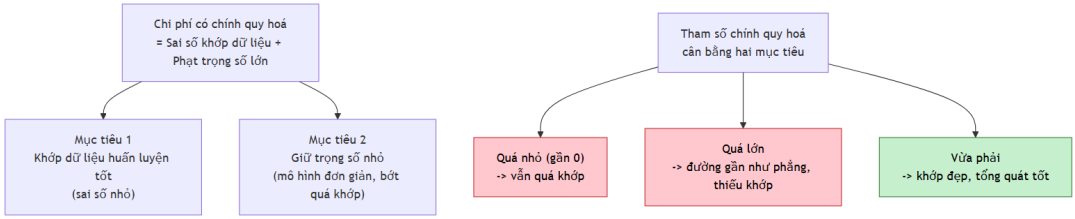
Trực giác: khi mô hình quá khớp, các trọng số thường *lớn*. Nếu ta ép trọng số nhỏ lại, đường khớp sẽ đơn giản và mượt hơn, bớt ngoằn ngoèo, nên bớt quá khớp. Chính quy hoá làm điều đó bằng cách thêm một **khoản phạt** vào hàm chi phí: ngoài việc phạt sai số khớp dữ liệu như trước, ta còn phạt khi trọng số

lớn. Thông thường ta phạt tất cả các trọng số (vì khó biết trước trọng số nào quan trọng), nhưng không phạt độ chệch — trên thực tế có phạt độ chệch hay không gần như không khác biệt.

Như vậy hàm chi phí mới cân bằng *hai mục tiêu* (Hình 2.10): (1) khớp dữ liệu huấn luyện tốt (sai số nhỏ) và (2) giữ trọng số nhỏ (mô hình đơn giản, bớt quá khớp). Mức độ cân bằng do một con số gọi là **tham số chính quy hoá** điều khiển:

- Nếu tham số này *quá nhỏ* (gần 0): gần như không chính quy hoá, mô hình vẫn quá khớp.
- Nếu *quá lớn*: khoản phạt lấn át, mọi trọng số bị ép gần 0, mô hình thành một đường gần như phẳng và thiếu khớp.
- Nếu *vừa phải*: mô hình khớp đẹp và tổng quát hoá tốt.

Một chi tiết quy ước: khoản phạt được chia theo cỡ tập huấn luyện, giúp cùng một giá trị tham số chính quy hoá vẫn phù hợp khi tập dữ liệu lớn lên.



Hình 2.10: Chính quy hoá cân bằng giữa khớp dữ liệu và giữ trọng số nhỏ; tham số chính quy hoá quyết định nghiêng về bên nào.

2.5.4 Chính quy hoá trong gradient descent

Khi áp chính quy hoá, bước cập nhật gradient descent gần như giữ nguyên, chỉ thêm một chút cho phần trọng số. Hiệu ứng có thể diễn giải rất gọn: ở mỗi vòng lặp, trước khi thực hiện bước cập nhật thông thường, mỗi trọng số được *nhân với một số nhỏ hơn 1 một chút* (ví dụ 0,9998). Việc nhân này "co" trọng số lại một ít sau mỗi vòng — đó chính là cách chính quy hoá ép trọng số nhỏ dần. Độ chệch không bị chính quy hoá nên bước cập nhật của nó không đổi.

Cách làm này áp dụng được cho *cả* hồi quy tuyến tính và hồi quy logistic: công thức gần như giống nhau, chỉ khác ở chỗ hàm dự đoán của logistic có thêm bước sigmoid. Nhờ chính quy hoá, ngay cả khi dùng nhiều đặc trưng (kể cả đặc trưng đa thức bậc cao), ta vẫn thu được ranh giới quyết định hợp lý và mô hình tổng quát hoá tốt.

2.6 Tổng kết chương

Chương này đã đi qua nền tảng của học máy có giám sát:

- **Khái niệm**: học máy học từ dữ liệu; học có giám sát dùng dữ liệu có nhãn, chia thành hồi quy (dự đoán số) và phân loại (dự đoán nhóm).
- **Hồi quy tuyến tính**: khớp đường thẳng (hoặc đường cong qua kỹ thuật đặc trưng); dùng hàm chi phí sai số bình phương để đo độ khớp; dùng gradient descent để tối ưu; tăng tốc bằng vector hoá, chuẩn hoá đặc trưng, và chọn tốc độ học hợp lý.
- **Hồi quy logistic**: dùng cho phân loại; đưa tổ hợp tuyến tính qua sigmoid để ra xác suất; đặt ngưỡng để quyết định nhóm; dùng hàm chi phí riêng (lồi) và gradient descent để huấn luyện.
- **Quá khớp và chính quy hoá**: nhận biết thiếu khớp (độ chệch cao) và quá khớp (phương sai cao); giảm quá khớp bằng thêm dữ liệu, bớt đặc trưng, hoặc chính quy hoá.

- **Thực chiến**: trong dự án thực tế, ưu tiên dùng thư viện như scikit-learn (`SGDRegressor`, `LinearRegression`, `LogisticRegression`) thay vì tự cài đặt thuật toán từ đầu.

Chỉ với hồi quy tuyến tính và hồi quy logistic cùng kỹ năng nhận biết và xử lý quá khớp, bạn đã có thể xây dựng nhiều ứng dụng giá trị. Chương sau sẽ mở rộng sang mạng nơ-ron và các thuật toán học nâng cao hơn.

Chương 3

Advanced Learning Algorithms

Chương trước dừng ở hồi quy tuyến tính và hồi quy logistic — đủ để giải nhiều bài toán, nhưng chưa đủ cho những dữ liệu phức tạp như ảnh, âm thanh hay văn bản. Chương này bước sang nhóm thuật toán học nâng cao, với hai trụ cột lớn: **mạng nơ-ron** (neural networks, hay học sâu) và **cây quyết định** (decision trees) cùng các biến thể ensemble. Xen giữa là một phần cực kỳ thực chiến: *lời khuyên khi áp dụng học máy* — cách chẩn đoán vì sao mô hình chạy chưa tốt và nên làm gì tiếp theo.

Cách tiếp cận vẫn như chương trước: *ý tưởng trước, công cụ sau*. Ta hiểu trực giác vì sao một kỹ thuật ra đời, rồi dùng nó qua framework như TensorFlow/Keras, scikit-learn hay XGBoost thay vì tự cài đặt lại từ đầu. Mọi công thức toán học được diễn giải bằng lời và sơ đồ; thay vì ký hiệu, ta gọi tên các đại lượng theo đúng ý nghĩa ("trọng số", "độ chệch", "tín hiệu kích hoạt", "xác suất", "độ tinh khiết"…) để nắm logic mà không bị rào cản ký hiệu.

3.1 Mạng nơ-ron: suy luận

3.1.1 Từ nơ-ron sinh học tới nơ-ron nhân tạo

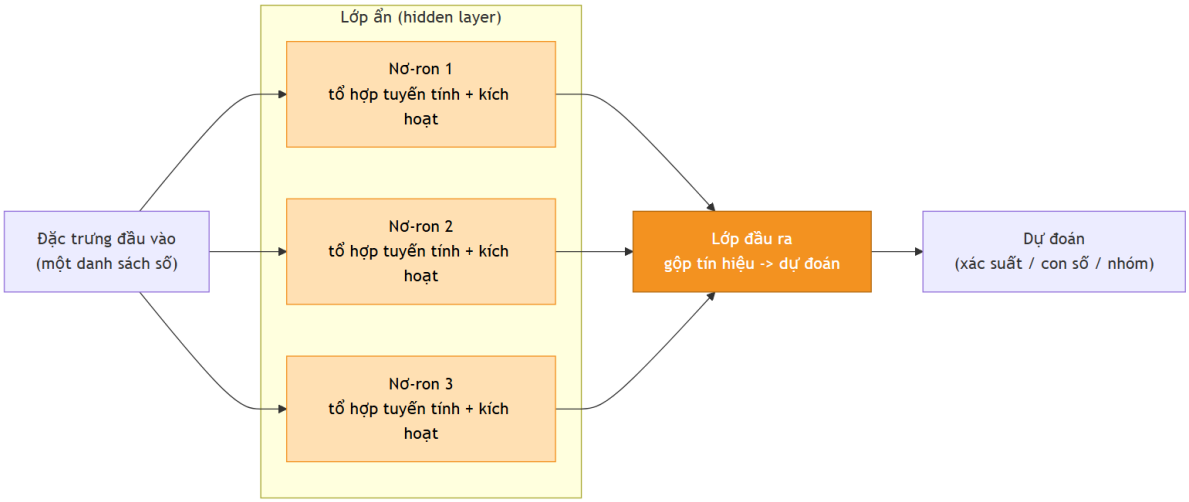
Mạng nơ-ron ra đời từ ý tưởng mô phỏng cách não bộ hoạt động. Não gồm vô số tế bào thần kinh (nơ-ron) nối với nhau: mỗi nơ-ron nhận tín hiệu từ các nơ-ron khác, xử lý, rồi phát tín hiệu đi tiếp. Mạng nơ-ron nhân tạo mượn hình ảnh đó, nhưng thực chất đơn giản hơn nhiều: nó chỉ là nhiều "đơn vị tính toán" nhỏ xếp thành lớp, mỗi đơn vị làm đúng một việc quen thuộc — lấy các đầu vào nhân với trọng số, cộng lại, cộng độ chệch, rồi đưa qua một hàm kích hoạt.

Nói cách khác, **một nơ-ron nhân tạo chính là một mô hình hồi quy logistic (hoặc tuyến tính) thu nhỏ**. Nó nhận một danh sách số đầu vào, tính một tổ hợp tuyến tính, rồi đưa qua hàm kích hoạt (ví dụ sigmoid) để cho ra một con số gọi là **tín hiệu kích hoạt** (activation). Điều làm nên sức mạnh không nằm ở một nơ-ron, mà ở chỗ ta xếp *nhiều* nơ-ron thành *nhiều lớp* nối tiếp nhau.

3.1.2 Nơ-ron và lớp

Một **lớp** (layer) là một nhóm nơ-ron cùng nhận chung đầu vào và cùng cho ra các tín hiệu kích hoạt. Một mạng nơ-ron điển hình gồm ba loại lớp:

- **Lớp đầu vào** (input layer): chính là danh sách đặc trưng ban đầu, ví dụ các con số mô tả một căn nhà hay các điểm ảnh của một bức ảnh.
- **Lớp ẩn** (hidden layer): các lớp ở giữa. Mỗi nơ-ron ở đây tự học cách tổ hợp đầu vào để tạo ra một đặc trưng mới, tinh vi hơn. Gọi là "ẩn" vì ta không trực tiếp quan sát giá trị của chúng trong dữ liệu huấn luyện.
- **Lớp đầu ra** (output layer): lớp cuối cùng, gộp mọi tín hiệu để cho ra dự đoán (một xác suất, một con số, hay một nhóm).



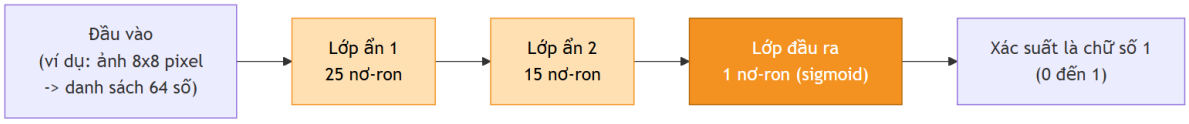
Hình 3.1: Một lớp gồm nhiều nơ-ron: mỗi nơ-ron nhận cùng đầu vào, tự tính một tín hiệu kích hoạt, rồi chuyển sang lớp sau.

Ý tưởng cốt lõi để "đọc" một mạng nơ-ron: **đầu ra của lớp này trở thành đầu vào của lớp kế tiếp**. Các đặc trưng gốc đi vào lớp đầu, mỗi lớp ẩn biến chúng thành một tập đặc trưng mới "cô đọng" hơn, và cứ thế cho tới lớp đầu ra. Ta không cần tự thiết kế các đặc trưng trung gian này — mạng tự học chúng trong quá trình huấn luyện. Đây chính là lý do mạng nơ-ron mạnh với dữ liệu phức tạp: nó tự khám phá ra các đặc trưng hữu ích thay vì bắt con người làm thủ công.

3.1.3 Lan truyền xuôi: cách mạng đưa ra dự đoán

Lan truyền xuôi (forward propagation) là quá trình đưa dữ liệu đi *xuôi* từ lớp đầu vào qua các lớp ẩn tới lớp đầu ra để tạo dự đoán. Diễn giải bằng lời: tính xong tín hiệu kích hoạt của lớp thứ nhất, ta dùng chúng làm đầu vào tính tín hiệu của lớp thứ hai, rồi tiếp tục tới lớp cuối. "Xuôi" là vì dữ liệu chỉ chảy theo một chiều, từ đầu vào ra đầu ra.

Lấy ví dụ nhận diện chữ số viết tay (phân biệt chữ số 0 và 1 từ ảnh nhỏ). Mỗi ảnh được trải phẳng thành một danh sách số (mỗi số là độ sáng một điểm ảnh). Danh sách đó đi qua hai lớp ẩn rồi tới một nơ-ron đầu ra dùng sigmoid, cho ra xác suất ảnh là chữ số 1 (Hình 3.2).



Hình 3.2: Lan truyền xuôi cho bài nhận diện chữ số: dữ liệu chảy một chiều từ ảnh đầu vào qua các lớp ẩn tới lớp đầu ra, cho ra một xác suất.

3.1.4 Suy luận với TensorFlow/Keras

Trong thực tế, ta không tự viết các phép nhân cộng của từng nơ-ron. **TensorFlow** là một framework học sâu phổ biến, và **Keras** là giao diện cấp cao giúp dựng mạng chỉ bằng vài dòng. Hai thành phần hay dùng nhất:

- `Sequential`: xếp các lớp nối tiếp nhau thành một mạng.
- `Dense`: một lớp "kết nối đầy đủ" — mỗi nơ-ron trong lớp nối tới mọi đầu vào. Khi tạo một lớp Dense, ta khai báo số nơ-ron và hàm kích hoạt của lớp đó.

Ví dụ bài rang cà phê (coffee roasting) của khoá học: dự đoán một mẻ cà phê có ngon không dựa trên hai yếu tố là nhiệt độ và thời gian rang. Mạng gồm một lớp ẩn 3 nơ-ron và một lớp đầu ra 1 nơ-ron, tất cả dùng sigmoid:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Normalization

# X: hai đặc trưng (nhiệt độ, thời gian); Y: ngon (1) hay không (0)
# Chuẩn hoá đặc trưng cho đồng thang đo (giống z-score ở chương trước)
norm_l = Normalization(axis=-1)
norm_l.adapt(X)          # học trung bình và độ lệch chuẩn từ dữ liệu
Xn = norm_l(X)

# Dựng mạng: 1 lớp ẩn 3 nơ-ron + 1 lớp đầu ra 1 nơ-ron
model = Sequential([
    tf.keras.Input(shape=(2,)),
    Dense(3, activation='sigmoid', name='layer1'),
    Dense(1, activation='sigmoid', name='layer2'),
])
model.summary()          # in cấu trúc và số tham số của mạng
```

Để suy luận (dự đoán) trên dữ liệu mới, ta nhớ chuẩn hoá đầu vào mới bằng *cùng* bộ chuẩn hoá đã học, rồi gọi `predict`. Đầu ra là xác suất; muốn ra quyết định 0/1 thì so với ngưỡng 0,5:

```
# Hai mẫu mới: một mẻ "tốt" và một mẻ "xấu"
X_test = np.array([
    [200, 13.9],   # kỳ vọng ngon
    [200, 17]      # kỳ vọng không ngon
])
X_testn = norm_l(X_test)           # chuẩn hoá bằng cùng bộ đã học
predictions = model.predict(X_testn)   # ra xác suất (0..1)

# Quyết định nhóm bằng ngưỡng 0.5
yhat = (predictions >= 0.5).astype(int)
print(yhat)   # ví dụ: [[1], [0]]
```

Theo kết quả đã lưu trong bài thực hành, mẻ "tốt" cho xác suất gần 1 còn mẻ "xấu" cho xác suất gần 0, nên sau khi so ngưỡng ta được quyết định `[[1], [0]]` — đúng như kỳ vọng. Điểm cần nhớ ở đây là *mẫu sử dụng*: dựng mạng bằng `Sequential` và `Dense`, chuẩn hoá đầu vào, rồi `predict`.

3.1.5 Mạng nơ-ron chỉ là các phép tính trên danh sách số

Để không thấy mạng nơ-ron "huyền bí", hãy nhớ: bên dưới framework, một lớp Dense chỉ làm đúng việc đã học ở chương trước, lặp lại cho nhiều nơ-ron cùng lúc. Mỗi nơ-ron lấy đầu vào nhân trọng số của nó, cộng lại, cộng độ chệch, rồi đưa qua hàm kích hoạt. Khi gộp mọi nơ-ron trong một lớp, toàn bộ phép tính được viết gọn bằng các thao tác trên danh sách số (vector và ma trận) của NumPy — chính là *vector hoá* đã bàn ở chương trước, giúp chạy nhanh nhờ phần cứng song song.

Bài thực hành của khoá học có một bản cài đặt lan truyền xuôi "thủ công" bằng NumPy cho mạng rang cà phê, và kết quả của nó trùng khớp với bản chạy bằng TensorFlow. Thông điệp: framework chỉ gói gọn các phép tính quen thuộc lại cho tiện, chứ không làm điều gì khác về bản chất. Trong dự án thực tế, ta dùng thẳng framework.

3.1.6 Thực hành tuần 1: nhận diện chữ số viết tay

Bài tập lớn tuần 1 xây một mạng nơ-ron nhận diện chữ số viết tay 0 và 1 từ ảnh. Dữ liệu gồm 5000 ảnh, mỗi ảnh kích thước 20×20 điểm ảnh được trải phẳng thành một danh sách 400 số. Mạng gồm hai lớp ẩn (lần lượt 25 và 15 nơ-ron) và một nơ-ron đầu ra sigmoid:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import tensorflow as tf

model = Sequential([
    tf.keras.Input(shape=(400,)),
    Dense(25, activation='sigmoid', name='L1'),
    Dense(15, activation='sigmoid', name='L2'),
    Dense(1,  activation='sigmoid', name='L3'),
], name='my_model')
```

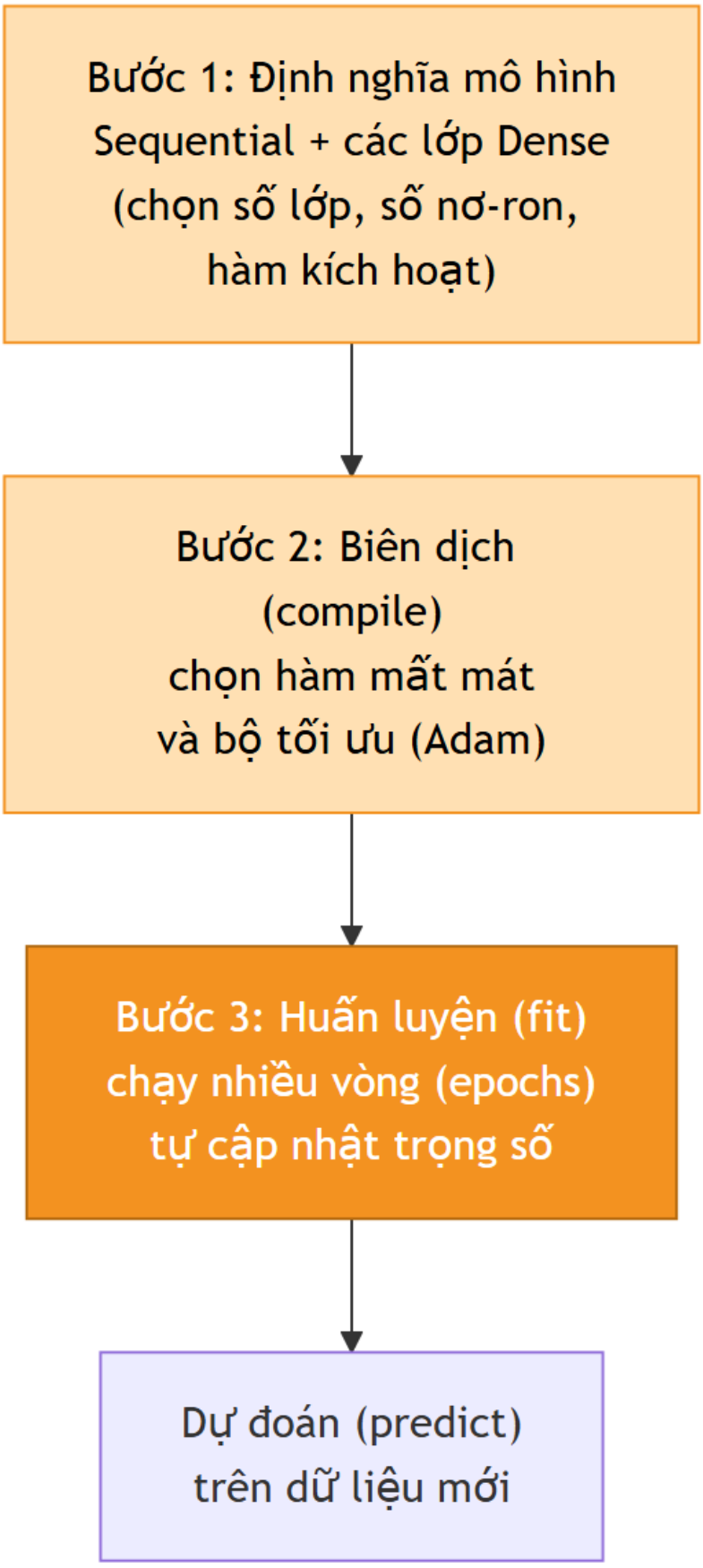
Bài thực hành xác nhận hình dạng (shape) các khối trọng số đúng như kỳ vọng: lớp 1 có khối trọng số cỡ 400×25, lớp 2 cỡ 25×15, lớp 3 cỡ 15×1 (mỗi lớp kèm một danh sách độ chệch tương ứng). Sau khi huấn luyện, mô hình dự đoán đúng các ảnh mẫu trong bài. Phần huấn luyện và lựa chọn hàm mất mát sẽ được nói kỹ ở mục sau; ở tuần 1 trọng tâm là hiểu *suy luận* — cách mạng đưa ra dự đoán qua lan truyền xuôi.

3.2 Huấn luyện mạng nơ-ron

3.2.1 Ba bước huấn luyện với TensorFlow/Keras

Huấn luyện một mạng nơ-ron trong Keras luôn theo đúng ba bước, và ba bước này cũng tương ứng với ba câu hỏi quen thuộc từ chương trước (Hình 3.3):

1. **Định nghĩa mô hình** với `Sequential` và các lớp `Dense`: trả lời câu hỏi "hàm dự đoán trông thế nào" (chọn số lớp, số nơ-ron, hàm kích hoạt).
2. **Biên dịch** (`compile`): chọn *hàm mất mát* (đo mức sai của dự đoán) và *bộ tối ưu* (cách cập nhật trọng số). Đây là bước trả lời "đo sai thế nào" và "tối ưu bằng gì".
3. **Huấn luyện** (`fit`): chạy qua dữ liệu nhiều vòng (epochs), mỗi vòng mạng tự điều chỉnh trọng số để giảm mất mát.



Hình 3.3: Ba bước huấn luyện trong Keras: định nghĩa, biên dịch, rồi huấn luyện.

Minh hoạ cho bài phân loại nhị phân (ví dụ nhận diện chữ số 0/1 ở tuần 1):

```
from tensorflow.keras.losses import BinaryCrossentropy

# Bước 2: biên dịch -- chọn hàm mất mát và bộ tối ưu
model.compile(
    loss=BinaryCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
)

# Bước 3: huấn luyện -- chạy 20 vòng qua dữ liệu
history = model.fit(X, y, epochs=20)
```

Hàm mất mát `BinaryCrossentropy` chính là hàm chi phí của hồi quy logistic ở chương trước (phạt nặng khi mô hình tự tin nhưng sai), chỉ khác tên gọi trong TensorFlow. Khi `fit` chạy, mạng tự thực hiện lan truyền xuôi để dự đoán, so với nhãn đúng để tính mất mát, rồi điều chỉnh toàn bộ trọng số — ta không phải tự viết các bước này.

3.2.2 Lan truyền ngược: trực giác, không công thức

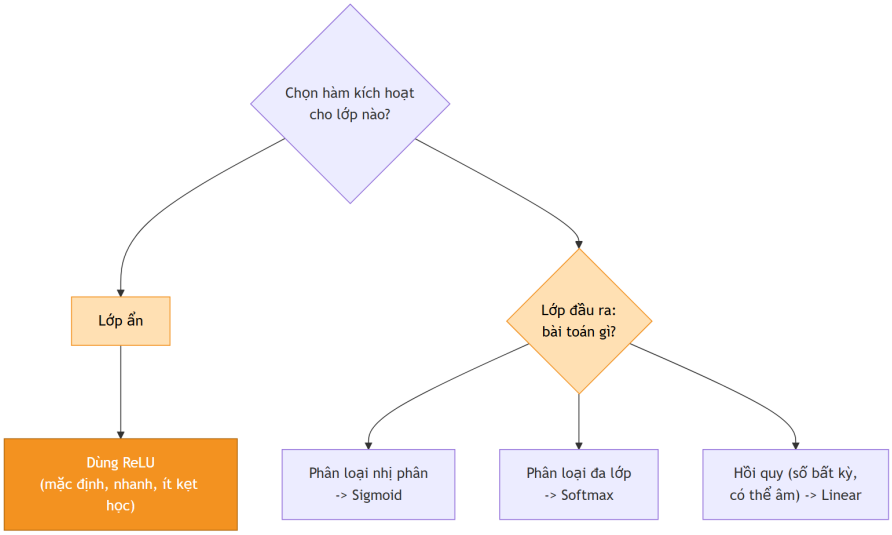
Làm sao mạng biết cần chỉnh mỗi trọng số theo hướng nào? Câu trả lời là **lan truyền ngược** (backpropagation). Diễn giải bằng lời: sau khi tính xong dự đoán và biết mình sai bao nhiêu ở đầu ra, mạng "truy ngược" từ lớp cuối về lớp đầu để tính xem *mỗi* trọng số đã góp bao nhiêu vào sai số đó. Trọng số nào góp làm tăng sai số thì được chỉnh giảm, và ngược lại — đúng tinh thần gradient descent, chỉ là áp cho rất nhiều trọng số cùng lúc.

Điểm then chốt khiến lan truyền ngược hiệu quả: nó *tái sử dụng* các kết quả tính trung gian thay vì tính lại từ đầu cho từng trọng số. Nhờ vậy chi phí tính toán để huấn luyện cả một mạng khổng lồ vẫn ở mức chấp nhận được. Trong thực tế, TensorFlow tự lo toàn bộ phần này; ta chỉ cần gọi `fit`. Hiểu trực giác là đủ để dùng tốt.

3.2.3 Hàm kích hoạt: chọn thế nào

Hàm kích hoạt quyết định mỗi nơ-ron "phát tín hiệu" ra sao. Ba hàm thường gặp:

- **Sigmoid**: ép giá trị về khoảng 0 đến 1. Hợp cho đầu ra là *xác suất* trong phân loại nhị phân.
- **ReLU** (rectified linear unit): giữ nguyên số dương, biến số âm thành 0. Diễn giải: "tắt" khi tín hiệu âm, "bật tỉ lệ thuận" khi tín hiệu dương. Đây là lựa chọn mặc định cho các lớp ẩn.
- **Linear** (tuyến tính, tức không biến đổi gì): giữ nguyên con số, cho ra giá trị bất kỳ (âm hay dương). Hợp cho đầu ra của bài *hồi quy*.



Hình 3.4: Cách chọn hàm kích hoạt: lớp ẩn dùng ReLU; lớp đầu ra chọn theo loại bài toán.

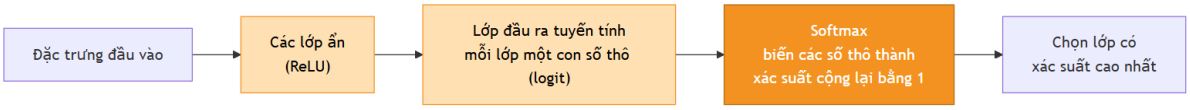
Vì sao lớp ẩn thường dùng ReLU chứ không phải sigmoid. Sigmoid bị "bão hoà" ở hai đầu: khi đầu vào rất lớn hoặc rất nhỏ, đường cong gần như phẳng, độ dốc gần bằng 0. Độ dốc nhỏ khiến gradient descent cập nhật rất chậm (hiện tượng "gradient tiêu biến"). ReLU chỉ phẳng ở phía âm, phía dương luôn có độ dốc ổn định, nên mạng học nhanh hơn. Bài thực hành `ReLU` của khoá học minh hoạ: nhờ chỗ "gãy" tại 0, ReLU cho phép mạng ghép nhiều đoạn thẳng lại thành những đường cong phức tạp, mô tả dữ liệu phi tuyến tốt hơn.

Vì sao không thể dùng toàn hàm tuyến tính. Nếu mọi lớp đều dùng kích hoạt tuyến tính, cả mạng dù sâu bao nhiêu cũng chỉ tương đương một mô hình tuyến tính đơn giản — xếp chồng nhiều phép tuyến tính vẫn ra một phép tuyến tính. Hàm kích hoạt phi tuyến (như ReLU) mới là thứ cho mạng khả năng học các quan hệ cong, phức tạp.

3.2.4 Phân loại đa lớp và softmax

Nhiều bài toán có *nhiều hơn hai* nhóm: nhận diện chữ số 0–9 (10 nhóm), phân loại ảnh thành nhiều loài, v.v. Đó là **phân loại đa lớp** (multiclass classification). Công cụ cho bài này là **softmax**.

Diễn giải bằng lời: lớp đầu ra có một nơ-ron cho mỗi nhóm, mỗi nơ-ron cho ra một con số thô (gọi là *logit*). Softmax biến danh sách số thô đó thành một danh sách *xác suất* không âm và cộng lại đúng bằng 1. Nhóm nào có xác suất cao nhất là dự đoán của mô hình. Nói ngắn gọn, softmax là bản mở rộng của sigmoid từ hai nhóm sang nhiều nhóm (Hình 3.5).



Hình 3.5: Kiến trúc phân loại đa lớp: các lớp ẩn ReLU, lớp đầu ra tuyến tính cho ra số thô, rồi softmax biến thành xác suất của từng nhóm.

Mẹo quan trọng: softmax ổn định số học. Nếu để lớp đầu ra tính softmax rồi mới so với nhãn, các con số trung gian có thể rất lớn hoặc rất nhỏ, gây sai số làm tròn. Cách làm được khuyến nghị trong khoá học: cho lớp đầu ra dùng kích hoạt *tuyến tính* (xuất ra số thô), rồi báo cho hàm mất mát biết đầu vào là số thô bằng tuỳ chọn `from_logits=True`. Khi đó TensorFlow gộp softmax vào trong hàm mất mát và tính theo cách ổn định hơn về mặt số học.

```
from tensorflow.keras.losses import SparseCategoricalCrossentropy

# Lớp đầu ra dùng kích hoạt tuyến tính (xuất số thô / logit)
model = Sequential([
    tf.keras.Input(shape=(2,)),
    Dense(25, activation='relu'),
    Dense(15, activation='relu'),
    Dense(4,  activation='linear'),   # 4 nhóm, xuất số thô
])

# Báo cho hàm mất mát biết đầu vào là logit -> softmax tính ổn định hơn
model.compile(
    loss=SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(0.001),
)
model.fit(X, y, epochs=10)

# Khi dự đoán, tự áp softmax lên số thô để lấy xác suất
import tensorflow as tf
logits = model(X)
probs = tf.nn.softmax(logits).numpy()
```

Lưu ý thực hành: vì lớp đầu ra giờ xuất số thô, muốn có xác suất ta phải tự áp softmax lên kết quả như dòng cuối ở trên. Bài thực hành `SoftMax` của khoá học cho thấy hai cách (tính softmax ngay ở lớp đầu ra so với dùng `from_logits=True`) ra cùng dự đoán, nhưng cách thứ hai chính xác hơn về số học và được khuyến nghị dùng.

3.2.5 Adam: tự điều chỉnh tốc độ học

Ở chương trước, chọn tốc độ học là việc thủ công và nhạy cảm: quá nhỏ thì chậm, quá lớn thì không hội tụ. **Adam** (Adaptive Moment estimation) là một bộ tối ưu thông minh hơn: nó *tự điều chỉnh* tốc độ học trong lúc huấn luyện, thậm chí dùng tốc độ riêng cho mỗi trọng số. Trực giác: nếu một trọng số cứ đi đều một hướng, Adam tăng tốc; nếu nó dao động qua lại, Adam giảm tốc. Nhờ vậy huấn luyện thường nhanh và ổn định hơn gradient descent cơ bản. Trong Keras chỉ cần truyền `optimizer=Adam(...)` khi biên dịch; đây là lựa chọn mặc định tốt cho hầu hết mạng nơ-ron.

3.2.6 Thực hành tuần 2: nhận diện chữ số 0–9

Bài tập lớn tuần 2 mở rộng từ 0/1 (tuần 1) sang nhận diện đủ *mười* chữ số 0–9, nên dùng lớp đầu ra softmax (10 nhóm). Mạng gồm hai lớp ẩn ReLU (25 và 15 nơ-ron) và lớp đầu ra 10 nơ-ron, huấn luyện với `SparseCategoricalCrossentropy(from_logits=True)` và Adam. Theo kết quả đã lưu trong bài, sau huấn luyện mô hình chỉ sai **15 ảnh trên tổng số 5000 ảnh** — một tỉ lệ lỗi rất thấp cho một mạng nhỏ. Bài thực hành cũng hiển thị một số ảnh bị đoán sai; đa phần là những chữ số viết nguệch ngoạc mà người xem cũng dễ nhầm. Điểm rút ra: với vài lớp Dense và quy trình compile/fit chuẩn, ta đã giải được một bài thị giác máy tính cỡ nhỏ.

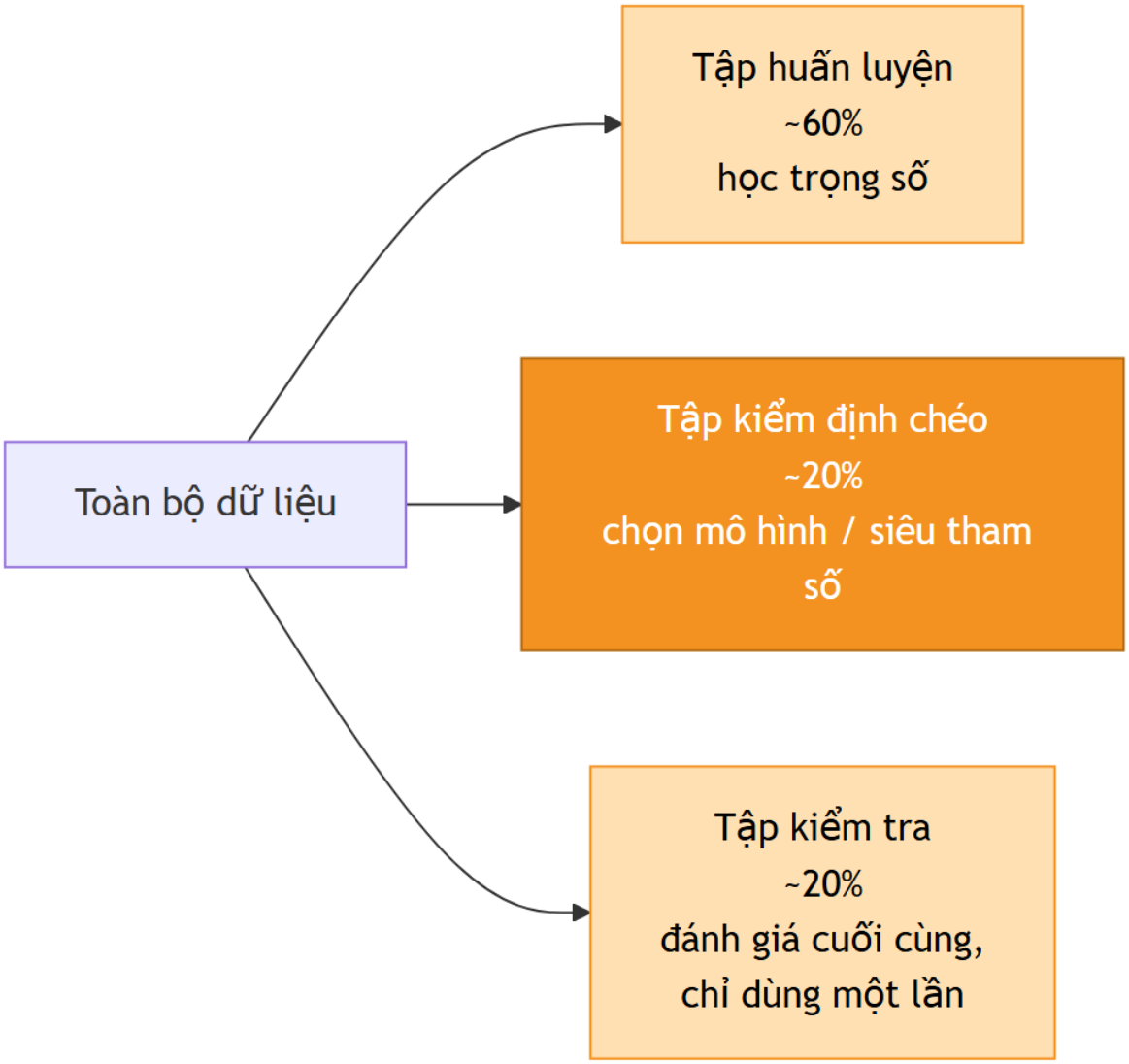
3.3 Lời khuyên thực tiễn khi áp dụng Machine Learning

Phần này có lẽ là phần "đáng tiền" nhất của cả chương: khi mô hình chạy chưa tốt, bạn nên làm gì tiếp theo? Thu thập thêm dữ liệu? Thêm hay bớt đặc trưng? Tăng hay giảm chính quy hoá? Chọn sai hướng có thể tốn hàng tháng trời. Những công cụ chẩn đoán dưới đây giúp quyết định nhanh và đúng.

3.3.1 Chia dữ liệu: train / cross-validation / test

Không thể đánh giá mô hình chỉ dựa trên dữ liệu nó đã học, vì nó có thể "học thuộc lòng". Cách làm chuẩn là chia dữ liệu thành ba phần (Hình 3.6):

- **Tập huấn luyện** (training set): để học trọng số.
- **Tập kiểm định chéo** (cross-validation / dev set): để *chọn* giữa các phương án — số lớp, số nơ-ron, mức chính quy hoá, loại mô hình. Vì ta dùng tập này để ra quyết định nên không được coi nó là thước đo khách quan cuối cùng.
- **Tập kiểm tra** (test set): chỉ dùng *một lần* ở cuối để ước lượng mô hình sẽ chạy ra sao trên dữ liệu thật, chưa từng thấy.



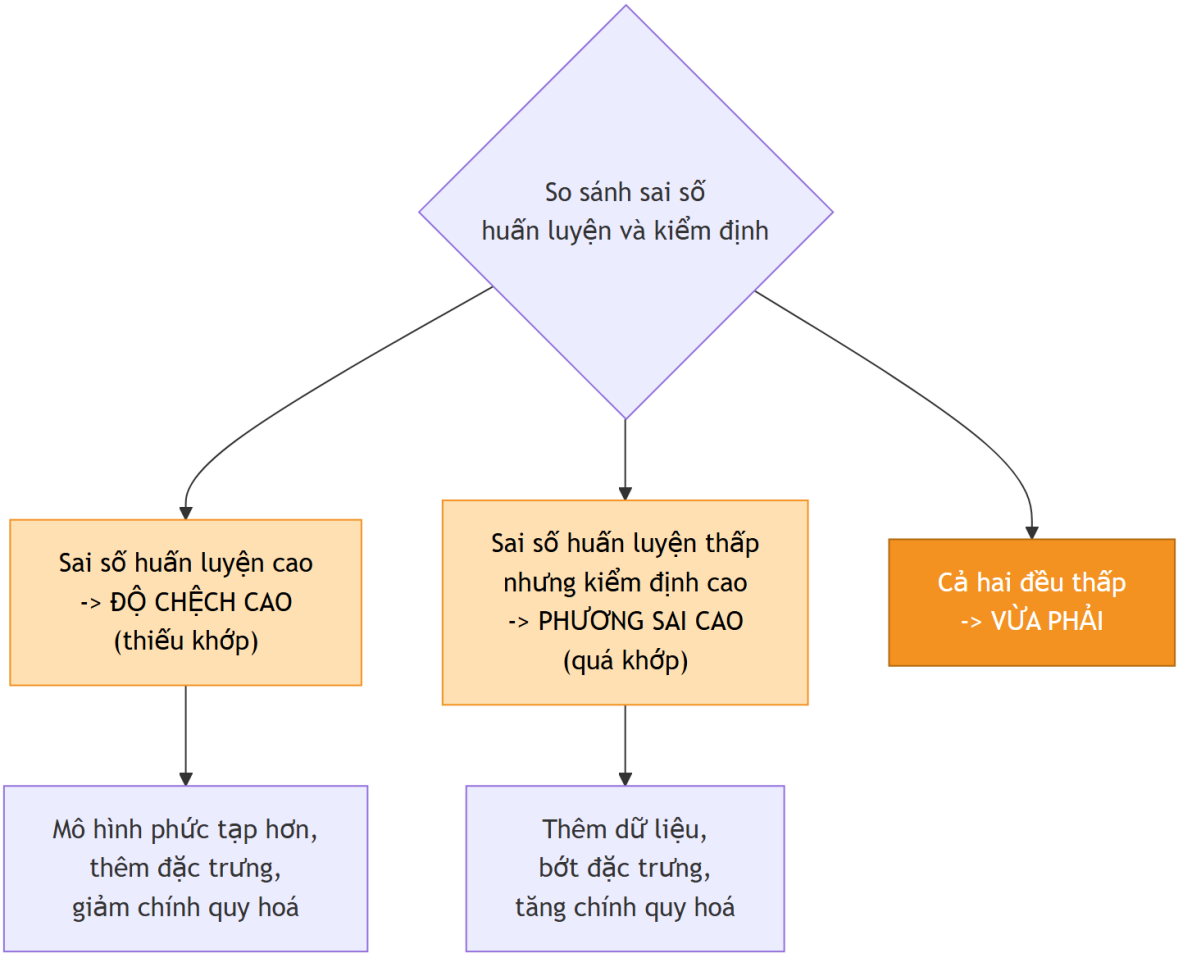
Hình 3.6: Chia dữ liệu thành ba phần với vai trò khác nhau; tập kiểm tra để dành cho đánh giá cuối cùng.

Nguyên tắc vàng: **mọi lựa chọn** đều dựa trên tập huấn luyện và kiểm định; tập kiểm tra chỉ chạm vào đúng một lần. Nếu dùng tập kiểm tra để tinh chỉnh, con số cuối cùng sẽ lạc quan giả tạo. Bài thực hành đánh giá và lựa chọn mô hình (Model Evaluation and Selection) minh hoạ đúng quy trình này: huấn luyện vài mô hình độ phức tạp khác nhau trên tập huấn luyện, chọn mô hình có sai số kiểm định thấp nhất, rồi mới báo cáo sai số trên tập kiểm tra.

3.3.2 Bias và variance: la bàn chẩn đoán

Đây là cặp khái niệm quan trọng nhất để chẩn đoán. Chỉ cần so sánh hai con số — sai số trên tập huấn luyện và sai số trên tập kiểm định — ta biết ngay vấn đề nằm ở đâu (Hình 3.7):

- **Độ chệch cao** (high bias, thiếu khớp): sai số huấn luyện đã *cao*. Mô hình quá đơn giản, học không nổi cả dữ liệu đã thấy.
- **Phương sai cao** (high variance, quá khớp): sai số huấn luyện *thấp* nhưng sai số kiểm định *cao hơn nhiều*. Mô hình học thuộc dữ liệu huấn luyện nhưng tổng quát hoá kém.
- **Vừa phải**: cả hai sai số đều thấp và gần nhau.



Hình 3.7: Chẩn đoán bias/variance bằng cách so sai số huấn luyện và kiểm định, kèm hướng xử lý tương ứng.

Cách xử lý theo chẩn đoán. Mỗi vấn đề có nhóm giải pháp ngược nhau, nên chẩn đoán đúng là then chốt:

Vấn đề	Hướng xử lý
Độ chệch cao (thiếu khớp)	Mô hình phức tạp hơn (thêm lớp/nơ-ron), thêm đặc trưng, giảm mức chính quy hoá.
Phương sai cao (quá khớp)	Thu thập thêm dữ liệu, bớt đặc trưng, tăng mức chính quy hoá.

Bảng 3.1: Giải pháp đối ứng cho độ chệch cao và phương sai cao

Lưu ý ngược đời cần nhớ: *thêm dữ liệu* giúp chữa phương sai cao nhưng gần như *không* giúp gì cho độ chệch cao — nếu mô hình quá đơn giản, cho thêm bao nhiêu dữ liệu nó vẫn khớp dở. Vì vậy đừng vội đi gom dữ liệu khi bệnh thực ra là độ chệch cao.

3.3.3 Mức chuẩn và tác động của chính quy hoá

"Sai số huấn luyện cao" là cao so với *cái gì*? Ta cần một **mức chuẩn** (baseline) để so sánh — ví dụ mức con người làm được, hay một hệ thống hiện có. Nếu con người cũng sai cỡ 10% ở một bài khó (ví dụ nghe audio nhiễu), thì mô hình sai 10% không phải độ chệch cao. Khoảng cách giữa mức chuẩn và sai số huấn luyện cho biết mức độ độ chệch; khoảng cách giữa sai số huấn luyện và sai số kiểm định cho biết mức độ phương sai.

Về **chính quy hoá**: tham số chính quy hoá điều khiển sự đánh đổi. Nếu nó quá lớn, mô hình bị ép quá đơn giản — độ chệch cao. Nếu quá nhỏ, mô hình tự do quá mức — phương sai cao. Ta thử một dải giá trị, dùng *tập kiểm định* để chọn giá trị cho sai số kiểm định thấp nhất.

3.3.4 Đường cong học

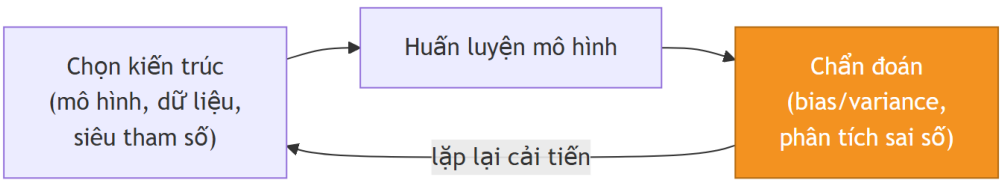
Đường cong học (learning curve) vẽ sai số theo *lượng dữ liệu huấn luyện*, cho ta nhìn trực quan về bias/variance:

- **Độ chệch cao**: khi tăng dữ liệu, hai đường sai số (huấn luyện và kiểm định) tiến lại gần nhau nhưng *cùng dừng ở mức cao*. Dấu hiệu: thêm dữ liệu không cứu được — phải đổi mô hình.
- **Phương sai cao**: có một *khoảng cách* lớn giữa sai số huấn luyện (thấp) và kiểm định (cao); khoảng cách này thu hẹp dần khi thêm dữ liệu. Dấu hiệu: thêm dữ liệu sẽ giúp ích.

Bài thực hành chẩn đoán bias/variance (Diagnosing Bias and Variance) vẽ đúng những đường cong này cho các mô hình đa thức bậc khác nhau, cho thấy bậc quá thấp dẫn tới độ chệch cao, bậc quá cao dẫn tới phương sai cao, và có một bậc "vừa phải" ở giữa.

3.3.5 Quy trình phát triển lặp

Xây một hệ thống học máy là một vòng lặp (Hình 3.8): chọn kiến trúc, huấn luyện, rồi *chẩn đoán* để biết cải tiến gì tiếp theo — và lặp lại. Lời khuyên thực chiến: hãy xây một phiên bản *đơn giản* thật nhanh, rồi dùng chẩn đoán để cải tiến dần, thay vì cố hoàn hảo ngay từ đầu.



Hình 3.8: Vòng lặp phát triển: xây nhanh một bản đơn giản, chẩn đoán, rồi cải tiến.

Phân tích sai số. Một công cụ rẻ mà hiệu quả: xem thủ công một nhóm ví dụ mà mô hình đoán *sai*, gom chúng theo chủ đề để biết loại lỗi nào phổ biến nhất. Ví dụ với bộ lọc thư rác, nếu phần lớn email bị bỏ sót là email lừa đảo bán thuốc, thì nên ưu tiên thu thập thêm dữ liệu dạng đó. Phân tích sai số giúp dồn công sức vào đúng chỗ mang lại cải thiện lớn nhất.

Thêm dữ liệu và tăng cường dữ liệu. Khi cần thêm dữ liệu, hai kỹ thuật hay dùng: **tăng cường dữ liệu** (data augmentation) — tạo thêm mẫu bằng cách biến đổi mẫu có sẵn (xoay, phóng to, thêm nhiễu cho ảnh; thêm tạp âm cho audio) sao cho vẫn giữ nhãn; và **tổng hợp dữ liệu** (data synthesis) — tạo mẫu mới từ đầu, ví dụ sinh ảnh chữ bằng các phông chữ máy tính cho bài nhận dạng ký tự.

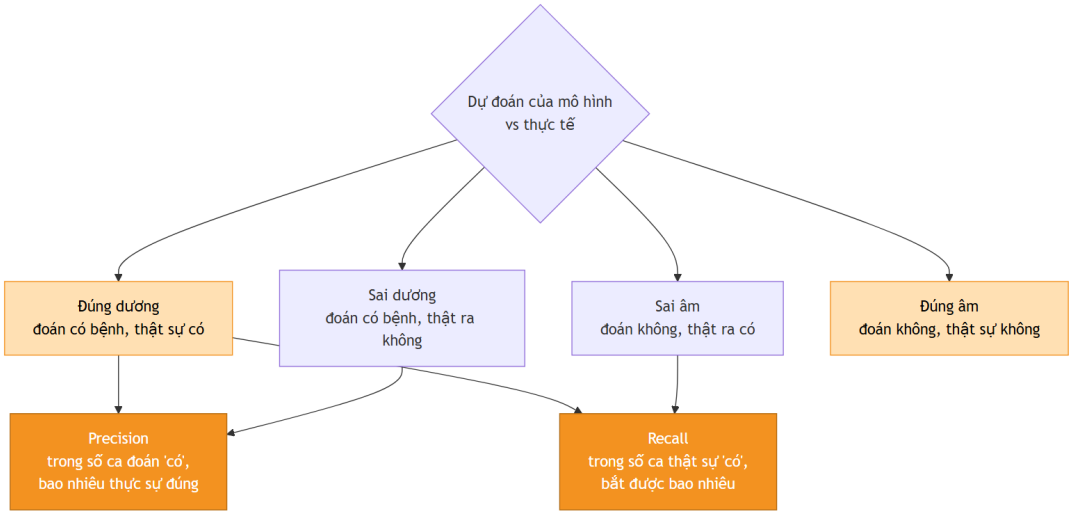
Học chuyển giao (transfer learning). Khi dữ liệu của bạn ít, có thể tận dụng một mạng đã huấn luyện sẵn trên một tập dữ liệu lớn (ví dụ hàng triệu ảnh), rồi *tinh chỉnh* cho bài toán của mình. Trực giác: các lớp đầu đã học những đặc trưng chung (cạnh, góc, hình khối) hữu ích cho nhiều bài; ta chỉ cần huấn luyện lại phần cuối cho nhiệm vụ cụ thể. Đây là cách rất hiệu quả khi dữ liệu hạn chế.

MLOps và chu trình triển khai. Mô hình tốt chưa đủ — nó còn phải được *triển khai* và vận hành. **MLOps** là tập hợp các thực hành để đưa mô hình vào sản phẩm: đóng gói mô hình thành dịch vụ, theo dõi hiệu năng khi dữ liệu thực tế thay đổi theo thời gian, và huấn luyện lại khi cần. Một mô hình đang chạy tốt có thể xuống cấp khi thế giới đổi khác, nên giám sát liên tục là bắt buộc.

3.3.6 Dữ liệu lệch: precision và recall

Khi dữ liệu *lệch nhóm nặng* (ví dụ chỉ 0,5% bệnh nhân mắc một bệnh hiếm), độ chính xác (accuracy) trở nên gây hiểu lầm: một mô hình lười luôn đoán "không bệnh" đã đúng 99,5% nhưng vô dụng. Ta cần hai thước đo tinh hơn (Hình 3.9):

- **Precision** (độ chuẩn xác): trong số các ca mô hình *đoán là dương*, bao nhiêu thực sự dương. Trả lời: "khi mô hình báo động, nó đáng tin cỡ nào?"
- **Recall** (độ bao phủ): trong số các ca *thực sự dương*, mô hình bắt được bao nhiêu. Trả lời: "mô hình bỏ sót nhiều không?"



Hình 3.9: Precision và recall được tính từ các loại đúng/sai của dự đoán; chúng phản ánh hai khía cạnh khác nhau của chất lượng mô hình.

Đánh đổi và điểm F1. Precision và recall thường *đánh đổi* nhau: nâng ngưỡng quyết định làm precision tăng nhưng recall giảm, và ngược lại. Chọn ngưỡng nào tuỳ bài toán — ví dụ sàng lọc bệnh nguy hiểm thì ưu tiên recall cao (đừng bỏ sót), còn gợi ý nội dung nhạy cảm thì ưu tiên precision. Khi cần *một* con số để so sánh các mô hình, dùng **điểm F1** — một kiểu trung bình "nghiêng về giá trị nhỏ hơn" giữa precision và recall, nên chỉ cao khi *cả hai* đều khá. F1 phạt nặng mô hình giỏi mặt này nhưng tệ mặt kia.

3.3.7 Thực hành tuần 3

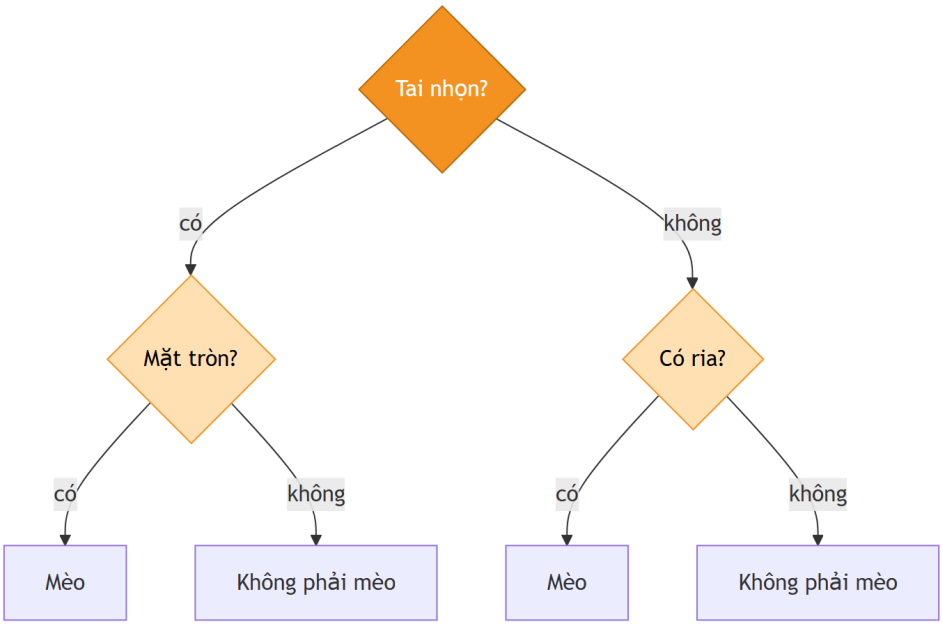
Bài tập lớn tuần 3 áp dụng toàn bộ quy trình chẩn đoán vào thực tế: chia dữ liệu train/cv/test, huấn luyện các mô hình với độ phức tạp và mức chính quy hoá khác nhau, vẽ đường cong học, đọc bias/variance để quyết định bước tiếp theo, rồi mới đánh giá trên tập kiểm tra. Thông điệp xuyên suốt: phần lớn giá trị của một kỹ sư học máy nằm ở khả năng *chẩn đoán đúng* và chọn hành động tiếp theo, chứ không chỉ ở việc huấn luyện mô hình.

3.4 Cây quyết định

Mạng nơ-ron rất mạnh với dữ liệu phi cấu trúc (ảnh, âm thanh, văn bản). Nhưng với dữ liệu *dạng bảng* (các cột số và cột danh mục, như một bảng tính), **cây quyết định** và các biến thể ensemble thường thắng về cả tốc độ lẫn độ chính xác — và rất dễ diễn giải.

3.4.1 Cây quyết định là gì

Một cây quyết định đưa ra dự đoán bằng cách hỏi một chuỗi câu hỏi "có/không". Lấy ví dụ phân loại một con vật có phải mèo không, dựa trên các đặc trưng như hình tai, hình mặt, có ria hay không (Hình 3.10). Bắt đầu từ **nút gốc** ở trên cùng, mỗi câu hỏi đưa ta rẽ xuống một nhánh, cho tới khi tới một **nút lá** chứa kết luận. Cách làm này rất giống lối suy nghĩ của con người, nên kết quả dễ giải thích.



Hình 3.10: Một cây quyết định phân loại mèo: mỗi nút trong là một câu hỏi, mỗi nút lá là một kết luận.

Thuật ngữ cơ bản:

Thuật ngữ	Ý nghĩa
Nút gốc (root)	Câu hỏi đầu tiên, ở trên cùng, chứa toàn bộ dữ liệu.
Nút trong (decision node)	Một câu hỏi dựa trên một đặc trưng, rẽ dữ liệu xuống các nhánh.
Nút lá (leaf)	Nút cuối, đưa ra dự đoán (nhóm hoặc con số).

Bảng 3.2: Thuật ngữ của cây quyết định

3.4.2 Chọn câu hỏi để chia: độ tinh khiết và information gain

Làm sao cây biết nên hỏi câu nào trước? Mục tiêu ở mỗi bước là chọn đặc trưng chia dữ liệu sao cho các nhánh con *thuần nhất* nhất có thể — lý tưởng là mỗi nhánh chỉ còn một nhóm.

Để đo mức "lộn xộn" của một nhóm dữ liệu, ta dùng khái niệm **entropy** (độ hỗn loạn), diễn giải bằng lời:

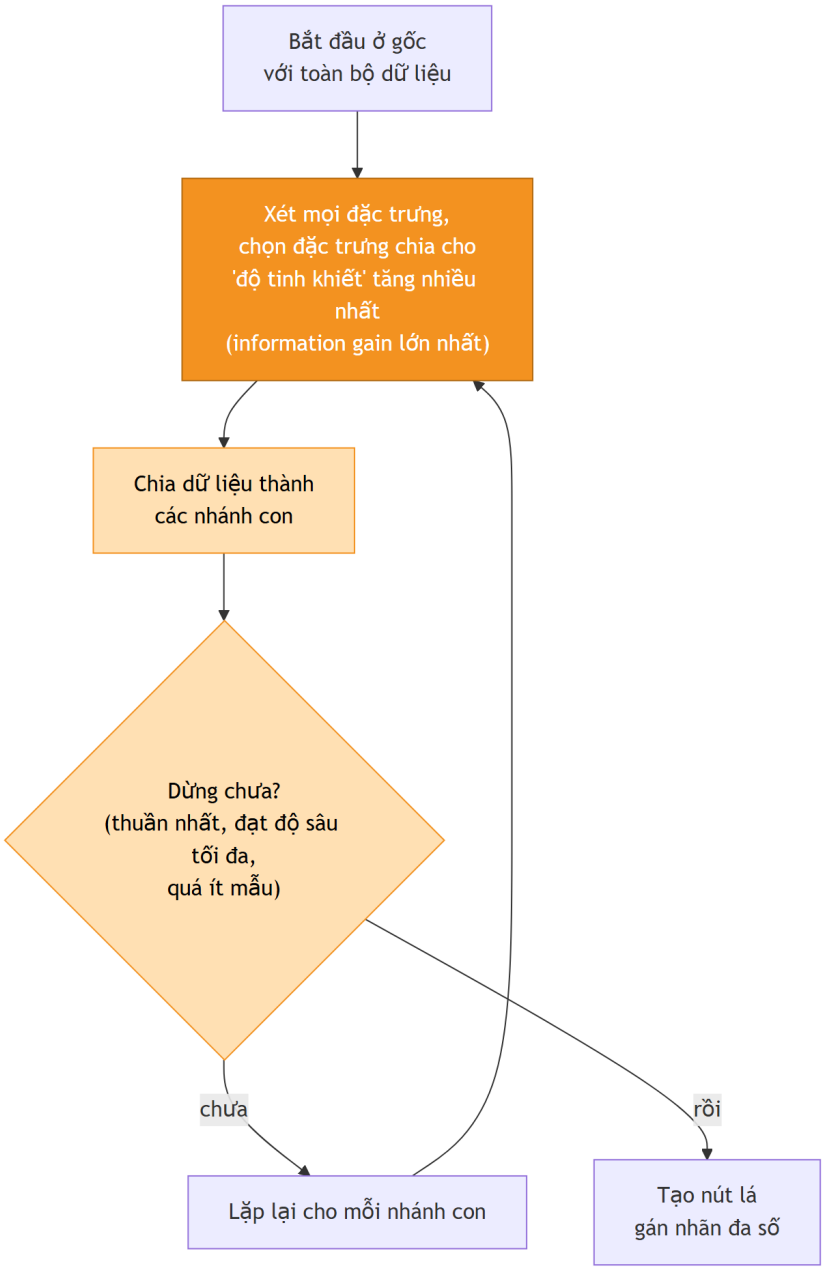
- Entropy *cao nhất* khi nhóm bị trộn lẫn 50–50 (ví dụ nửa mèo nửa không mèo) — lúc này dữ liệu "lộn xộn" nhất, khó đoán nhất.
- Entropy *bằng 0* khi nhóm hoàn toàn thuần nhất (toàn mèo, hoặc toàn không mèo) — không còn gì bất định.
- Càng lệch về một phía, entropy càng thấp.

Có thể hình dung entropy như "mức độ bất ngờ": nhóm càng cân bằng thì kết quả càng khó đoán, càng bất ngờ, entropy càng cao.

Information gain (độ lợi thông tin) là mức *giảm* entropy nhờ một phép chia. Diễn giải: lấy entropy của nhóm *trước khi* chia, trừ đi entropy trung bình (có trọng số theo số lượng mẫu) của các nhánh *sau khi* chia. Giá trị này càng lớn nghĩa là phép chia càng làm dữ liệu bớt lộn xộn — tức càng tốt. Ở mỗi nút, cây thử mọi đặc trưng và chọn đặc trưng cho information gain lớn nhất.

3.4.3 Xây cây: chia đệ quy

Thuật toán xây cây lặp đi lặp lại một việc đơn giản (Hình 3.11): ở mỗi nút, chọn phép chia tốt nhất (information gain lớn nhất), tách dữ liệu thành các nhánh, rồi *lặp lại* y hệt trên từng nhánh con. Đây là một quá trình **đệ quy**.



Hình 3.11: Quá trình xây cây đệ quy: chọn phép chia tốt nhất, tách dữ liệu, rồi lặp lại trên mỗi nhánh cho tới khi gặp điều kiện dừng.

Khi nào *dừng* chia một nhánh (để tránh cây quá sâu, quá khớp)?

- Nhánh đã thuần nhất (chỉ còn một nhóm).
- Đạt *độ sâu tối đa* cho phép (một siêu tham số ta đặt trước).
- Information gain từ phép chia tiếp theo quá nhỏ.
- Số mẫu trong nhánh còn *quá ít* để chia tiếp.

Các điều kiện dừng này đóng vai trò như chính quy hoá: giới hạn độ sâu hay yêu cầu tối thiểu số mẫu giúp cây đơn giản hơn, bớt quá khớp.

3.4.4 Đặc trưng danh mục và đặc trưng liên tục

Đặc trưng danh mục nhiều giá trị. Nếu một đặc trưng có nhiều hơn hai giá trị (ví dụ hình tai: nhọn / cụp / oval), ta dùng **one-hot encoding**: biến nó thành nhiều cột "có/không", mỗi cột cho một giá trị. Nhờ đó cây chỉ phải xử lý các câu hỏi "có/không" quen thuộc. One-hot encoding cũng chính là cách chuẩn bị dữ liệu danh mục cho cả mạng nơ-ron.

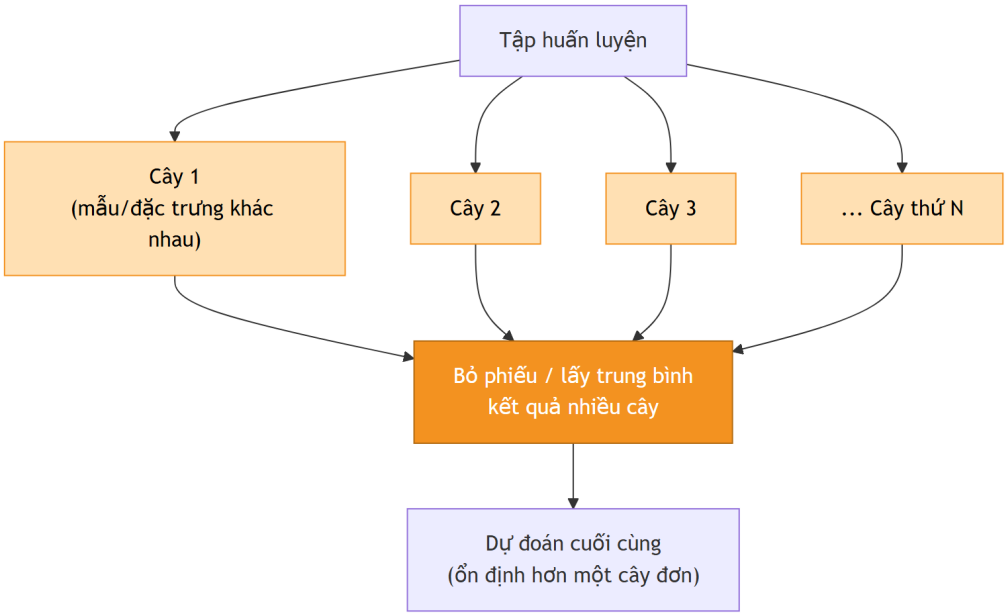
Đặc trưng liên tục (số thực). Với một đặc trưng là con số (ví dụ cân nặng), cây chọn một *ngưỡng* để chia ("cân nặng dưới 9 kg?"). Nó thử nhiều ngưỡng khác nhau và chọn ngưỡng cho information gain lớn nhất, rồi dùng ngưỡng đó như một câu hỏi "có/không".

3.4.5 Cây hồi quy

Cây quyết định không chỉ để phân loại; nó cũng *dự đoán số* (cây hồi quy). Khác biệt duy nhất: thay vì giảm entropy, phép chia nhằm giảm *độ phân tán* của giá trị số trong mỗi nhánh (muốn mỗi nhánh gom các mẫu có giá trị gần nhau). Dự đoán tại một nút lá là *trung bình* các giá trị mục tiêu rơi vào lá đó.

3.4.6 Ensemble: nhiều cây mạnh hơn một cây

Một cây đơn lẻ khá *nhạy cảm*: đổi dữ liệu một chút là cây có thể đổi hẳn, dẫn tới phương sai cao. Giải pháp là **ensemble** — huấn luyện *nhiều* cây rồi cho chúng bỏ phiếu (phân loại) hoặc lấy trung bình (hồi quy). Nhiều cây gộp lại cho dự đoán ổn định và chính xác hơn hẳn một cây (Hình 3.12).



Hình 3.12: Ensemble: nhiều cây được huấn luyện trên các phần dữ liệu/đặc trưng khác nhau, rồi gộp kết quả để cho dự đoán ổn định hơn.

Random forest (rừng ngẫu nhiên). Ý tưởng: tạo nhiều cây, mỗi cây huấn luyện trên một *mẫu ngẫu nhiên* của dữ liệu, và tại mỗi phép chia chỉ xét một *tập con ngẫu nhiên* các đặc trưng. "Ngẫu nhiên" theo hai cách này khiến các cây khác nhau đủ để khi gộp lại chúng bù trừ sai sót cho nhau.

Boosting và XGBoost. **Boosting** xây cây *tuần tự*: mỗi cây mới tập trung nhiều hơn vào những mẫu mà các cây trước còn làm sai. Giống như ôn tập có trọng tâm: càng về sau càng luyện kỹ những chỗ hay sai. **XGBoost** là thư viện boosting cây được dùng rất rộng rãi, nổi tiếng vì nhanh, chính xác, và thường thắng trong các cuộc thi học máy trên dữ liệu dạng bảng.

3.4.7 Thực hành tuần 4

Bài thực hành tuần 4 dùng scikit-learn và XGBoost để dựng cây và ensemble bằng vài dòng. Lưu ý: các kết quả in ra trong những lab này đã bị xoá khỏi notebook lưu lại, nên dưới đây chỉ mô tả cách dùng và các *siêu tham số đã chọn* trong bài, không trích số liệu đo đạc nào.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# Một cây đơn: giới hạn độ sâu và số mẫu tối thiểu để tránh quá khớp
tree = DecisionTreeClassifier(max_depth=3, min_samples_split=50)
tree.fit(X_train, y_train)

# Rừng ngẫu nhiên: nhiều cây gộp lại
forest = RandomForestClassifier(
    n_estimators=100, max_depth=8, min_samples_split=10
)
forest.fit(X_train, y_train)

# XGBoost: dừng sớm khi sai số kiểm định không giảm thêm
xgb = XGBClassifier(n_estimators=500, learning_rate=0.1)
xgb.fit(X_train, y_train,
        eval_set=[(X_cv, y_cv)], early_stopping_rounds=10)
# Trong bài, XGBoost dừng tốt nhất quanh vòng lặp thứ 16 (best_iteration)
```

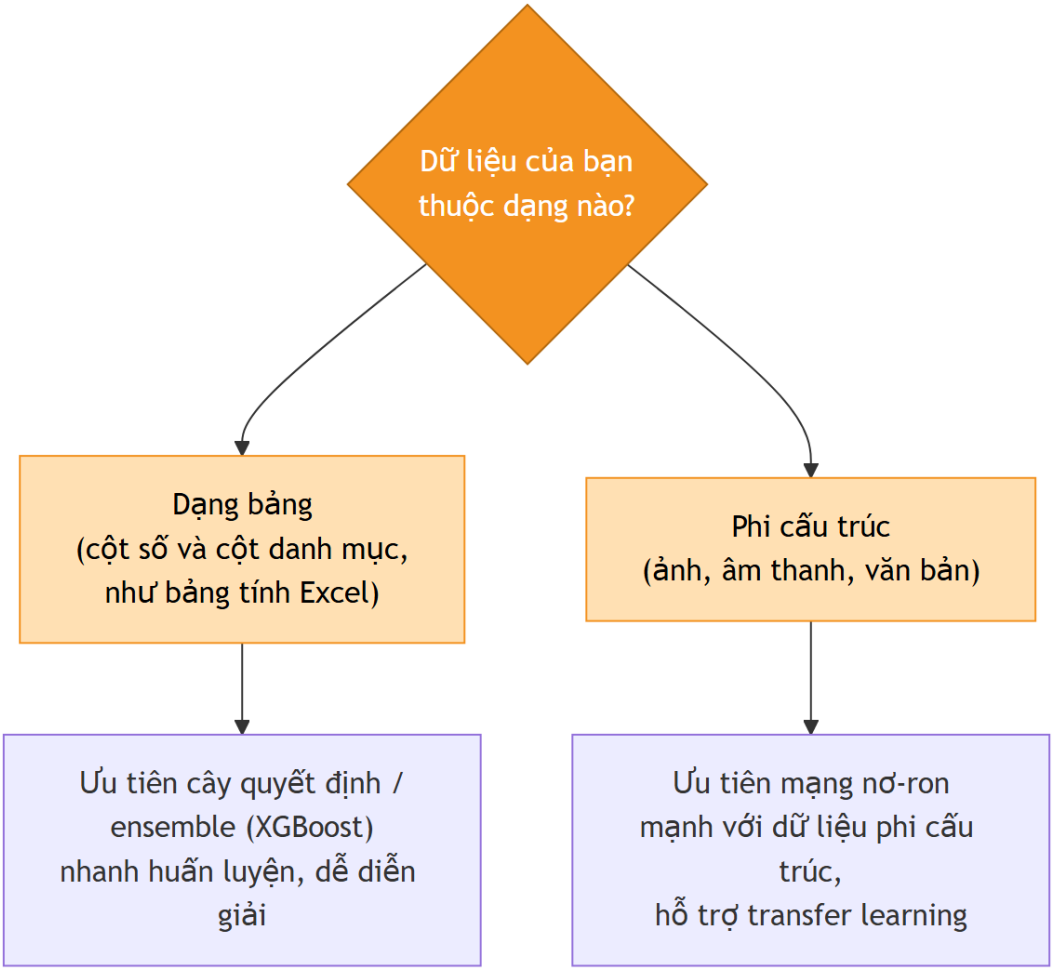
Qua các lệnh trên có thể thấy mẫu dùng rất giống nhau giữa các mô hình (`fit` rồi `predict`), và các siêu tham số chính đều nhằm kiểm soát độ phức tạp: `max_depth` và `min_samples_split` giới hạn mỗi cây; `n_estimators` đặt số cây; còn XGBoost dùng *dừng sớm* (early stopping) để tự chọn số cây tốt nhất dựa trên tập kiểm định.

3.4.8 Cây quyết định so với mạng nơ-ron

Khi nào dùng cái nào? Quy tắc thực chiến dựa chủ yếu vào *dạng dữ liệu* (Hình 3.13):

Tiêu chí	Cây / Ensemble	Mạng nơ-ron
Dữ liệu bảng (số, danh mục)	Rất hợp, thường là lựa chọn đầu tiên	Dùng được nhưng ít lợi thế
Dữ liệu phi cấu trúc (ảnh, âm thanh, văn bản)	Ít hợp	Rất mạnh
Tốc độ huấn luyện	Nhanh (nhất là một cây hay random forest)	Thường chậm hơn
Khả năng diễn giải	Một cây nhỏ rất dễ giải thích	Khó diễn giải (hộp đen)
Học chuyển giao	Không hỗ trợ	Hỗ trợ tốt, tận dụng mô hình huấn luyện sẵn
Ghép nhiều mô hình trong một hệ thống	Khó nối chuỗi	Dễ ghép nhiều mạng lại với nhau

Bảng 3.3: So sánh cây quyết định/ensemble và mạng nơ-ron



Hình 3.13: Chọn giữa cây và mạng nơ-ron chủ yếu dựa vào dạng dữ liệu: bảng thì ưu tiên cây/ ensemble, phi cấu trúc thì ưu tiên mạng nơ-ron.

Tóm lại: với dữ liệu dạng bảng, hãy thử ensemble cây (đặc biệt XGBoost) trước — nhanh và mạnh. Với ảnh, âm thanh, văn bản, hoặc khi cần học chuyển giao và ghép nhiều mô hình, mạng nơ-ron là lựa chọn tự nhiên.

3.5 Tổng kết chương

Chương này đã mở rộng bộ công cụ học máy lên tầm nâng cao:

- **Mạng nơ-ron — suy luận**: mỗi nơ-ron là một mô hình nhỏ; xếp thành lớp, đầu ra lớp này là đầu vào lớp sau; lan truyền xuôi tạo dự đoán; dựng và suy luận bằng `Sequential` + `Dense` trong TensorFlow/Keras.
- **Mạng nơ-ron — huấn luyện**: quy trình ba bước định nghĩa / biên dịch / huấn luyện; chọn hàm kích hoạt (ReLU cho lớp ẩn; sigmoid, softmax hay linear cho lớp đầu ra tuỳ bài toán); softmax cho đa lớp với mẹo `from_logits=True` để ổn định số học; Adam để tối ưu nhanh và ổn định.
- **Lời khuyên thực tiễn**: chia train/cv/test đúng cách; chẩn đoán bias/variance để biết nên làm gì tiếp; đường cong học; mức chuẩn; phân tích sai số; tăng cường dữ liệu và học chuyển giao; MLOps; precision/recall/F1 cho dữ liệu lệch.

- **Cây quyết định**: chia theo độ tinh khiết (entropy, information gain); xây cây đệ quy với điều kiện dừng; one-hot cho danh mục, ngưỡng cho đặc trưng liên tục; cây hồi quy; ensemble (random forest, XGBoost); và cách chọn giữa cây và mạng nơ-ron theo dạng dữ liệu.

Thông điệp xuyên suốt vẫn là: hiểu trực giác để chọn đúng công cụ, dùng framework trưởng thành (TensorFlow/Keras, scikit-learn, XGBoost) thay vì tự cài đặt, và — quan trọng nhất — biết *chẩn đoán* để quyết định bước đi tiếp theo trong một vòng lặp phát triển thực tế.

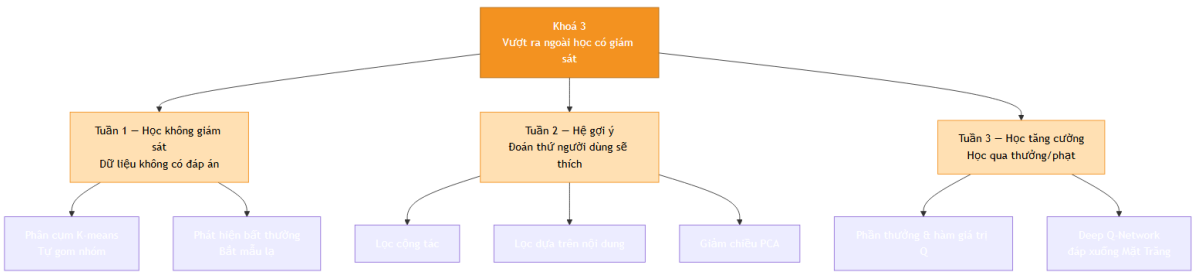
Chương 4

Unsupervised Learning, Recommenders, Reinforcement Learning

Hai chương trước xoay quanh **học có giám sát** — nơi mỗi mẫu dữ liệu đều kèm sẵn đáp án đúng để mô hình bắt chước. Chương này bước sang ba nhóm kỹ thuật *vượt ra ngoài* khuôn khổ đó, mở rộng bộ công cụ của bạn theo ba hướng rất khác nhau:

- **Học không giám sát** (unsupervised learning): dữ liệu không có đáp án, thuật toán tự tìm cấu trúc — gom nhóm (phân cụm) hoặc bắt những mẫu lạ (phát hiện bất thường).
- **Hệ gợi ý** (recommender systems): đoán xem người dùng sẽ thích món nào, công nghệ đứng sau các trang mua sắm, xem phim, nghe nhạc, và cả quảng cáo trực tuyến.
- **Học tăng cường** (reinforcement learning): học qua thưởng và phạt, không ai chỉ cho hành động đúng mà chỉ nói kết quả tốt hay xấu — cách người ta dạy robot, dạy máy chơi game, và trong chương này là đáp một tàu vũ trụ mô phỏng xuống Mặt Trăng.

Cách tiếp cận vẫn như trước: *ý tưởng trước, công cụ sau*. Ta nắm trực giác vì sao mỗi kỹ thuật ra đời, rồi áp dụng qua thư viện như scikit-learn và TensorFlow/Keras thay vì tự cài đặt lại từ đầu. Mọi công thức toán học được diễn giải bằng lời và sơ đồ; thay vì ký hiệu, ta gọi tên các đại lượng theo đúng ý nghĩa ("tâm cụm", "độ khả dĩ", "phần thưởng", "lợi ích tích luỹ", "giá trị hành động"…) để nắm logic mà không bị rào cản ký hiệu.



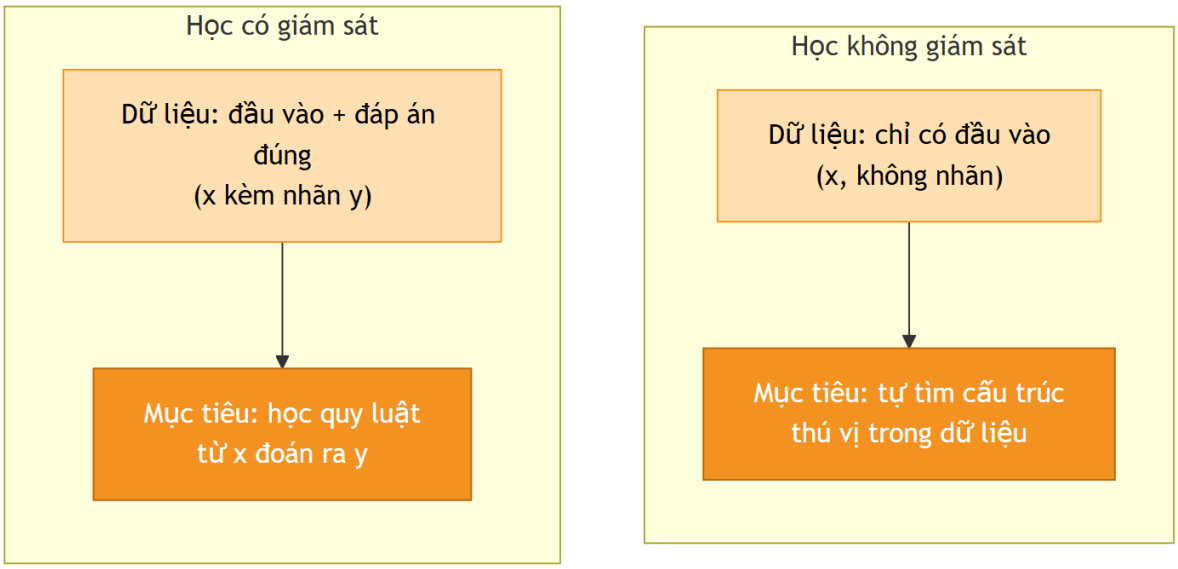
Hình 4.1: Bản đồ chương: ba tuần, ba nhóm kỹ thuật đi xa hơn học có giám sát.

4.1 Học không giám sát

4.1.1 Học không giám sát là gì

Khác biệt cốt lõi so với học có giám sát nằm ở *dữ liệu*. Trong học có giám sát, mỗi mẫu đầu vào đi kèm một nhãn — đáp án đúng mà ta muốn mô hình học cách dự đoán. Trong học không giám sát, ta *chỉ có*

đầu vào, không có nhãn. Vì không có "đáp án đúng" để chỉ cho thuật toán, ta đổi câu hỏi: thay vì "hãy dự đoán y", ta hỏi "hãy tìm điều gì đó thú vị, một cấu trúc nào đó, trong dữ liệu này" (Hình 4.2).



Hình 4.2: Hai thế giới: học có giám sát có nhãn để bắt chước; học không giám sát tự khám phá cấu trúc từ dữ liệu trần.

Chương này tập trung vào hai dạng cấu trúc hữu ích và được dùng rộng rãi trong thương mại: **phân cụm** (gom các điểm giống nhau thành nhóm) và **phát hiện bất thường** (bắt những mẫu khác lạ so với phần còn lại).

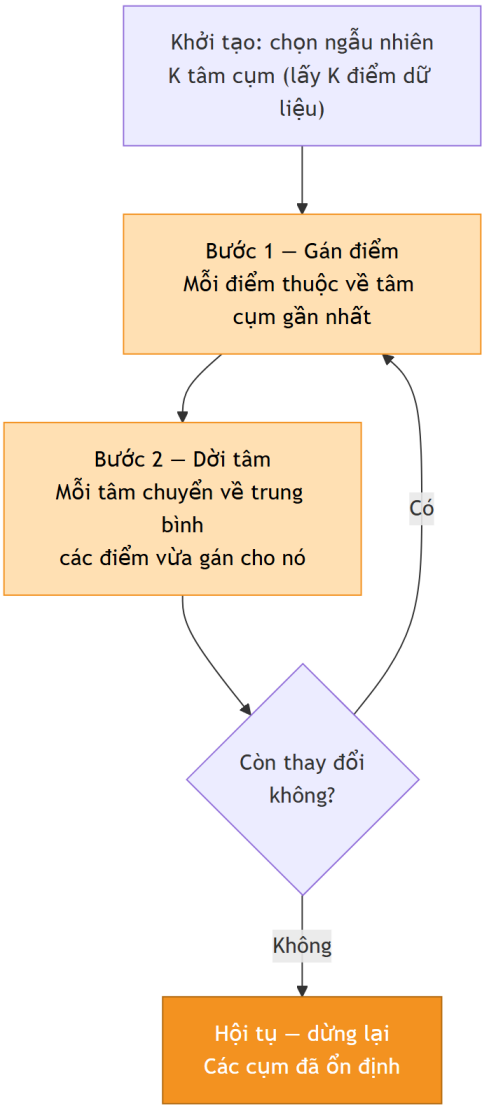
4.1.2 Phân cụm K-means: trực giác

Phân cụm (clustering) nhìn vào một đống điểm dữ liệu và tự tìm ra các nhóm điểm giống nhau. Ứng dụng thực tế rất nhiều: gom các bài báo cùng chủ đề, phân khúc khách hàng (ví dụ deeplearning.ai nhận ra người học đến vì các mục tiêu khác nhau — nâng kỹ năng, phát triển sự nghiệp, cập nhật về AI), phân tích dữ liệu gen theo đặc điểm di truyền, hay nhóm các thiên thể trong thiên văn học.

Thuật toán phân cụm phổ biến nhất là **K-means**. Trực giác của nó gói gọn trong hai việc lặp đi lặp lại:

1. **Gán điểm cho tâm cụm**: mỗi điểm dữ liệu được gắn với *tâm cụm* gần nó nhất. (Tâm cụm — cluster centroid — là điểm đại diện cho tâm của một nhóm.)
2. **Dời tâm cụm**: mỗi tâm chuyển về vị trí trung bình của tất cả các điểm vừa được gán cho nó.

Khởi đầu, K-means đặt các tâm cụm ở vị trí ngẫu nhiên. Sau đó nó lặp hai bước trên: tô màu mỗi điểm theo tâm gần nhất, rồi dời mỗi tâm về trung bình nhóm màu của mình. Cứ thế cho tới khi không còn gì thay đổi — lúc đó ta nói thuật toán đã **hội tụ** (Hình 4.3).



Hình 4.3: Vòng lặp K-means: gán điểm cho tâm gần nhất, rồi dời tâm về trung bình nhóm, lặp đến khi ổn định.

K-means hoạt động tốt cả khi các cụm *không* tách bạch rõ ràng. Ví dụ kinh điển: một hãng áo phông muốn chia cỡ S, M, L. Chiều cao và cân nặng của người mua thay đổi liên tục, không có nhóm rõ rệt, nhưng K-means với ba tâm cụm vẫn chia dữ liệu thành ba nhóm hợp lý, mỗi tâm cho biết số đo đại diện để thiết kế một cỡ áo.

K-means đang tối ưu cái gì. Dù không dùng gradient descent, K-means vẫn đang giảm dần một đại lượng gọi là **độ méo** (distortion): trung bình khoảng cách bình phương từ mỗi điểm tới tâm cụm của nó. Bước gán điểm làm giảm độ méo bằng cách chọn tâm gần nhất; bước dời tâm làm giảm độ méo bằng cách đặt tâm vào đúng trung bình nhóm. Vì mỗi bước đều kéo độ méo xuống (hoặc giữ nguyên), thuật toán được bảo đảm hội tụ. Nếu độ méo bỗng *tăng*, chắc chắn code có lỗi. Khi độ méo thôi giảm, đó là dấu hiệu đã hội tụ.

Khởi tạo ngẫu nhiên nhiều lần. Cách khởi tạo tâm cụm phổ biến là chọn ngẫu nhiên K điểm dữ liệu làm tâm ban đầu. Nhưng tuỳ cú chọn may rủi, K-means có thể kẹt ở một *cực tiểu địa phương* — một cách gom nhóm trông không ổn. Giải pháp đơn giản mà hiệu quả: chạy K-means nhiều lần (thường 50–1000

lần) với các khởi tạo khác nhau, mỗi lần tính độ méo, rồi *giữ lại kết quả có độ méo nhỏ nhất*. Đây gần như luôn là việc nên làm.

Chọn số cụm K bao nhiêu. Thường thì số cụm *thực sự mơ hồ* — cùng một dữ liệu, người thấy hai cụm, người thấy bốn, cả hai đều có lý. Có kỹ thuật "khuỷu tay" (elbow method): vẽ độ méo theo K và tìm chỗ đường cong gãy khúc, nhưng nhiều bài toán đường cong trơn tru không có khuỷu rõ. Lưu ý: *không* được chọn K để tối thiểu hoá độ méo, vì càng nhiều cụm độ méo càng nhỏ, ta sẽ luôn chọn K lớn nhất. Lời khuyên thực chiến: chọn K dựa trên *mục đích phía sau*. Với bài áo phông, chạy cả K = 3 và K = 5 rồi cân nhắc đánh đổi giữa độ vừa vặn và chi phí sản xuất thêm cỡ.

4.1.3 K-means với scikit-learn

Hai bước cốt lõi của K-means viết bằng NumPy rất gọn. Bài thực hành cài đặt chúng tay để hiểu bản chất — gán điểm cho tâm gần nhất, rồi tính lại tâm bằng trung bình nhóm:

```
import numpy as np

def find_closest_centroids(X, centroids):
    # Mỗi điểm X[i] -> chỉ số tâm cụm gần nhất
    idx = np.zeros(X.shape[0], dtype=int)
    for i in range(X.shape[0]):
        distances = np.linalg.norm(X[i] - centroids, axis=1)  # khoảng cách tới
        ↪  mỗi tâm
        idx[i] = np.argmin(distances)                         # chọn tâm gần nhất
    return idx

def compute_centroids(X, idx, K):
    # Dời mỗi tâm về trung bình các điểm thuộc cụm đó
    m, n = X.shape
    centroids = np.zeros((K, n))
    for k in range(K):
        points = X[idx == k]
        if len(points) > 0:
            centroids[k] = np.mean(points, axis=0)
    return centroids
```

Trong thực tế, ta hiếm khi tự viết vòng lặp — scikit-learn làm sẵn và tối ưu tốt. Khởi tạo nhiều lần được bật qua tham số `n_init`:

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, n_init=10, random_state=0)  # chạy 10 khởi tạo, giữ
↪  bản tốt nhất
kmeans.fit(X)
labels = kmeans.labels_          # mỗi điểm thuộc cụm nào
centers = kmeans.cluster_centers_  # vị trí các tâm cụm
```

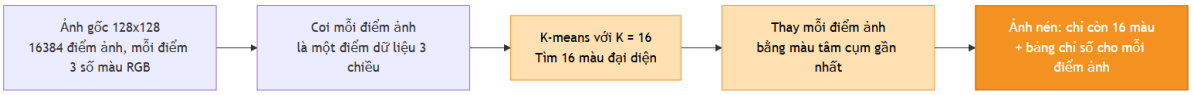
Ứng dụng vui: nén ảnh. Bài thực hành dùng K-means để nén ảnh — một minh hoạ trực quan rất đẹp. Một bức ảnh màu lưu mỗi điểm ảnh bằng ba số (đỏ, lục, lam). Coi mỗi điểm ảnh là một điểm dữ liệu 3 chiều, chạy K-means để tìm K màu đại diện, rồi thay mỗi điểm ảnh bằng màu tâm cụm gần nó nhất (Hình 4.4):

```
# original_img có kích thước (128, 128, 3) -> 16384 điểm ảnh, mỗi điểm 3 số màu
X_img = np.reshape(original_img, (128 * 128, 3))
```

```
K = 16            # nén xuống còn 16 màu
# ... chạy K-means trên X_img để tìm 16 tâm màu (centroids) ...

idx = find_closest_centroids(X_img, centroids)   # mỗi điểm ảnh -> màu gần nhất
X_recovered = centroids[idx, :]                   # thay bằng màu tâm cụm
X_recovered = np.reshape(X_recovered, original_img.shape)  # dựng lại ảnh
```

Trong bài, ảnh gốc 128×128 gồm 16384 điểm ảnh được nén xuống còn **16 màu**. Thay vì lưu màu đầy đủ cho từng điểm ảnh, ta chỉ cần lưu 16 màu đại diện cộng một bảng chỉ số cho biết mỗi điểm ảnh dùng màu nào — đó chính là sự tiết kiệm. Số màu K điều khiển đánh đổi giữa chất lượng ảnh và mức nén.

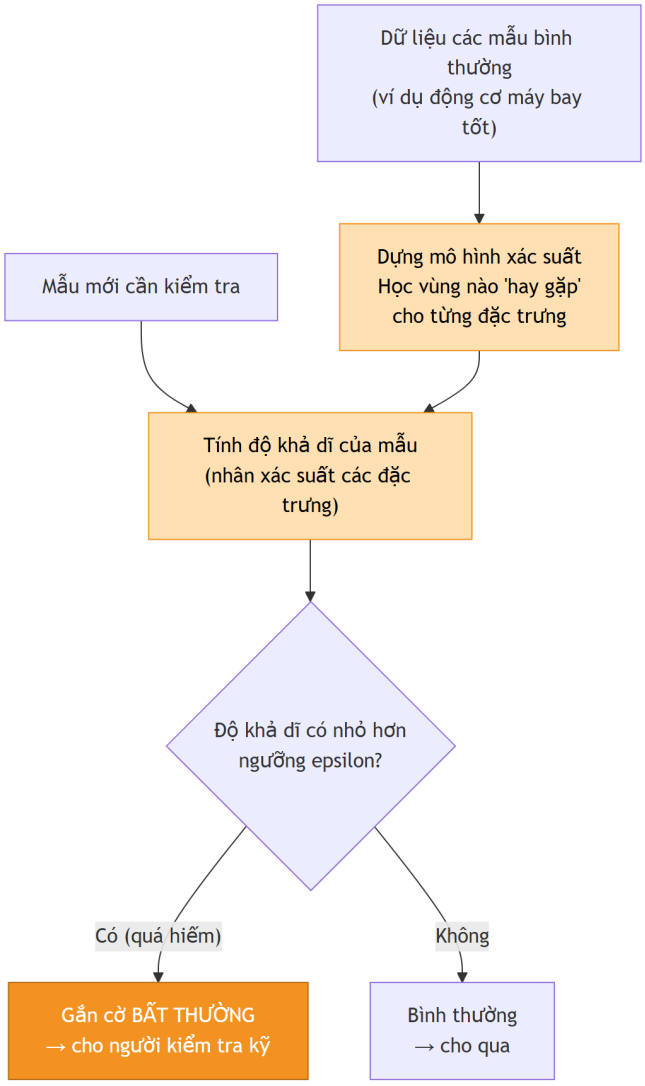


Hình 4.4: Nén ảnh bằng K-means: mỗi điểm ảnh là một điểm dữ liệu màu; K-means tìm 16 màu đại diện và thay thế toàn bộ điểm ảnh bằng chúng.

4.1.4 Phát hiện bất thường: trực giác

Phát hiện bất thường (anomaly detection) nhìn vào một tập dữ liệu gồm các sự kiện *bình thường* và học cách gắn cờ báo động khi gặp một sự kiện bất thường. Ví dụ kinh điển: kiểm tra động cơ máy bay sau khi sản xuất. Nhà máy làm ra rất nhiều động cơ tốt, nên dễ thu thập dữ liệu về cách động cơ bình thường vận hành (ví dụ nhiệt độ sinh ra và độ rung). Khi một động cơ mới xuất xưởng, ta hỏi: nó có giống những động cơ đã thấy không, hay có gì đó rất lạ cần kiểm tra kỹ trước khi lắp lên máy bay?

Cách làm phổ biến nhất là **ước lượng mật độ** (density estimation). Từ dữ liệu bình thường, thuật toán dựng một mô hình cho biết vùng giá trị nào *hay gặp* (độ khả dĩ cao) và vùng nào *hiếm gặp* (độ khả dĩ thấp). Khi có mẫu mới, ta tính độ khả dĩ của nó. Nếu độ khả dĩ *nhỏ hơn một ngưỡng* (ta gọi ngưỡng này là epsilon), mẫu đó bị gắn cờ bất thường (Hình 4.5).



Hình 4.5: Luồng phát hiện bất thường: học "bình thường trông ra sao", rồi gắn cờ mẫu nào có độ khả dĩ thấp hơn ngưỡng.

Phát hiện bất thường được dùng khắp nơi: phát hiện gian lận (hành vi người dùng lạ trên website, giao dịch tài chính bất thường), giám sát sản xuất (bắt sản phẩm lỗi), và theo dõi máy chủ trong trung tâm dữ liệu (một máy chạy khác thường có thể do hỏng ổ cứng hoặc bị tấn công).

Phân phối chuẩn — nền tảng của mô hình. Để mô hình hoá "vùng nào hay gặp", ta dùng **phân phối chuẩn** (Gaussian, hay phân phối hình chuông). Trực giác: nếu lấy rất nhiều số từ một phân phối chuẩn và vẽ biểu đồ tần suất, ta được một đường cong hình chuông — đỉnh ở giá trị trung bình, bề rộng do *độ lệch chuẩn* quyết định. Trung bình lớn thì đỉnh dịch sang phải; độ lệch chuẩn nhỏ thì chuông hẹp và cao, độ lệch chuẩn lớn thì chuông thấp và bè ra. Từ dữ liệu, ta ước lượng hai tham số này một cách đơn giản: trung bình chính là trung bình cộng của các mẫu, còn độ lệch (phương sai) là trung bình của bình phương khoảng cách từ mỗi mẫu tới trung bình đó.

Từ một đặc trưng tới nhiều đặc trưng. Bài toán thực tế có nhiều đặc trưng. Mô hình xử lý bằng cách khớp một phân phối chuẩn *riêng* cho mỗi đặc trưng, rồi *nhân* các độ khả dĩ lại với nhau để ra độ khả dĩ chung của cả mẫu. Ý nghĩa trực giác của phép nhân: một mẫu bị coi là bất thường nếu *dù chỉ một* đặc trưng của nó có giá trị quá lớn hoặc quá nhỏ so với dữ liệu bình thường — khi đó độ khả dĩ của đặc trưng ấy rất nhỏ, kéo tích chung xuống thấp và mẫu bị gắn cờ.

4.1.5 Phát hiện bất thường trong thực hành

Hai bước ước lượng tham số viết bằng NumPy rất ngắn — tính trung bình và phương sai cho mỗi đặc trưng:

```
import numpy as np

def estimate_gaussian(X):
    # Trả về trung bình (mu) và phương sai (var) cho từng đặc trưng
    m, n = X.shape
    mu  = (1 / m) * np.sum(X, axis=0)
    var = (1 / m) * np.sum((X - mu) ** 2, axis=0)
    return mu, var
```

Chọn ngưỡng epsilon và đánh giá. Dù về bản chất đây là học không giám sát (chỉ học từ dữ liệu bình thường), ta vẫn cần một ít mẫu bất thường có nhãn để *đánh giá* và chỉnh ngưỡng. Thực tế thường dùng một tập kiểm định (cross-validation) chứa phần lớn mẫu bình thường và một nhúm mẫu bất thường. Ta quét thử nhiều giá trị epsilon, mỗi lần đo chất lượng và giữ lại epsilon tốt nhất. Vì dữ liệu lệch nặng (rất ít mẫu bất thường), ta dùng **điểm F1** thay cho độ chính xác — đúng như đã bàn ở chương trước:

```
def select_threshold(y_val, p_val):
    best_epsilon, best_F1 = 0, 0
    step_size = (max(p_val) - min(p_val)) / 1000
    for epsilon in np.arange(min(p_val), max(p_val), step_size):
        predictions = (p_val < epsilon)                        # dưới ngưỡng ->
        ↪  bất thường
        tp = np.sum((predictions == 1) & (y_val == 1))         # đúng dương
        fp = np.sum((predictions == 1) & (y_val == 0))         # sai dương
        fn = np.sum((predictions == 0) & (y_val == 1))         # sai âm
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        rec  = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        F1 = (2 * prec * rec) / (prec + rec) if (prec + rec) > 0 else 0.0
        if F1 > best_F1:
            best_F1, best_epsilon = F1, epsilon
    return best_epsilon, best_F1
```

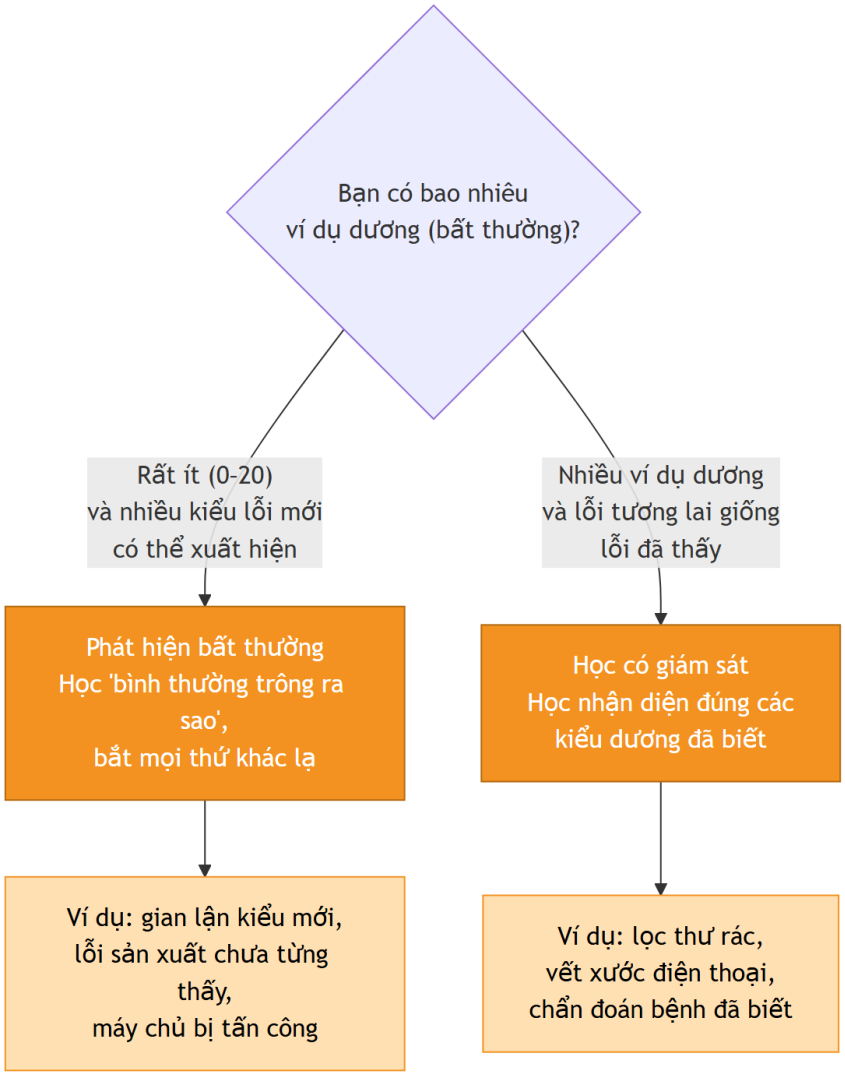
Trong bài thực hành phát hiện máy chủ lỗi (hai đặc trưng: thông lượng và độ trễ), quy trình này tìm được ngưỡng epsilon tốt nhất khoảng $8{,}99 \times 10^{-5}$ với điểm F1 trên tập kiểm định là $0{,}875$. Khi mở rộng sang bộ dữ liệu 11 đặc trưng, ngưỡng tốt nhất nhỏ hơn rất nhiều (khoảng $1{,}38 \times 10^{-18}$) với F1 là $0{,}615$, và thuật toán gắn cờ 117 mẫu bất thường. (Các con số này lấy trực tiếp từ kết quả đã lưu trong notebook.)

Mẹo về đặc trưng. Với phát hiện bất thường, chọn đặc trưng còn quan trọng hơn học có giám sát, vì không có nhãn để thuật toán tự biết nên bỏ qua đặc trưng nào. Hai mẹo thực chiến: (1) làm cho đặc trưng *gần hình chuông* hơn bằng các phép biến đổi như lấy log hay căn (nhớ áp dụng cùng phép biến đổi cho tập kiểm định và kiểm tra); (2) khi mô hình bỏ sót một mẫu bất thường, hãy xem mẫu đó và nghĩ ra *đặc trưng mới* làm nó nổi bật — ví dụ tạo đặc trưng tỉ lệ "tải CPU chia cho lưu lượng mạng" để bắt máy chủ tải cao mà lưu lượng thấp.

4.1.6 Phát hiện bất thường hay học có giám sát?

Khi đã có một ít mẫu dương (bất thường) có nhãn, vì sao không dùng luôn học có giám sát? Câu trả lời phụ thuộc vào bản chất bài toán (Hình 4.6):

- Chọn **phát hiện bất thường** khi có *rất ít* mẫu dương (thường 0–20) và có thể xuất hiện *nhiều kiểu lỗi mới* chưa từng thấy. Vì nó chỉ học "bình thường trông ra sao", nó bắt được cả những dạng lỗi hoàn toàn mới. Hợp với gian lận kiểu mới, lỗi sản xuất chưa từng gặp, máy chủ bị tấn công.
- Chọn **học có giám sát** khi có *nhiều* mẫu dương và lỗi tương lai nhiều khả năng *giống* lỗi đã thấy. Hợp với lọc thư rác, nhận diện vết xước điện thoại, chẩn đoán bệnh đã biết.



Hình 4.6: Khung quyết định: ít mẫu dương và nhiều kiểu lỗi mới thì dùng phát hiện bất thường; nhiều mẫu dương và lỗi lặp lại thì dùng học có giám sát.

4.2 Hệ gợi ý

Hệ gợi ý là một trong những công nghệ học máy có giá trị thương mại lớn nhất — đứng sau gợi ý sản phẩm trên trang mua sắm, gợi ý phim trên dịch vụ xem phim, gợi ý nhạc, và cả hệ thống quảng cáo. Với nhiều công ty, một phần lớn doanh thu đến từ hệ gợi ý. Ta sẽ dùng ví dụ xuyên suốt: dự đoán điểm người dùng chấm cho phim (một đến năm sao), từ đó gợi ý những phim họ có khả năng thích.

Bối cảnh: ta có một số người dùng và một số phim; mỗi người đã chấm điểm *một phần* phim, còn nhiều ô để trống. Mục tiêu là dự đoán các ô trống ấy để biết nên gợi ý gì.

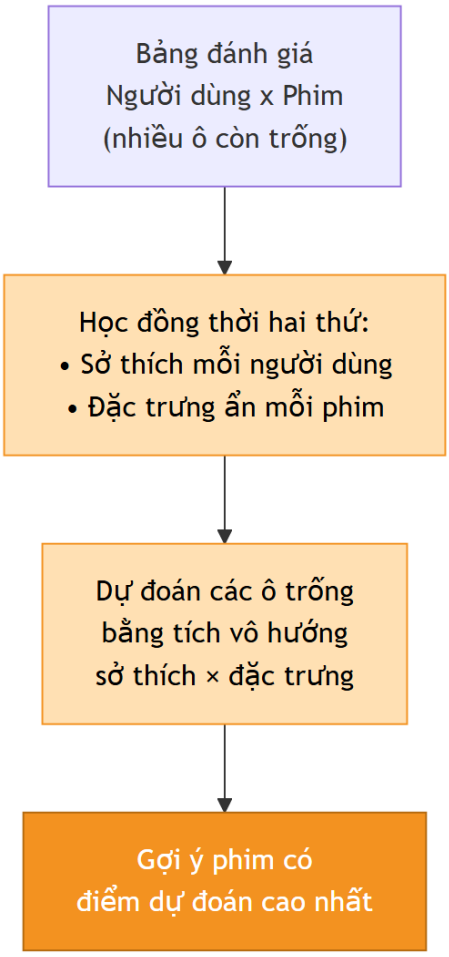
4.2.1 Lọc cộng tác: trực giác

Lọc cộng tác (collaborative filtering) dựa trên ý tưởng: nếu nhiều người có gu giống nhau cùng chấm điểm một phim, ta có thể dùng điều đó để đoán điểm cho những người khác.

Hãy bắt đầu từ trường hợp *đã có sẵn đặc trưng của phim* — ví dụ mỗi phim có hai con số: "mức độ lãng mạn" và "mức độ hành động". Khi đó, dự đoán điểm cho một người dùng giống hệt một bài **hồi quy tuyến tính nhỏ**: học một bộ "sở thích" cho người dùng đó (họ thích lãng mạn đến đâu, thích hành động đến đâu), rồi kết hợp sở thích với đặc trưng phim để ra điểm dự đoán. Mỗi người dùng có một bộ sở thích riêng, như thể ta huấn luyện một mô hình hồi quy tuyến tính riêng cho từng người.

Điều kỳ diệu của lọc cộng tác là: *ngay cả khi không có sẵn đặc trưng phim*, ta vẫn làm được. Nếu đã biết sở thích của *nhiều* người dùng và biết họ chấm một phim bao nhiêu điểm, ta có thể *đi ngược* để đoán xem đặc trưng của phim đó phải là gì thì mới khớp với các điểm số ấy. Chính vì có đánh giá từ nhiều người cho cùng một phim mà ta đoán được đặc trưng phim — điều không thể làm với hồi quy tuyến tính thông thường chỉ một người.

Gộp hai chiều lại, lọc cộng tác học *đồng thời* cả sở thích người dùng lẫn đặc trưng phim, bằng cách tối thiểu hoá sai số giữa điểm dự đoán và điểm thật trên những ô *đã có* đánh giá. Tên gọi "cộng tác" phản ánh việc dữ liệu từ nhiều người dùng cùng phối hợp giúp đoán đặc trưng phim, rồi đặc trưng đó lại giúp đoán điểm cho những người chưa xem (Hình 4.7).



Hình 4.7: Lọc cộng tác: từ bảng điểm thưa thớt, học đồng thời sở thích người dùng và đặc trưng ẩn của phim, rồi điền các ô trống để gợi ý.

Nhãn nhị phân. Nhiều ứng dụng chỉ có nhãn *thích / không thích* thay vì điểm sao (người dùng có mua, có bấm thích, có xem quá 30 giây, có nhấp vào quảng cáo hay không). Lọc cộng tác mở rộng sang trường hợp này giống như cách hồi quy tuyến tính mở rộng thành hồi quy logistic: thay vì dự đoán một con số, ta cho kết quả qua hàm sigmoid để ra *xác suất* người dùng sẽ thích, và đổi sang hàm mất mát kiểu phân loại (binary cross-entropy).

Chuẩn hoá trung bình. Một mẹo thực hành quan trọng: trừ đi điểm trung bình của mỗi phim trước khi học (gọi là chuẩn hoá trung bình theo hàng). Lợi ích rõ nhất là với *người dùng mới chưa chấm phim nào*: không chuẩn hoá, mô hình sẽ đoán họ chấm mọi phim 0 sao (vô dụng); có chuẩn hoá, mô hình đoán họ chấm bằng *điểm trung bình* của mỗi phim — hợp lý hơn nhiều. Chuẩn hoá cũng giúp thuật toán chạy nhanh hơn một chút.

Tìm món liên quan. Sau khi học xong, mỗi phim có một bộ đặc trưng ẩn. Muốn tìm các phim "tương tự" một phim cho trước, chỉ cần tìm những phim có bộ đặc trưng *gần nhất* (khoảng cách bình phương nhỏ nhất). Đây chính là cách các trang gợi ý "những món tương tự món bạn đang xem".

Hạn chế của lọc cộng tác. Hai điểm yếu đáng lưu ý. Thứ nhất là **khởi động lạnh** (cold start): phim mới chưa ai chấm, hoặc người dùng mới chưa chấm gì, thì rất khó gợi ý tốt. Thứ hai, nó *không có cách tự nhiên* để tận dụng thông tin phụ (side information) — như thể loại phim, dàn diễn viên, hay tuổi, giới tính, địa điểm của người dùng. Lọc dựa trên nội dung sẽ khắc phục những hạn chế này.

4.2.2 Lọc cộng tác với TensorFlow

Hàm mất mát của lọc cộng tác không khớp gọn vào khuôn `Dense` + `model.fit` quen thuộc, nên ta tự viết hàm chi phí và nhờ TensorFlow tính đạo hàm giúp (tính năng **Auto Diff** qua `GradientTape`). Trực giác của Auto Diff: ta chỉ cần nói cho TensorFlow cách tính chi phí, nó tự suy ra đạo hàm để tối ưu — không cần tự làm giải tích.

Hàm chi phí viết ở dạng vector hoá rất gọn: lấy tích các vector sở thích và đặc trưng, trừ đi điểm thật, chỉ tính trên các ô có đánh giá, cộng thêm số hạng chính quy hoá:

```
import tensorflow as tf

def cofi_cost_func_v(X, W, b, Y, R, lambda_):
    # X: đặc trưng phim, W,b: sở thích người dùng, Y: điểm thật, R: ô nào có đánh
    ↪ giá
    j = (tf.linalg.matmul(X, tf.transpose(W)) + b - Y) * R   # chỉ tính ô có đánh
    ↪ giá
    J = 0.5 * tf.reduce_sum(j**2) \
        + (lambda_ / 2) * (tf.reduce_sum(X**2) + tf.reduce_sum(W**2))
    return J
```

Vòng lặp huấn luyện tự viết: dùng `GradientTape` ghi lại cách tính chi phí, lấy đạo hàm theo cả ba nhóm tham số (đặc trưng phim `X`, sở thích `W`, độ chệch `b`), rồi để bộ tối ưu Adam cập nhật:

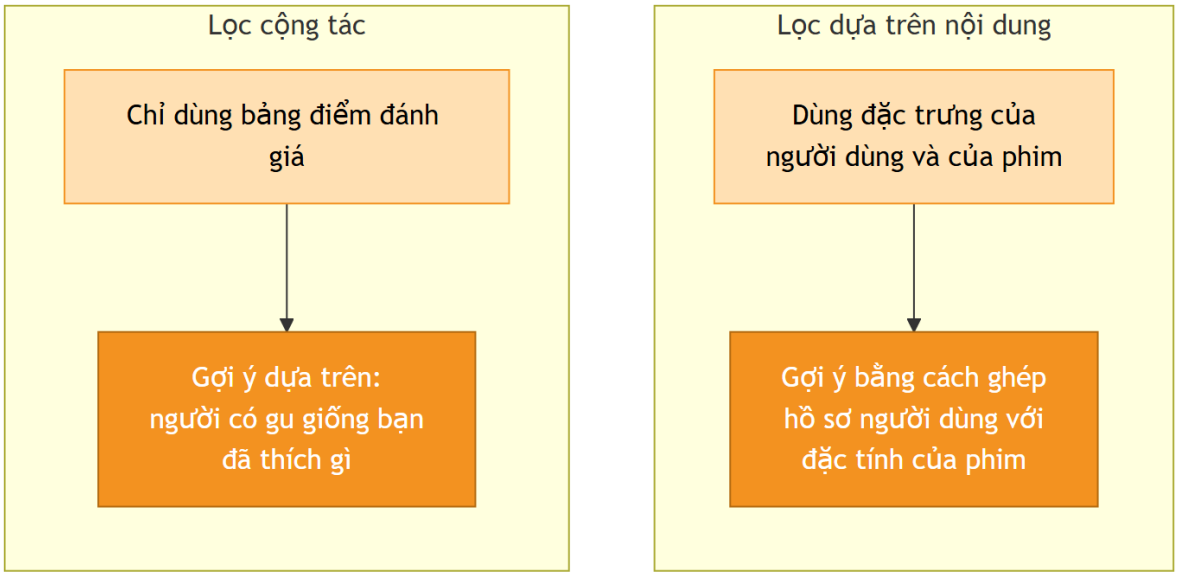
```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-1)

for iteration in range(200):
    with tf.GradientTape() as tape:
        cost = cofi_cost_func_v(X, W, b, Ynorm, R, lambda_)   # Ynorm: điểm đã
        ↪ chuẩn hoá
    grads = tape.gradient(cost, [X, W, b])          # TensorFlow tự tính đạo hàm
    optimizer.apply_gradients(zip(grads, [X, W, b]))
```

Bài thực hành dùng bộ dữ liệu phim thật MovieLens (ở quy mô rút gọn: 4778 phim, 443 người dùng). Trong lần chạy đã lưu, chi phí giảm đều từ hơn 2,3 triệu ở vòng lặp đầu xuống còn khoảng 2900 sau 180 vòng, và mô hình đưa ra các gợi ý có điểm dự đoán cao (ví dụ một phim dự đoán 4,49 sao). (Các con số lấy từ kết quả đã lưu trong notebook; ta không chạy lại.)

4.2.3 Lọc dựa trên nội dung: trực giác

Lọc dựa trên nội dung (content-based filtering) tiếp cận khác hẳn: nó gợi ý dựa trên *đặc trưng của người dùng* và *đặc trưng của phim*, tìm cặp khớp nhau tốt (Hình 4.8). Đặc trưng người dùng có thể là tuổi, giới tính, quốc gia, điểm trung bình họ chấm theo từng thể loại…; đặc trưng phim có thể là năm phát hành, thể loại, điểm trung bình nhận được…

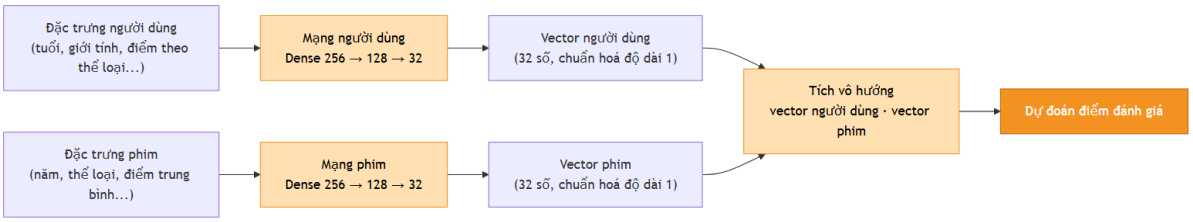


Hình 4.8: So sánh hai hướng: lọc cộng tác chỉ dùng bảng điểm; lọc dựa trên nội dung dùng đặc trưng của cả người dùng lẫn phim để ghép cặp.

Ý tưởng cốt lõi: từ đặc trưng người dùng, tính ra một *vector người dùng* tóm tắt sở thích; từ đặc trưng phim, tính ra một *vector phim* tóm tắt tính chất phim. Hai vector này phải cùng độ dài để lấy **tích vô hướng** — và tích đó chính là điểm dự đoán. Nếu vector người dùng nói "thích lãng mạn nhiều, hành động ít" và vector phim nói "lãng mạn nhiều, hành động ít", tích vô hướng sẽ lớn, nghĩa là khả năng thích cao.

4.2.4 Lọc nội dung bằng học sâu

Cách hiện đại để tính hai vector đó là dùng *hai mạng nơ-ron* — một kiến trúc gọi là "hai tháp" (two-tower). **Mạng người dùng** nhận đặc trưng người dùng và cho ra vector người dùng; **mạng phim** nhận đặc trưng phim và cho ra vector phim. Hai mạng có thể khác nhau về số lớp và số nơ-ron, nhưng *lớp đầu ra phải cùng kích thước* (ví dụ cùng 32 số) để lấy được tích vô hướng. Dự đoán cuối cùng là tích vô hướng của hai vector (Hình 4.9).



Hình 4.9: Kiến trúc hai tháp: mạng người dùng và mạng phim cho ra hai vector cùng độ dài, tích vô hướng của chúng là điểm dự đoán.

Điểm hay: không có hai quá trình huấn luyện tách rời. Cả hai mạng được huấn luyện *cùng lúc* bằng một hàm chi phí duy nhất — sai số giữa tích vô hướng dự đoán và điểm thật. Đây cũng là ví dụ đẹp cho việc ghép nhiều mạng nơ-ron lại thành một hệ lớn hơn, mạnh hơn.

Cài đặt trong Keras khá quen thuộc: mỗi tháp là một `Sequential` vài lớp `Dense`, đầu ra 32 số; sau đó chuẩn hoá độ dài vector về 1 (giúp thuật toán chạy tốt hơn) rồi lấy tích vô hướng bằng lớp `Dot`:

```
import tensorflow as tf
from tensorflow.keras.layers import Dense

user_NN = tf.keras.Sequential([        # tháp người dùng
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(32),
])
item_NN = tf.keras.Sequential([        # tháp phim
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(32),
])

vu = user_NN(input_user)
vu = tf.linalg.l2_normalize(vu, axis=1)   # chuẩn hoá độ dài về 1
vm = item_NN(input_item)
vm = tf.linalg.l2_normalize(vm, axis=1)

output = tf.keras.layers.Dot(axes=1)([vu, vm])   # tích vô hướng -> điểm dự đoán
model = tf.keras.Model([input_user, input_item], output)
model.compile(loss=tf.keras.losses.MeanSquaredError(),
              optimizer=tf.keras.optimizers.Adam(learning_rate=0.01))
```

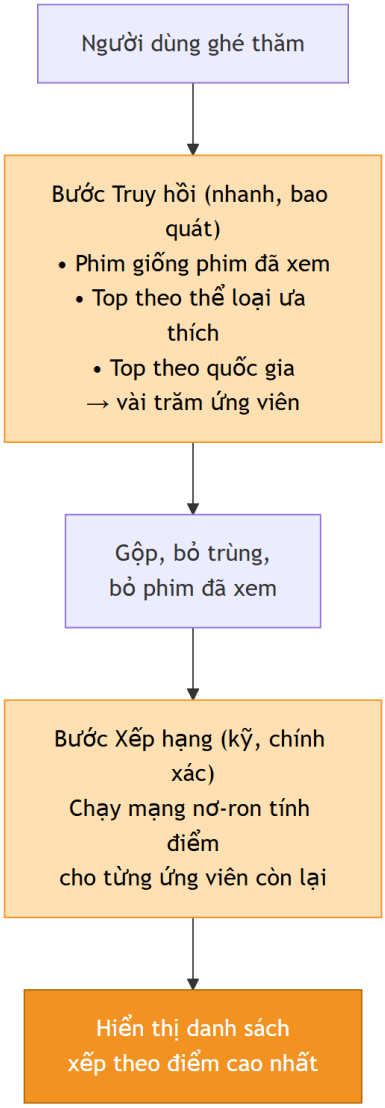
Bài thực hành dùng MovieLens rút gọn (847 phim, 397 người dùng, 14 thể loại). Trong lần chạy đã lưu, mất mát huấn luyện giảm từ khoảng 0,12 xuống 0,07 sau 30 vòng, và mất mát trên tập kiểm tra khoảng 0,08 — cho thấy mô hình tổng quát hoá tốt. Việc tìm phim tương tự cũng làm y như lọc cộng tác: so khoảng cách giữa các vector phim, và có thể tính trước toàn bộ qua đêm để tra cứu tức thì. (Số liệu lấy từ kết quả đã lưu.)

4.2.5 Mở rộng quy mô: truy hồi và xếp hạng

Một hệ gợi ý thực tế có thể phải chọn vài món từ *hàng triệu* ứng viên. Chạy mạng nơ-ron cho từng ấy món mỗi lần người dùng ghé thăm là bất khả thi về mặt tính toán. Giải pháp là chia làm hai bước (Hình 4.10):

- **Truy hồi** (retrieval): tạo nhanh một danh sách lớn các ứng viên hợp lý — ví dụ các phim giống phim người dùng vừa xem, top phim theo thể loại ưa thích, top phim theo quốc gia. Bước này cốt *bao quát*, chấp nhận lẫn vào vài món không hợp.

- **Xếp hạng** (ranking): chỉ với vài trăm ứng viên còn lại, chạy mạng nơ-ron tính điểm cẩn thận cho từng món rồi sắp xếp. Vì vector phim có thể tính trước, bước này rất nhanh.



Hình 4.10: Hai bước giúp hệ gợi ý vừa nhanh vừa chính xác trên kho hàng triệu món: truy hồi bao quát trước, xếp hạng tinh sau.

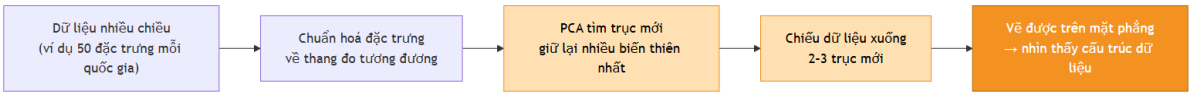
Khía cạnh đạo đức. Hệ gợi ý rất mạnh nhưng cũng có thể gây hại. Việc tối ưu cho lợi nhuận, cho thời gian xem, hay cho lượt nhấp quảng cáo đôi khi khuếch đại nội dung độc hại hoặc các ngành khai thác người dùng. Khi xây hệ gợi ý, hãy cân nhắc không chỉ lợi ích mà cả tác hại tiềm tàng, ưu tiên minh bạch với người dùng về tiêu chí gợi ý, và chỉ xây những thứ thực sự làm xã hội tốt hơn.

4.2.6 Giảm chiều dữ liệu với PCA

Phân tích thành phần chính (PCA — principal component analysis) là một thuật toán học không giám sát dùng chủ yếu để *trực quan hoá*. Khi dữ liệu có rất nhiều đặc trưng (50, 1000, hay hơn), ta không thể vẽ nó ra. PCA giảm số đặc trưng xuống còn hai hoặc ba để vẽ được trên mặt phẳng, mà vẫn giữ phần lớn thông tin.

Trực giác: PCA tìm một *trục mới* sao cho khi chiếu dữ liệu lên trục đó, các điểm còn *trải rộng* nhất có thể — tức giữ lại nhiều biến thiên nhất. Ví dụ với dữ liệu kích thước xe (chiều dài, chiều cao), thay vì giữ cả hai, PCA có thể tìm một trục "kích cỡ chung" nắm bắt phần lớn sự khác biệt chỉ bằng một con số. Trục mới quan trọng nhất gọi là *thành phần chính*; nếu lấy thêm trục nữa, trục sau luôn vuông góc với trục trước (Hình 4.11).

PCA *khác* hồi quy tuyến tính: hồi quy có một biến mục tiêu y được ưu ái và đo khoảng cách theo chiều dọc tới y, còn PCA không có nhãn, đối xử mọi đặc trưng như nhau, và tìm trục giữ lại biến thiên nhiều nhất.



Hình 4.11: Luồng PCA: chuẩn hoá, tìm trục giữ nhiều biến thiên nhất, chiếu dữ liệu xuống 2–3 trục để vẽ và hiểu dữ liệu.

Dùng PCA với scikit-learn gồm ba bước. Bước `fit` tìm trục (tự động trừ trung bình). Thuộc tính `explained_variance_ratio_` cho biết mỗi trục giữ được bao nhiêu phần biến thiên. Cuối cùng `transform` chiếu dữ liệu xuống các trục mới:

```
from sklearn.decomposition import PCA

pca_2 = PCA(n_components=2)          # giảm xuống 2 trục để vẽ
pca_2.fit(X)
print(pca_2.explained_variance_ratio_)   # ví dụ: [0.992, 0.008] -> trục 1 giữ
↪  99,2%
X_trans = pca_2.transform(X)        # mỗi mẫu còn 2 số, vẽ được trên mặt phẳng
```

Trong lab, với một ví dụ nhỏ, trục chính thứ nhất giữ được 99,2% biến thiên — nghĩa là chỉ cần một con số đã nắm gần trọn thông tin. Ngày nay PCA chủ yếu dùng để trực quan hoá; các ứng dụng cũ như nén dữ liệu hay tăng tốc huấn luyện giờ ít gặp hơn vì lưu trữ rẻ và học sâu thường xử lý tốt dữ liệu nhiều chiều mà không cần giảm chiều trước.

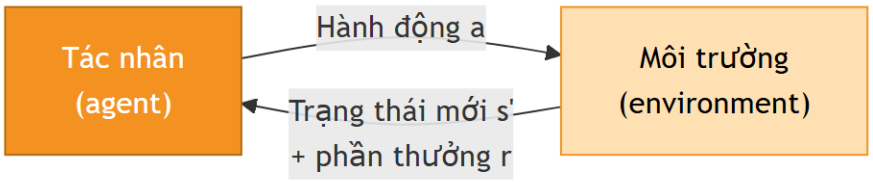
4.3 Học tăng cường

Học tăng cường (reinforcement learning) là trụ cột thứ ba của học máy, bên cạnh học có giám sát và không giám sát. Ý tưởng cốt lõi: thay vì chỉ cho thuật toán biết hành động đúng ở mỗi tình huống (rất khó, đôi khi mơ hồ), ta chỉ cần định nghĩa một **hàm phần thưởng** nói cho nó biết khi nào làm tốt, khi nào làm tệ — rồi để thuật toán tự tìm cách hành động tốt. Giống như dạy chó: không ra lệnh từng động tác, mà khen "giỏi" khi làm đúng và chê "hư" khi làm sai.

Cách tiếp cận này mạnh vì ta nói *cái gì* cần đạt chứ không phải *làm thế nào*. Nó đã được dùng để điều khiển robot (trực thăng tự lượn, chó robot vượt chướng ngại), tối ưu nhà máy, giao dịch tài chính, và chơi game. Trong chương này, đích đến là huấn luyện một tác nhân đáp con tàu đồ bộ Mặt Trăng mô phỏng xuống an toàn.

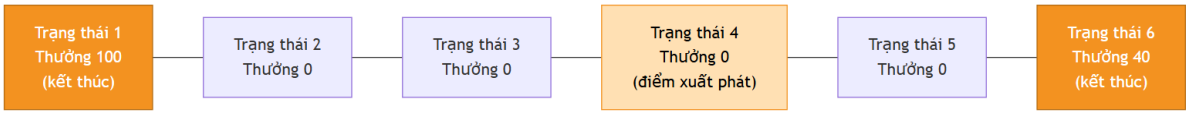
4.3.1 Khung bài toán: trạng thái, hành động, phần thưởng

Vòng lặp cơ bản của học tăng cường: một **tác nhân** (agent) quan sát **trạng thái** hiện tại của **môi trường**, chọn một **hành động**, rồi môi trường trả lại một trạng thái mới và một **phần thưởng** (Hình 4.12).



Hình 4.12: Vòng lặp tác nhân–môi trường: tác nhân chọn hành động, môi trường trả về trạng thái mới và phần thưởng.

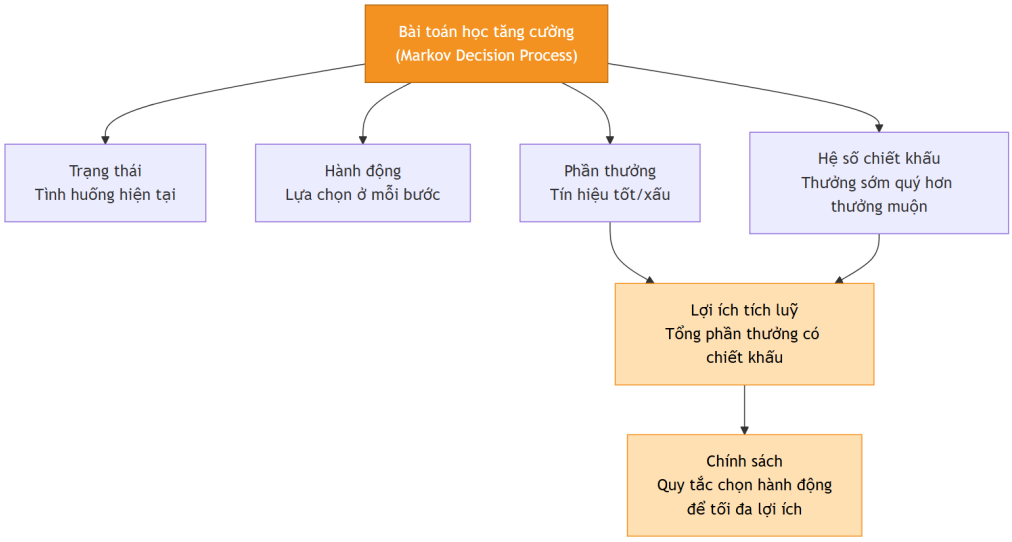
Để làm quen, ta dùng ví dụ đơn giản lấy cảm hứng từ xe tự hành sao Hoả (Mars rover). Xe có thể ở một trong sáu vị trí (sáu *trạng thái*), xuất phát ở trạng thái 4. Hai trạng thái đầu mút có phần thưởng: trạng thái 1 bên trái thưởng 100 (nơi khoa học thú vị nhất), trạng thái 6 bên phải thưởng 40; các trạng thái giữa thưởng 0. Mỗi bước xe chọn *đi trái* hoặc *đi phải*. Hai trạng thái đầu mút là **trạng thái kết thúc** (terminal): tới đó nhận thưởng rồi dừng (Hình 4.13).



Hình 4.13: Ví dụ xe tự hành sao Hoả: sáu trạng thái, hai trạng thái kết thúc có phần thưởng 100 và 40, xuất phát ở trạng thái 4.

Lợi ích tích luỹ và hệ số chiết khấu. Làm sao biết một chuỗi phần thưởng là tốt hay xấu? Ta dùng khái niệm **lợi ích tích luỹ** (return): tổng các phần thưởng, nhưng *phần thưởng đến muộn bị giảm giá trị* qua một **hệ số chiết khấu** (một số nhỏ hơn 1, ví dụ 0,9). Trực giác giống chuyện nhặt tiền: tờ 5 đô ngay dưới chân có khi hấp dẫn hơn tờ 10 đô phải đi bộ nửa tiếng mới tới. Hệ số chiết khấu khiến thuật toán "hơi thiếu kiên nhẫn" — thưởng sớm được tính trọn, thưởng càng muộn càng bị nhân với hệ số nhỏ dần. (Trong tài chính, hệ số này chính là lãi suất, giá trị thời gian của tiền.) Với phần thưởng âm, chiết khấu lại khuyến khích hệ thống *đẩy* chúng ra càng xa tương lai càng tốt.

Chính sách. Mục tiêu của học tăng cường là tìm một **chính sách** (policy) — một quy tắc nhận vào trạng thái và cho ra hành động nên làm — sao cho tối đa hoá lợi ích tích luỹ. Với xe sao Hoả, một chính sách tốt có thể là "đi trái ở trạng thái 2, 3, 4 nhưng đi phải ở trạng thái 5" (vì ở đó đã gần phần thưởng bên phải).



Hình 4.14: Các khái niệm cốt lõi của một bài toán học tăng cường (Markov Decision Process) và cách chúng nối với nhau.

Khung này có tên là **Markov Decision Process** (MDP). Chữ "Markov" nghĩa là tương lai chỉ phụ thuộc trạng thái hiện tại, không phụ thuộc đã đi đường nào để tới đó (Hình 4.14). Cùng một khung áp dụng được cho trực thăng tự hành (trạng thái là vị trí, hướng, tốc độ; hành động là cách đẩy cần điều khiển) hay chơi cờ (trạng thái là thế cờ; thưởng +1 nếu thắng, −1 nếu thua).

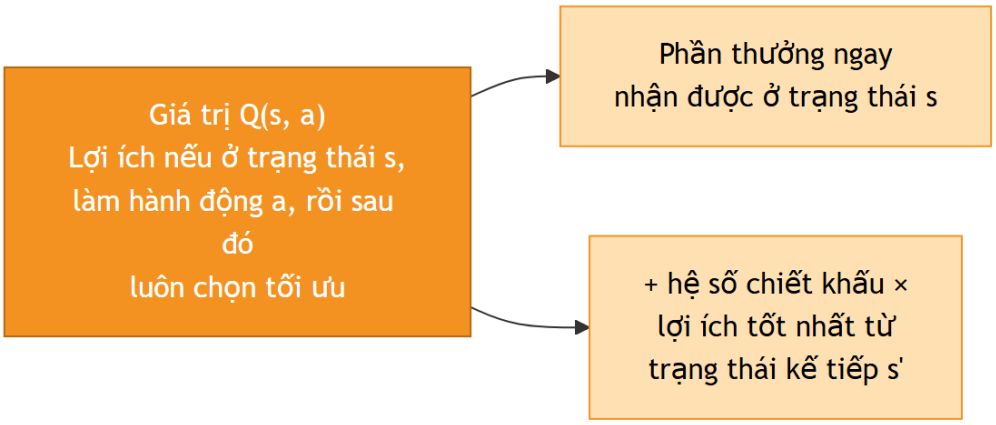
4.3.2 Hàm giá trị hành động và phương trình Bellman

Đại lượng then chốt mà thuật toán học tăng cường cố tính là **hàm giá trị hành động**, thường ký hiệu bằng chữ Q, nên cũng gọi là **hàm Q**. Giá trị Q của một cặp (trạng thái, hành động) là: *lợi ích tích luỹ nếu bắt đầu ở trạng thái đó, làm hành động đó một lần, rồi sau đó luôn hành động tối ưu*.

Vì sao hàm Q hữu ích? Vì nếu biết giá trị Q cho mọi hành động ở một trạng thái, chính sách tốt nhất đơn giản là: *chọn hành động có giá trị Q lớn nhất*. Lợi ích tốt nhất có thể đạt từ một trạng thái cũng chính là giá trị Q lớn nhất tại đó.

Phương trình Bellman là công thức quan trọng nhất để tính hàm Q. Nó nói rằng giá trị Q của một cặp (trạng thái, hành động) có thể tách làm hai phần (Hình 4.15):

- **Phần thưởng ngay** nhận được tại trạng thái hiện tại (immediate reward).
- Cộng với **hệ số chiết khấu nhân lợi ích tốt nhất từ trạng thái kế tiếp** — tức giá trị Q lớn nhất có thể đạt được sau khi đã bước sang trạng thái mới.



Hình 4.15: Phương trình Bellman tách giá trị Q thành phần thưởng ngay cộng lợi ích tốt nhất (đã chiết khấu) từ trạng thái kế tiếp.

Trực giác: tổng lợi ích từ hiện tại bằng cái nhận được ngay bây giờ, cộng với tất cả cái sẽ nhận được từ đây về sau (chính là lợi ích tốt nhất khởi đi từ trạng thái kế tiếp, bị chiết khấu một bậc). Bài thực hành tuỳ chọn cho phép chỉnh phần thưởng, hệ số chiết khấu và quan sát hàm Q cùng chính sách tối ưu thay đổi ra sao — ví dụ hệ số chiết khấu nhỏ khiến xe "thiếu kiên nhẫn" và chọn phần thưởng gần dù nhỏ hơn.

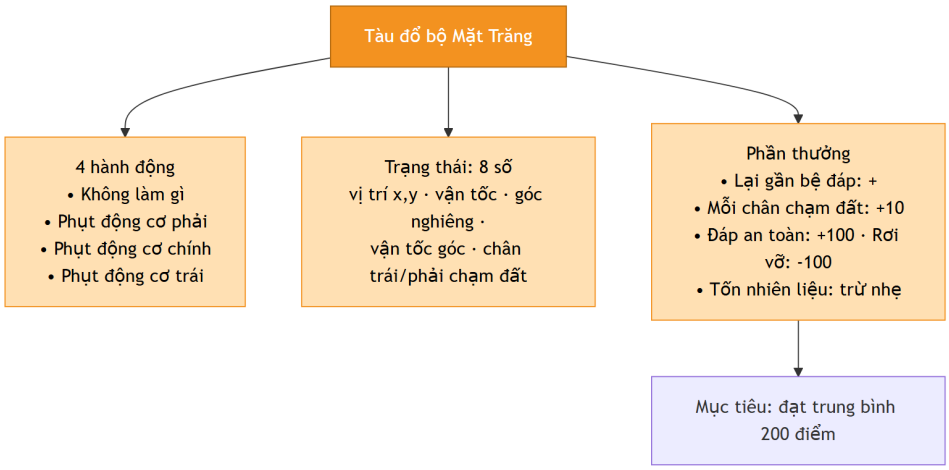
Môi trường ngẫu nhiên. Trong thực tế, hành động không phải lúc nào cũng cho kết quả như ý (robot trượt chân, gió thổi lệch). Khi đó trạng thái kế tiếp mang tính ngẫu nhiên, và mục tiêu đổi thành tối đa hoá *lợi ích tích luỹ trung bình* (kỳ vọng) — tức giá trị trung bình nếu thử chính sách rất nhiều lần. Phương trình Bellman khi đó thêm một phép lấy trung bình ở phần tương lai. Xe càng ít kiểm soát được (xác suất đi lệch càng cao) thì các giá trị Q càng giảm.

4.3.3 Trạng thái liên tục và tàu đổ bộ Mặt Trăng

Nhiều bài toán điều khiển có **trạng thái liên tục** — trạng thái không phải một trong vài giá trị rời rạc mà là một *vector số* có thể nhận vô vàn giá trị. Ví dụ một chiếc xe tự lái có trạng thái gồm vị trí, hướng, và các vận tốc; một trực thăng tự hành cần tới 12 số (ba toạ độ vị trí, ba góc xoay, và sáu vận tốc tương ứng).

Bài thực hành là **tàu đổ bộ Mặt Trăng** (lunar lander) — một trò chơi mô phỏng kinh điển trong giới nghiên cứu học tăng cường (Hình 4.16):

- **Hành động** (4 lựa chọn): không làm gì, phụt động cơ trái, phụt động cơ chính, phụt động cơ phải.
- **Trạng thái** (8 số): toạ độ ngang/dọc, vận tốc ngang/dọc, góc nghiêng, vận tốc góc, và hai giá trị nhị phân cho biết chân trái và chân phải đã chạm đất chưa.
- **Phần thưởng**: lại gần bệ đáp thì cộng, ra xa thì trừ; mỗi chân chạm đất +10; đáp an toàn +100, rơi vỡ −100; mỗi lần phụt động cơ bị trừ nhẹ để khỏi phí nhiên liệu.



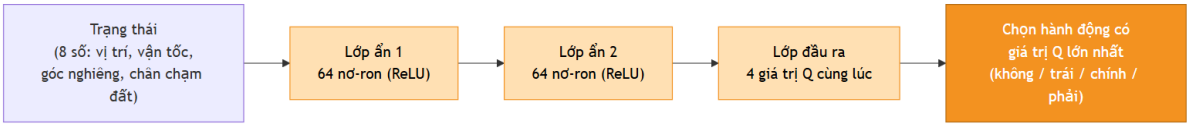
Hình 4.16: Bài toán tàu đổ bộ Mặt Trăng: 4 hành động, trạng thái 8 số, và một hàm phần thưởng được thiết kế kỹ để khuyến khích đáp an toàn.

Hàm phần thưởng này khá phức tạp — người thiết kế đã cân nhắc kỹ hành vi mong muốn và mã hoá vào phần thưởng. Đây là bài học chung: định nghĩa phần thưởng thường cần suy nghĩ, nhưng vẫn dễ hơn nhiều so với việc chỉ ra hành động đúng ở từng trạng thái. Môi trường được coi là *giải xong* nếu đạt trung bình 200 điểm. Hệ số chiết khấu dùng ở đây khá gần 1 (0,985), tức tàu khá "kiên nhẫn".

4.3.4 Deep Q-Network: học hàm Q bằng mạng nơ-ron

Với trạng thái liên tục, ta không thể lập bảng Q cho mọi trạng thái. Thay vào đó, ta huấn luyện một *mạng nơ-ron* để xấp xỉ hàm Q — gọi là **Deep Q-Network** (DQN).

Kiến trúc hiệu quả nhất (dùng trong bài thực hành): mạng nhận 8 số trạng thái, qua hai lớp ẩn 64 nơ-ron ReLU, và cho ra *bốn* giá trị Q cùng lúc — mỗi giá trị cho một hành động. Nhờ vậy chỉ cần chạy mạng một lần là có đủ bốn giá trị để chọn hành động Q lớn nhất (Hình 4.17):



Hình 4.17: Mạng Q cho tàu đổ bộ: vào 8 số trạng thái, ra 4 giá trị Q cùng lúc; chọn hành động có Q lớn nhất.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam

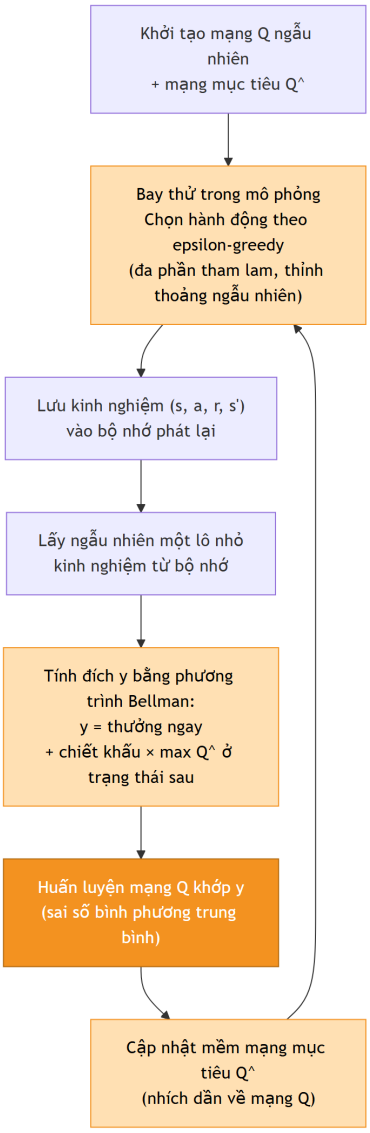
q_network = Sequential([
    Input(shape=state_size),                        # 8 số trạng thái
    Dense(units=64, activation='relu'),
    Dense(units=64, activation='relu'),
    Dense(units=num_actions, activation='linear'), # 4 giá trị Q
])
# Mạng mục tiêu Q^ có kiến trúc y hệt, dùng để tạo đích huấn luyện ổn định
target_q_network = Sequential([
    Input(shape=state_size),
    Dense(units=64, activation='relu'),
    Dense(units=64, activation='relu'),
    Dense(units=num_actions, activation='linear'),
```

```
])
optimizer = Adam(learning_rate=ALPHA)   # ALPHA = 1e-3
```

Biến bài toán RL thành bài học có giám sát. Mẹo của DQN là tạo một tập huấn luyện kiểu (đầu vào, đích) từ kinh nghiệm, rồi huấn luyện mạng như học có giám sát. Mỗi lần tác nhân trải nghiệm một bước, ta lưu bộ bốn (trạng thái, hành động, phần thưởng, trạng thái kế tiếp). Đầu vào là (trạng thái, hành động); *đích* được tính bằng vế phải phương trình Bellman: phần thưởng ngay cộng chiết khấu nhân giá trị Q lớn nhất ở trạng thái kế tiếp.

Thuật toán đầy đủ gọi là **DQN với phát lại kinh nghiệm** (experience replay), gồm vài cải tiến quan trọng (Hình 4.18):

- **Bộ nhớ phát lại** (replay buffer): lưu một số lượng kinh nghiệm gần nhất (ở đây 100000), rồi mỗi lần huấn luyện lấy ngẫu nhiên một *lô nhỏ* (64 mẫu). Lấy mẫu ngẫu nhiên phá vỡ tương quan giữa các bước liên tiếp, giúp học ổn định hơn.
- **Mạng mục tiêu** (target network): dùng một bản sao mạng để tạo đích Bellman, tránh "đích di động" gây dao động. Mạng mục tiêu được cập nhật *mềm* — nhích dần về mạng Q một chút mỗi lần, thay vì sao chép đột ngột.
- **Chính sách epsilon-greedy**: phần lớn thời gian chọn hành động Q lớn nhất (khai thác), nhưng thỉnh thoảng chọn ngẫu nhiên (thăm dò) để không bỏ sót hành động tốt chưa từng thử. Thường bắt đầu epsilon cao (thăm dò nhiều) rồi giảm dần.



Hình 4.18: Vòng lặp DQN với phát lại kinh nghiệm: bay thử, lưu kinh nghiệm, lấy lô nhỏ, tính đích Bellman, huấn luyện mạng Q, cập nhật mềm mạng mục tiêu.

Phần cốt lõi là tính đích rồi so với dự đoán bằng sai số bình phương trung bình. Mẹo khéo: nhân với `(1 - done_vals)` để ở trạng thái kết thúc thì đích chỉ còn phần thưởng ngay (không có tương lai):

```
import tensorflow as tf
from tensorflow.keras.losses import MSE

def compute_loss(experiences, gamma, q_network, target_q_network):
    states, actions, rewards, next_states, done_vals = experiences
    # Giá trị Q lớn nhất ở trạng thái kế tiếp (do mạng mục tiêu đưa ra)
    max_qsa = tf.reduce_max(target_q_network(next_states), axis=-1)
    # Đích Bellman: nếu kết thúc thì y = thưởng; ngược lại y = thưởng + chiết
    ↪  khấu * max Q^
    y_targets = rewards + (gamma * max_qsa * (1 - done_vals))
    # Giá trị Q mạng đang dự đoán cho đúng hành động đã làm
    q_values = q_network(states)
    q_values = tf.gather_nd(q_values, tf.stack(
        [tf.range(q_values.shape[0]), tf.cast(actions, tf.int32)], axis=1))
```

```
    return MSE(y_targets, q_values)
```

Bước cập nhật trọng số dùng `GradientTape` để lấy đạo hàm, cập nhật mạng Q bằng Adam, rồi cập nhật mềm mạng mục tiêu:

```
@tf.function
def agent_learn(experiences, gamma):
    with tf.GradientTape() as tape:
        loss = compute_loss(experiences, gamma, q_network, target_q_network)
    gradients = tape.gradient(loss, q_network.trainable_variables)
    optimizer.apply_gradients(zip(gradients, q_network.trainable_variables))
    # Cập nhật mềm: mạng mục tiêu nhích dần về mạng Q (TAU rất nhỏ, ví dụ 1e-3)
    update_target_network(q_network, target_q_network)
```

Vòng huấn luyện ngoài làm đúng như sơ đồ: ở mỗi bước, chọn hành động theo epsilon-greedy, thực thi để nhận phần thưởng và trạng thái mới, lưu kinh nghiệm, và cứ vài bước lại lấy một lô để huấn luyện. Môi trường coi như giải xong khi đạt trung bình 200 điểm trong 100 tập gần nhất.

Trong lần chạy đã lưu của notebook, điểm trung bình cải thiện dần qua các tập: khoảng -150 ở tập 100, -106 ở tập 200, -77 ở tập 300 — cho thấy tác nhân đang học đi đúng hướng. (Phần ghi lại của notebook dừng ở tập 306 với trung bình khoảng -73 và không lưu lại thời điểm giải xong hay tổng thời gian chạy, nên ở đây ta chỉ mô tả xu hướng chứ không nêu con số mà notebook không có.) Lưu ý thực chiến từ bài học: thuật toán học tăng cường "đỏng đảnh" hơn học có giám sát — chọn sai siêu tham số có thể khiến thời gian học lâu gấp hàng chục, hàng trăm lần, nên cần kiên nhẫn tinh chỉnh.

4.3.5 Học tăng cường trong bức tranh lớn

Dù hấp dẫn và nhiều tiềm năng, học tăng cường hiện vẫn ít ứng dụng thương mại hơn hẳn học có giám sát và không giám sát. Nhiều kết quả ấn tượng đến từ *mô phỏng* — và chuyển từ mô phỏng sang robot thật thường khó hơn nhiều so với kỳ vọng. Lời khuyên: khi xây một ứng dụng thực, khả năng học có giám sát hoặc không giám sát là công cụ phù hợp cao hơn nhiều. Dù vậy, học tăng cường vẫn là một trụ cột quan trọng với nhiều nghiên cứu sôi động, và hiểu nó giúp bạn trở thành một kỹ sư học máy toàn diện hơn.